

INTERCONNEXION DE RESEAUX ET INTERNET

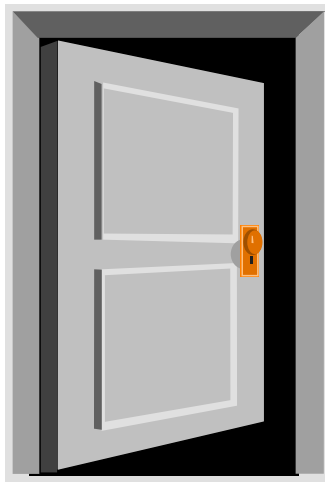
UF « Temps réel et Réseaux »

SUPPORT DE COURS

Christophe CHASSOT

INSA Toulouse / DGEI - LAAS CNRS / SARA

e-mail : chassot@insa-toulouse.fr / chassot@laas.fr



INSA Toulouse 2023/2024 – 4AE/SE
Département Génie Electrique et Informatique

PLAN DU COURS

I. Introduction à l'interconnexion de réseaux.....	4
1. Définition et motivation.....	5
1.1. Qu'est ce qu'une IRL ?	5
1.2. Motivations	6
2. Technologies d'interconnexion	9
II. Interconnexion par répéteur et par pont.....	10
1. Interconnexion par répéteur ou par HUB.....	11
1.1. Niveau opératoire et fonctions	11
1.2. Limites des répéteurs et des HUB.....	13
2. Interconnexion par pont.....	15
2.1. Niveau opératoire et fonctions	15
2.2. Pont transparent et switch Ethernet.....	16
2.3. Problème des boucles et algo. du <i>Spanning Tree (non étudié)</i>	23
III. Les réseaux locaux virtuels (VLAN).....	30
1. Le concept de VLAN.....	31
2. Configuration d'un VLAN.....	32
IV. Interconnexion par routeur IP.....	38
1. Introduction.....	39
2. IP : Internet Protocol.....	41
2.1. Service et fonctions	42
2.2. Format des paquets IP	44
2.3. Fragmentation et réassemblage.....	46
2.4. Adressage et algorithme de routage des paquets IP	49
2.5. Le routage au sein de sous-réseaux IP (<i>Subnetting</i>)	66
3. Protocoles de routage (courte introduction).....	78
3.1. Introduction.....	78
3.2. Le protocole de routage intra domaine RIP	82
4. ARP, RARP/DHCP, Proxy ARP et ICMP	91
4.1. Le protocole ARP	91
4.2. Les protocoles RARP et DHCP	98
4.3. Proxy ARP (<i>hors du cours de cette année</i>).....	100
4.4. Le protocole ICMP	105
V. VPN, Proxy Applicatif et NAT, Multicast IP	109
1. Introduction.....	110
2. Le concept de réseau privé virtuel (VPN).....	113
2.1. Concept initial	113

2.2. Principe de mise en œuvre	114
3. Proxy applicatif et translation d'adresse réseau (NAT).....	116
3.1. Position du problème	116
3.2. Proxy applicatif : principes de base	117
3.3. NAT : principes de base.....	121
3.4. NAT multi adresses et NAT avec translation de port	125
3.5. <i>Port forwarding</i> : principes de base	128
4. Couplage du VPN et du NAT	129
5. Gestion du multicast IP dans l'Internet (bases).....	131
5.1. Introduction.....	131
5.2. IP Multicast.....	132
VI. Les protocoles de Transport UDP et TCP	142
1. Introduction.....	143
1.1. Liaison vs Transport	143
1.2. Services d'une couche Transport	148
1.3. Fonctions d'une couche Transport.....	151
2. UDP : User Datagram Protocol [RFC 768]	154
2.1. Format des PDU.....	154
2.2. Mécanismes protocolaires.....	155
3. TCP : Transmission Control Protocol [RFC 793, 1323]	156
3.1. Format des PDU.....	156
3.2. Phase d'établissement de connexion.....	159
3.3. Phase de transfert des données.....	163
3.4. Phase de libération de connexion.....	189

* * *

*

I. Introduction à l'interconnexion de réseaux

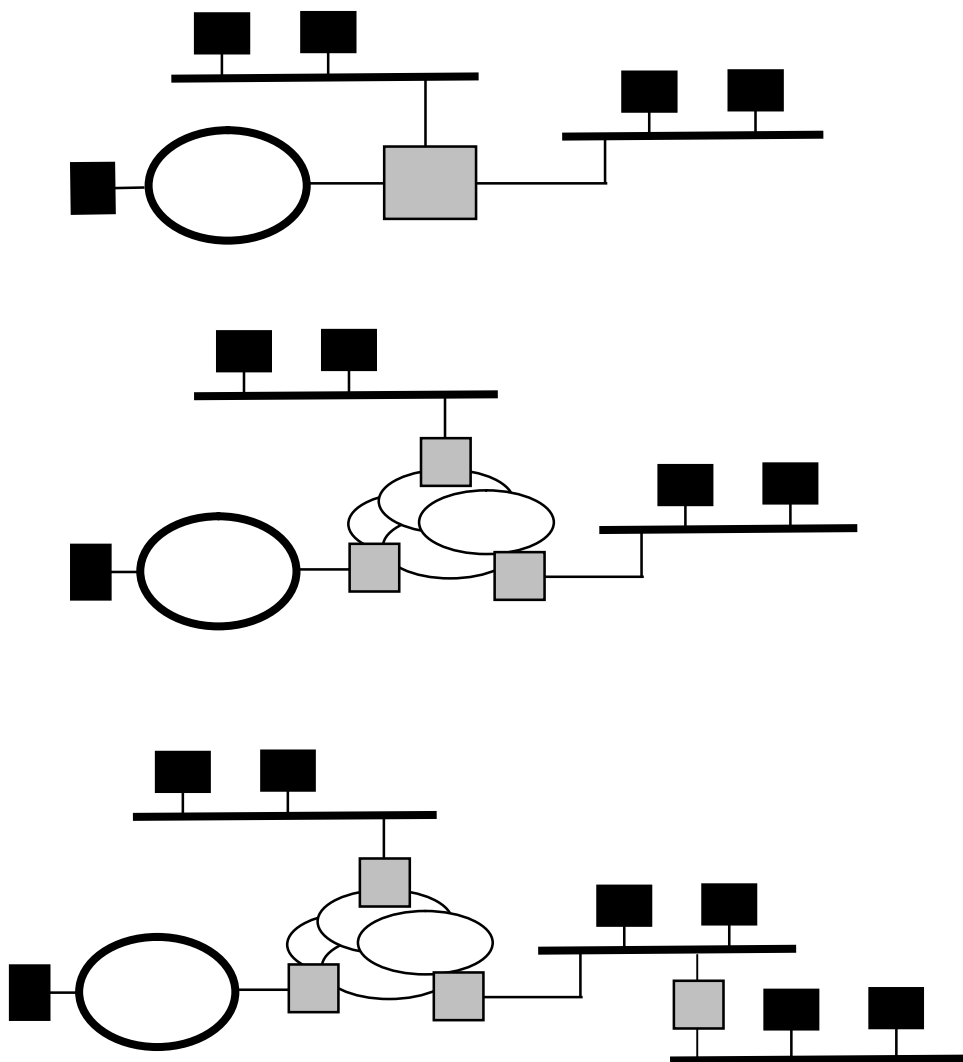
1. Définition et motivations

2. Technologies d'interconnexion

1. Définition et motivation

1.1. Qu'est ce qu'une IRL ?

- **IRL** = Interconnexion de réseaux locaux (LAN) :
 - à topologie et / ou architecture éventuellement $\neq$
 - au sein d'un même site ou entre sites $\neq$
 - au moyen d'un ou plusieurs équipements d'interconnexion (EI)
- **Ex**



1.2. Motivations

▪ 2 types de motivations

– « Naturelles »

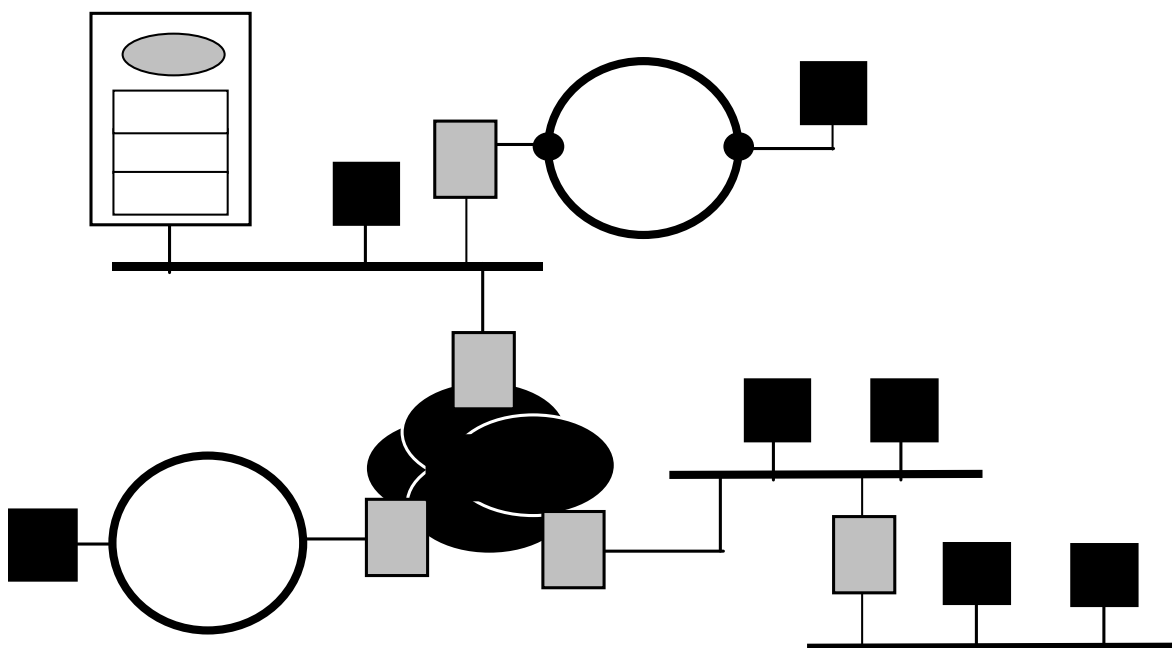
- Permettre la communication entre programmes applicatifs s'exécutant sur des machines n'appartenant pas au même LAN

– 2 paramètres à considérer :

- l'**architecture** des réseaux à interconnecter
- la **distance** séparant les réseaux à interconnecter

– « Stratégiques »

- Distinguer plusieurs domaines de collision
- Sécuriser certains endroits du réseau
- ...

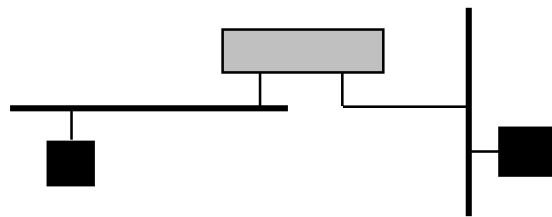


▪ Le paramètre « distance »

– Indépendamment de leur architecture, l'interconnexion de 2 réseaux locaux LAN peut être envisagée :

- en raccordant les LAN à un même EI

→ cas où la **distance** séparant les LAN est **petite**



- en raccordant les LAN à des EI différents

– eux-mêmes connectés l'un à l'autre par :

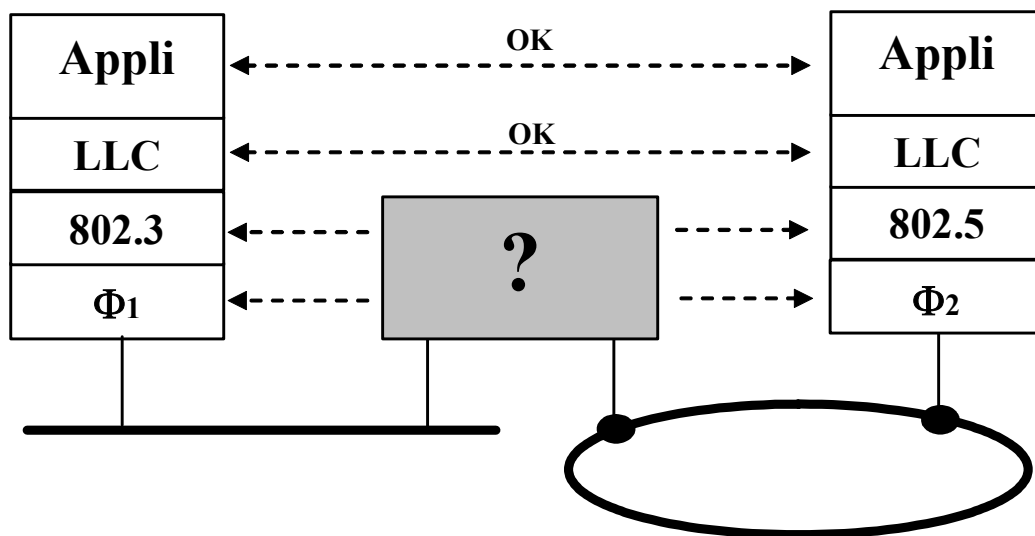
- une liaison spécialisée
- un réseau commuté (ATM, ...)
- un MAN (feu FDDI, ...)
- plusieurs de ces réseaux dans le cadre d'une interconnexion par routeur IP ($\Leftrightarrow$ Internet)

→ cas où la **distance** séparant les LAN est **grande**



▪ Le paramètre « architecture »

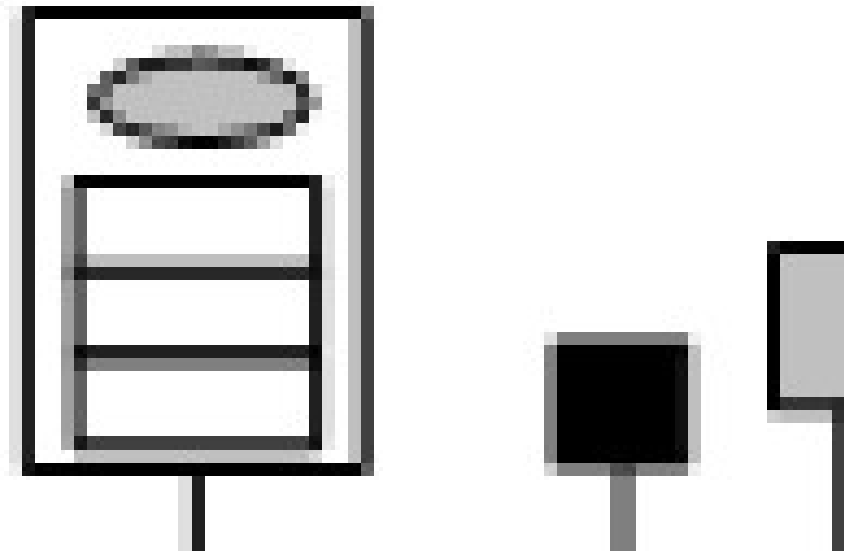
- Indépendamment de la distance séparant deux LAN
 - les fonctionnalités d'un EI dépendent de l'**architecture** déployée sur les machines de chaque LAN
 - et de leur degré d'hétérogénéité
- Exemple :
 - Interconnexion d'un réseau Ethernet 802.3 avec un réseau Token Ring 802.5 (NB : réseaux à titre d'exemple)



2. Technologies d'interconnexion

▪ 3 principaux types de technologies « classiques »

- les *répéteurs*
- les *ponts*
- les *routeurs*



▪ Evolutions (surtout dans la technologie Ethernet)

- HUB
 - ⇔ répéteur multiports + administration SNMP
- Commutateur de trames ⇔ switch Ethernet
 - ⇔ pont « transparent » multi ports
 - incluant d'autres fonctions : administration SNMP, VLAN, ...

II. Interconnexion par répéteur et par pont

1. Interconnexion par répéteur ou par HUB

2. Interconnexion par pont

1. Interconnexion par répéteur ou par HUB

1.1. Niveau opératoire et fonctions

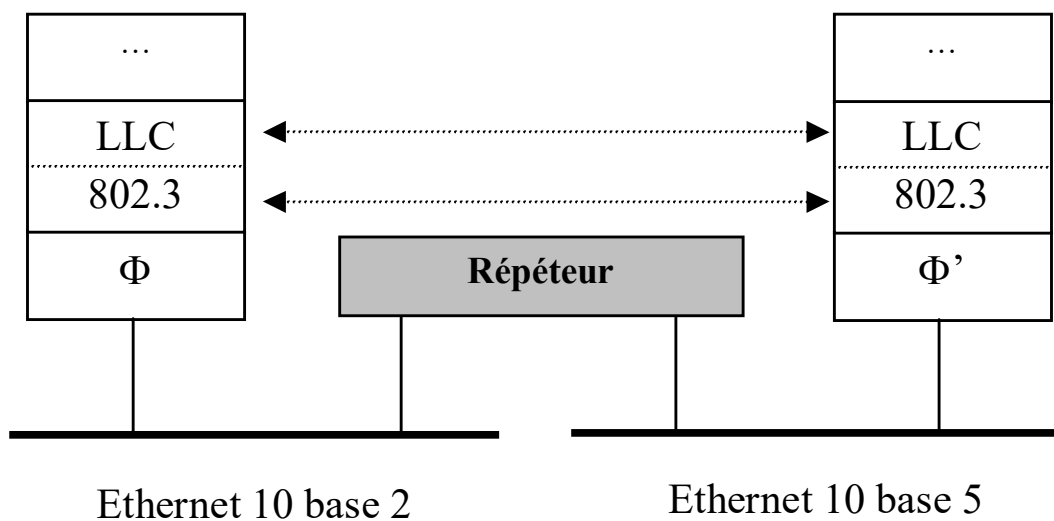
- **Répéteurs et HUB opèrent au niveau physique**

⇒ ils n'interprètent pas les trames MAC

- **Fonction de base**

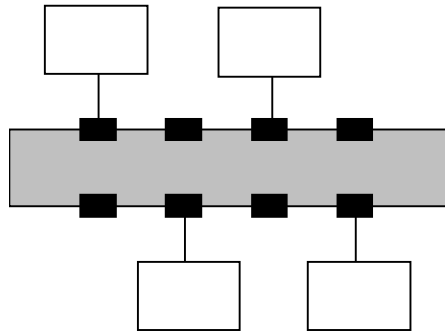
- Régénérer le signal arrivé en entrée vers un LAN de même architecture (excepté éventuellement au niveau physique)

- Ex



➤ Rmq

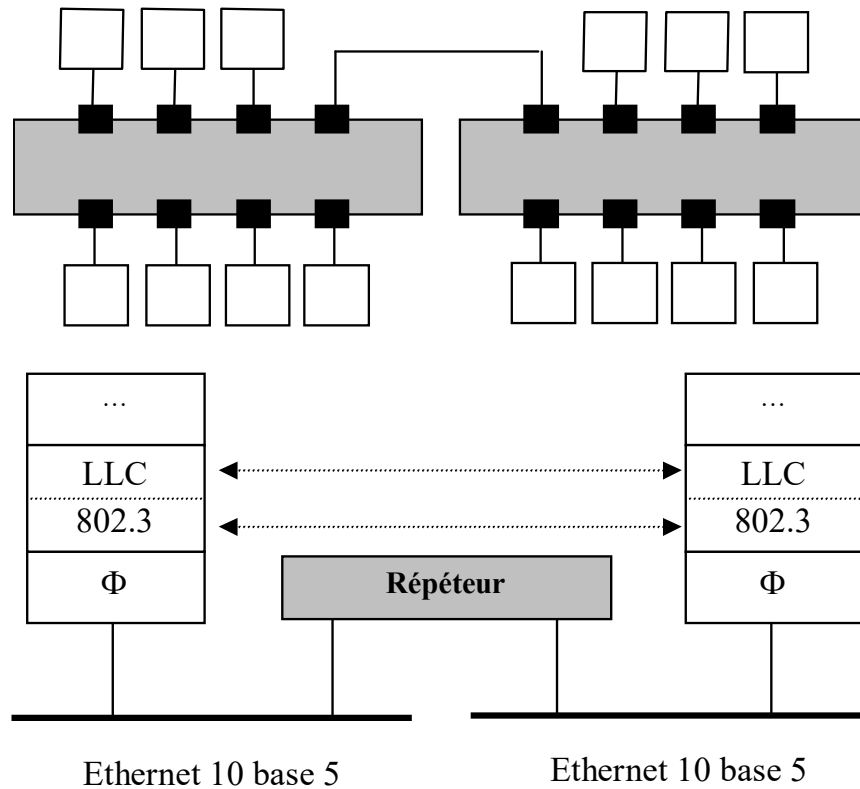
- Dans la technologie Ethernet, les HUB sont (étaient) surtout utilisés pour interconnecter des hôtes



- Répéteurs et HUB sont surtout utilisés pour leur fonction de régénérateur du signal

→ ce qui permet d'étendre le réseau

(même si compatibilité de couche physique)

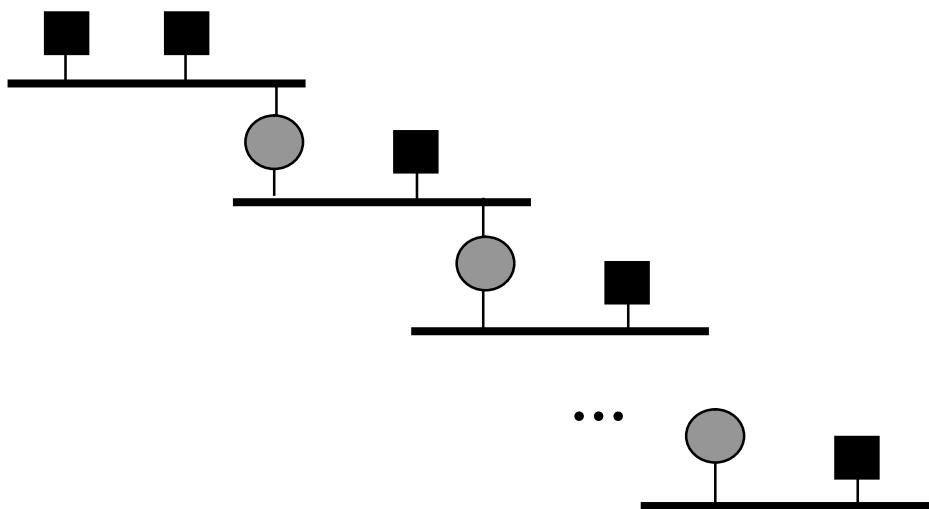


1.2. Limites des répéteurs et des HUB

■ Trois limites principales

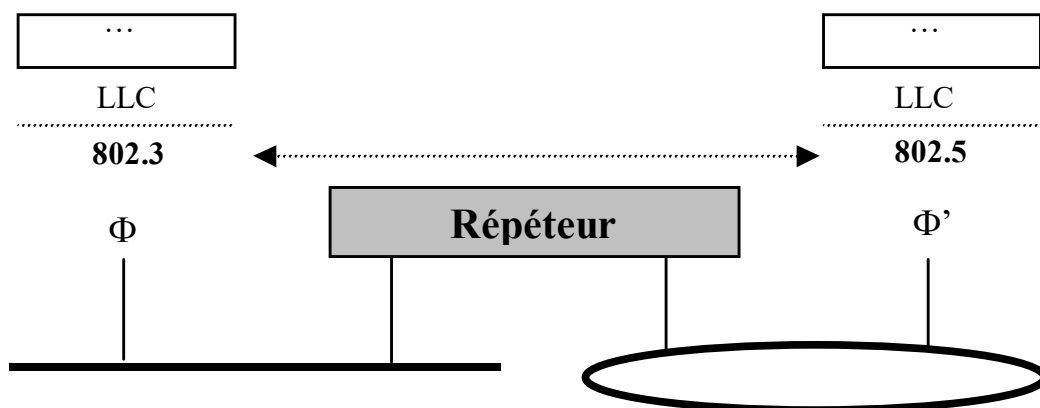
1] Interconnexion de LAN via répéteur(s) non envisageable :
sur une grande distance => problème de performance

– Ex



2] Interconnexion de LAN via répéteur(s) non envisageable :
si les LAN diffèrent à partir du niveau MAC (cf. ex. introductif)

– Ex



3] Un répéteur régénère toujours un signal reçu sur un port vers le ou le(s) autre(s) LAN(s) auxquels il est connecté



un seul « domaine de collision », i.e. :

« ensemble de machines soumises au même contrôle d'accès au médium »

ou bien encore :

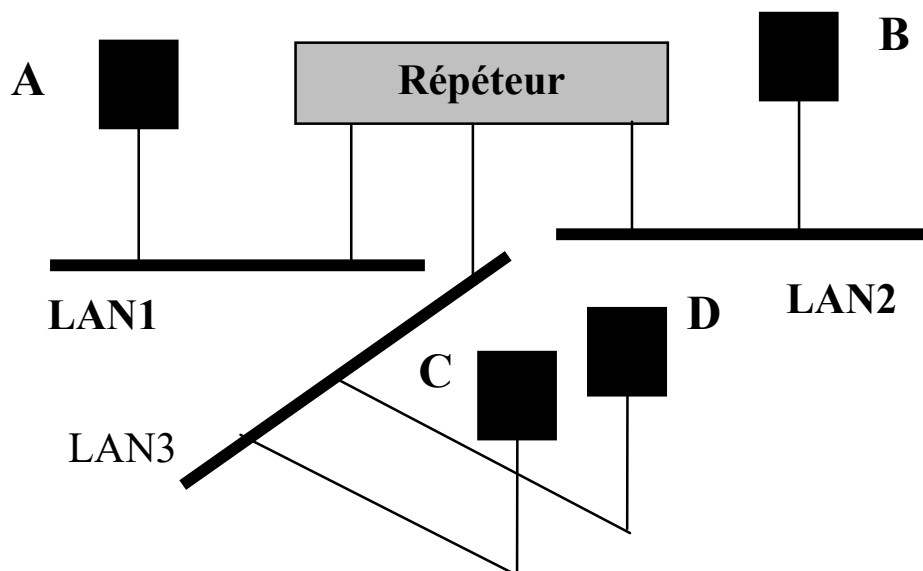
« ensemble de machines, telles que si 2 d'entre elles émettent une trame en même temps, celles-ci entrent en collision »



- non optimal en termes d'utilisation de bande passante
- potentiellement pénalisant pour certaines machines

▪ **Ex**

- A émet une trame à destination de B => ??

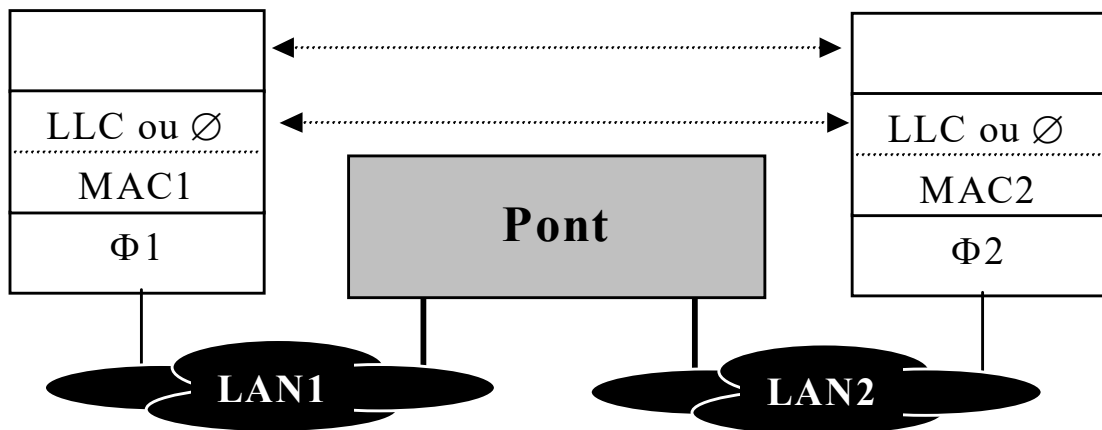


2. Interconnexion par pont

2.1. Niveau opératoire et fonctions

- **Un pont opère au niveau Φ et au niveau MAC**

⇒ un pont interprète les trames MAC (adresse, ...)



- **Fonctions de base initiales**

- *Interconnexion de 2 ou plusieurs LAN*
 - éventuellement incompatibles au niveau MAC
 - mais dont les protocoles de niveau supérieur identiques
- **Distinction de plusieurs domaines de collisions** (un par port)
- « **Filtrage** » des trames lors de leur passage d'un port à un autre

- **Aujourd'hui**

- Seules les 2 dernières fonctions de base sont implantées dans les ponts encore en activité, ou leurs évolutions : **switch Ethernet**, ...

⇔ **ponts « transparents »**

2.2. Pont transparent et switch Ethernet

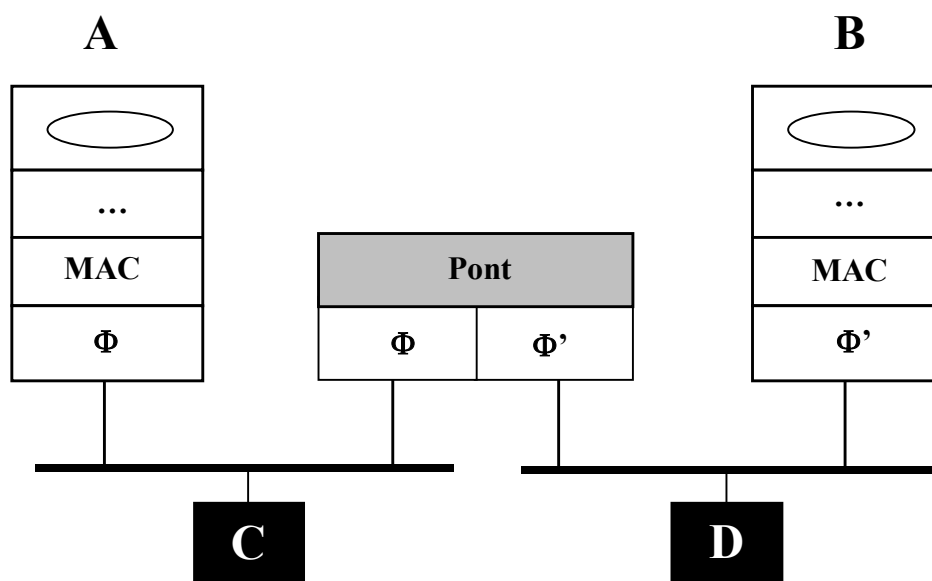
♦ 1^{ère} fonction : distinction de plusieurs domaines de collision

▪ A la $\neq$ d'un répéteur :

- un pont transparent considère un domaine de collision par port, i.e. :
 - les trames peuvent être circulées **en même temps** sur les $\neq$ LAN
 - ce que ne permet pas un répéteur !
 - mais lorsqu'une trame est échangée entre 2 LAN $\neq$ (LAN 1 et 2) :
 - le pont applique le contrôle d'accès du LAN 2 vers lequel il doit commuter la trame en provenance du LAN 1
 - **stockage** de la trame avant commutation
 - risque ? : ...

▪ Ex

- Hyp : MAC = 802.3

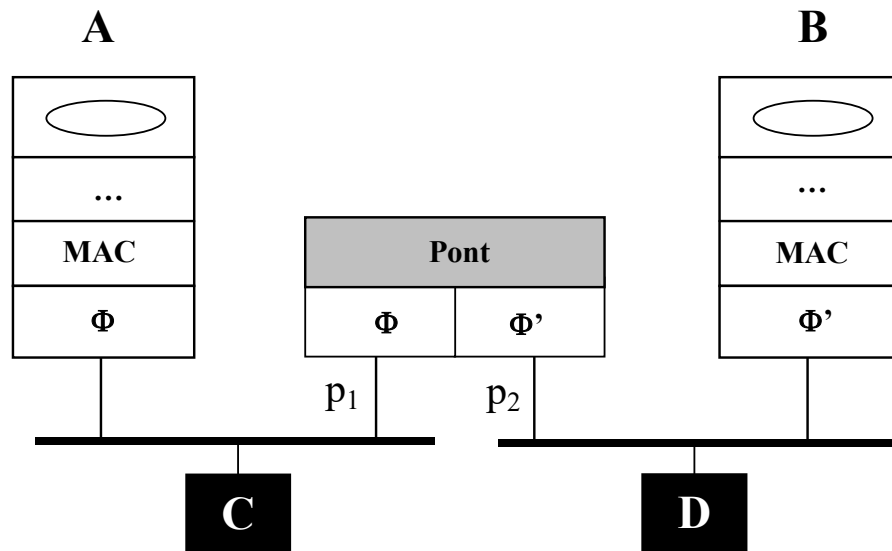


♦ 2^{ème} fonction : filtrage des trames

- **Action** $\Leftrightarrow$ ne pas laisser passer « systématiquement » les trames d'un LAN vers un autre

- **Ex**

- Hyp : MAC = 802.3
- p_1 = port de rattachement du pont au LAN de A et C (LAN 1)
- p_2 = port de rattachement du pont au LAN de B et D (LAN 2)



- Action du pont si émission par A d'une trame (MAC)
 - à destination de B :
 - recevant T sur le port p_1 , P propage T vers B via le port p_2
 - à destination de C :
 - recevant T sur le port p_1 , P ne propage pas T sur le port p_2
 - à condition qu'il sache que $C \in$ au même LAN que A !

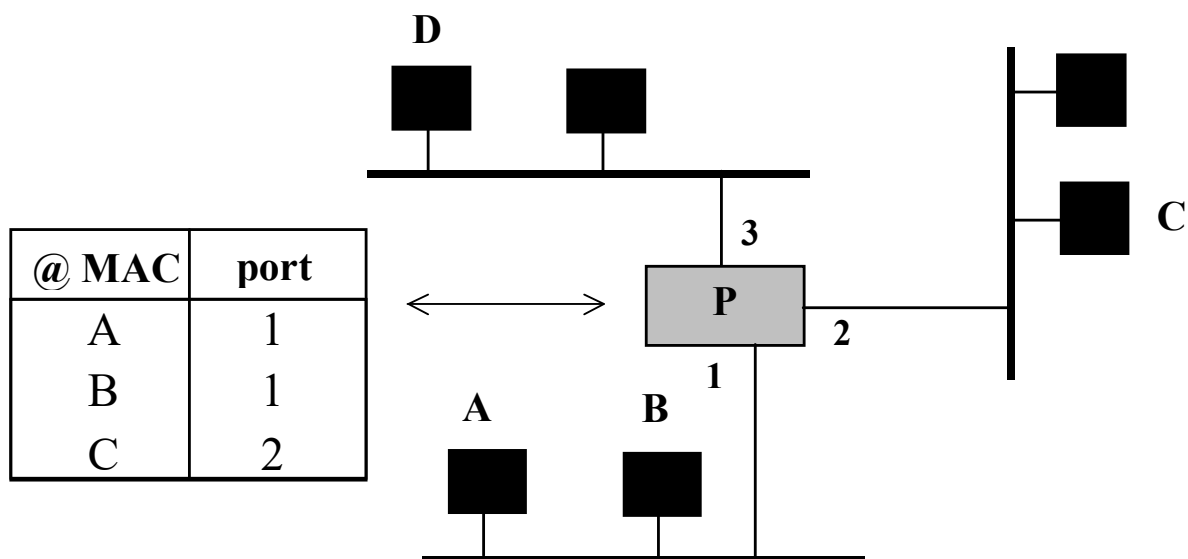
▪ **Algo : à la $\neq$ d'un répéteur, un pont**

- ne relaye pas systématiquement une trame reçue sur un port vers tous les autres ports
- mais applique un filtre basée sur la consultation :

- de l'@ MAC_{dest} de la trame à relayer
- d'une table gérée localement contenant une liste de correspondances « @ MAC, port » signifiant :

« la station d'adresse MAC "@ MAC" se trouve sur le LAN accessible par le port physique "port" »

– Ex



- Hyp : émission d'une trame de A vers :
 - B ➔ action du pont ? réponse = ...
 - C ➔ action du pont ? réponse = ...
 - D ➔ action du pont ? réponse = ...

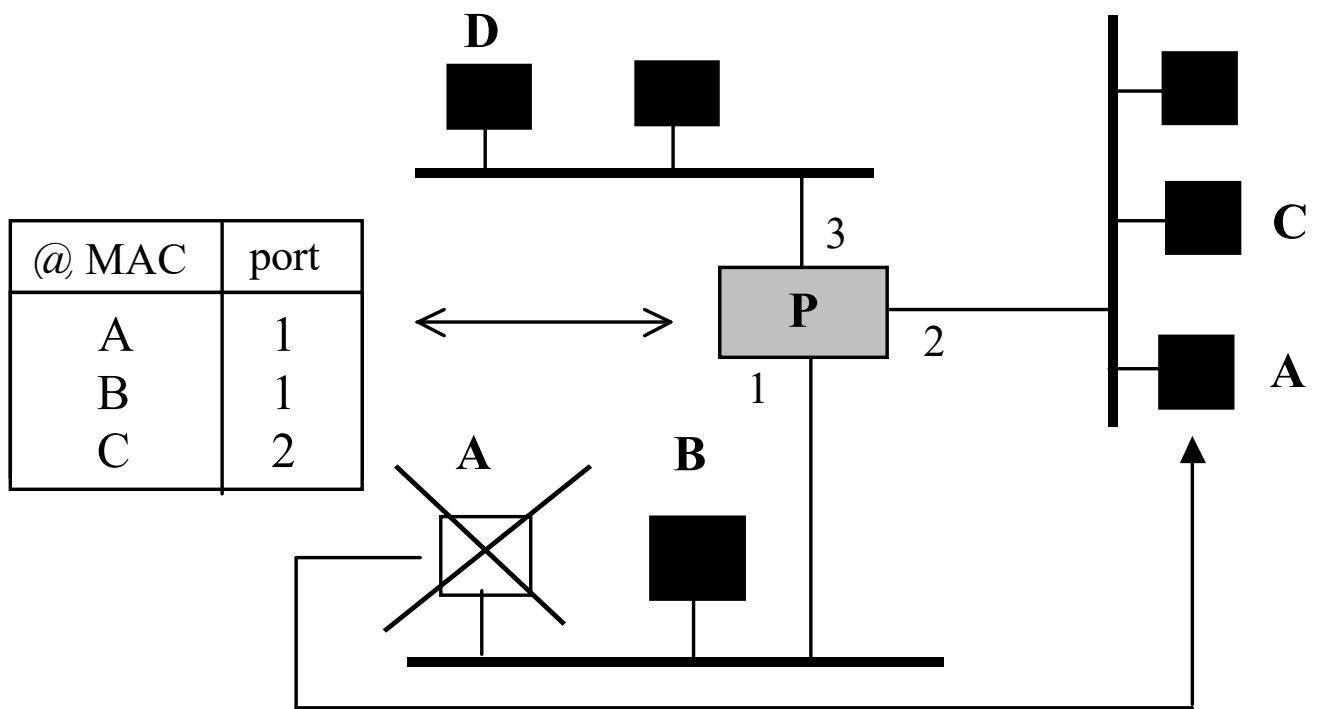
▪ Algorithme d'auto apprentissage

⇔ algorithme de mise à jour de la table d'un pont

- Initialement, la table du pont P est vide
- Sur réception d'une trame sur l'un de ses ports (x par ex)
 - P consulte l'@ MAC_{src} de la trame
 - Si cette entrée n'existe pas dans sa table :
 - P efface toute autre entrée « @ MAC lue, y ≠ x »
 - P crée une entrée « @ MAC lue, x » dans sa table
 - P active un timer associé à cette entrée
 - sinon :
 - P réactive le timer
 - P consulte l'@ MAC_{dest} de la trame
 - si ∃ une entrée « @ MAC lue, x » alors :
 - la trame n'est relayée nul part (elle est rejetée)
 - si ∃ une entrée « @MAC lue, y ≠ x » alors :
 - la trame est relayée sur le port y
 - s'il n'existe pas d'entrée alors :
 - la trame est relayée sur tous les ports de P excepté x
- *Actions supplémentaires : ?*

➤ Rmq

- Si une station est déplacée d'un LAN vers un autre ou bien si elle est supprimée
 - Ex : A est déplacée sur le LAN de C
 - Problème et solution ?



⇒ retour à l'algo précédent (*cf action supplémentaire*)

♦ Limites des ponts transparents

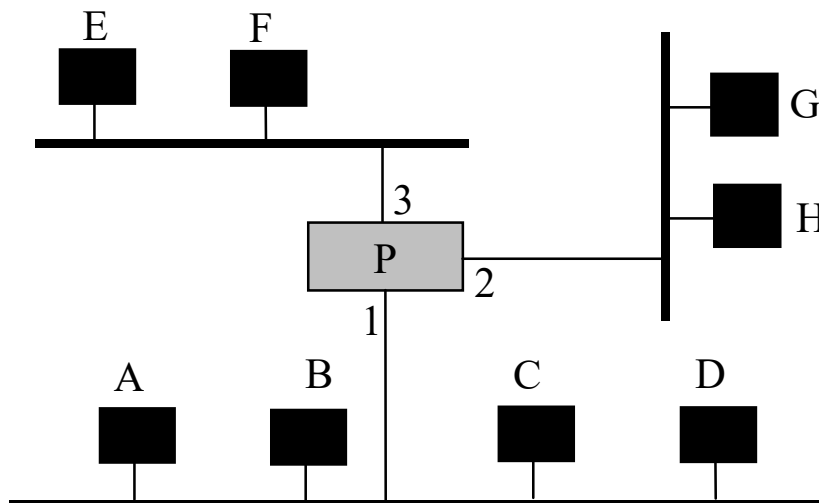
▪ 2 limites principales liées :

- 1] à l'hétérogénéité des LAN
 - Interconnexion de 2 LAN via un pont transparent
 - IMPOSSIBLE si leurs architectures diffèrent au niveau MAC
- 2] à la non distinction de $\neq$ domaines de broadcast / multicast
 - $\Leftrightarrow$ un pont ne filtre pas les trames de broadcast / multicast



Non optimal en termes d'utilisation de bande passante et pénalisant pour certaines machines

- Ex



♦ Switch Ethernet versus pont transparent

■ Quand il est utilisé pour interconnecter des HUB

- un switch Ethernet $\Leftrightarrow$ un pont transparent (Ethernet)

$\Rightarrow$ mêmes fonctions de base :

- distinction d'un domaine de collision par port et filtrage

■ Mais en plus, un switch :

- permet d'interconnecter des machines (pas besoin de HUB)

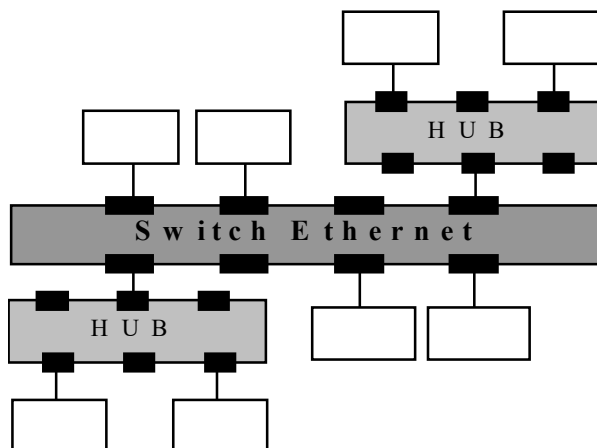
- possède d'autres caractéristiques :

- transmission full duplex sur le lien machine - switch
- plus performant (fiabilité, débit, ...)
- multi-ports (8, 12, 16, ...)

- intègre potentiellement (selon son prix) des fonctions sup :

- administration SNMP $\rightarrow$ switch "*manageable*"
- configuration de « réseaux locaux virtuels » (VLAN)
- configuration du switch en tant que routeur IP (!)

} cf. suite

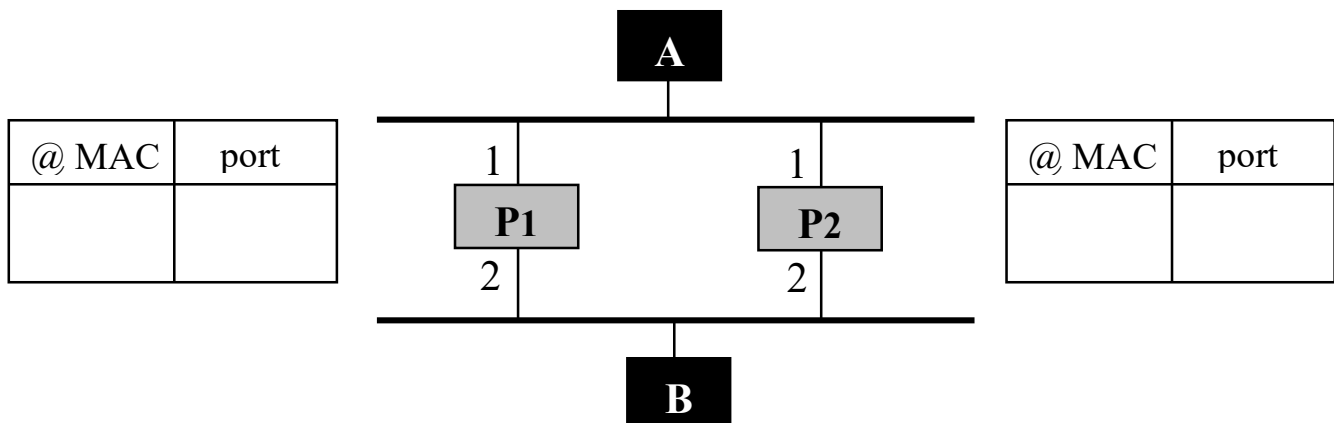


2.3. Problème des boucles et algo. du *Spanning Tree* (non étudié)

♦ Introduction

- Problème dans une interconnexion de niveau 2 : **les boucles**
 - si utilisation de plusieurs ponts (ou commutateurs Ethernet)

■ Ex



1) A émet une trame T à destination de B

- **Action de P1** : Table P1 $\leftarrow$ « @ MAC A , 1 »
Relayage de T sur le port 2
- **Action de P2** : Table P2 $\leftarrow$ « @ MAC A , 1 »
Relayage de T sur le port 2

2) T est relayée sur le LAN 2 via (P1, 2)

- **Action de B** : Consommation de T
- **Action de P2** : Effacement de « @ MAC A , 1 » de Table P2
Table P2 $\leftarrow$ « @ MAC A , 2 »
Relayage de T sur le port 1

3) T est relayée sur le LAN 2 via (P2, 2)

- **Action de B** : Consommation de T
- **Action de P1** : Effacement de $\langle @ \text{ MAC A , 1 } \rangle$ de Table P1
Table P1 $\leftarrow \langle @ \text{ MAC A , 2 } \rangle$
Relayage de T sur le port 1

4) ...

▪ **Conclusion**

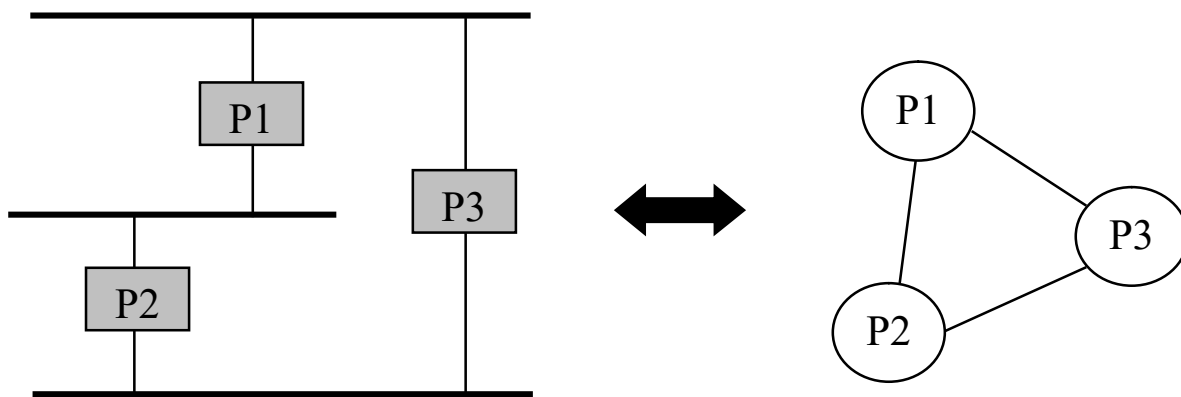
- **Si** présence de boucle(s) physiques **Alors** :
 - mise à jour inutile des tables des $\neq$ ponts
 - utilisation inutile de bande passante :
 - relayage « abusif » des trames
 - mise à contribution inutile des stations
 - consommation des trames redondantes au niveau MAC
 - rejet des trames redondantes au niveau supérieur

- **Solution** = mise en œuvre au sein du réseau d'un algorithme qui évite au trafic de parcourir des boucles
 - 2 algorithmes principaux :
 - l'algorithme du « Spanning Tree »
 - l'algorithme du « **Source Routing** »

♦ Algorithme du *Spanning Tree*

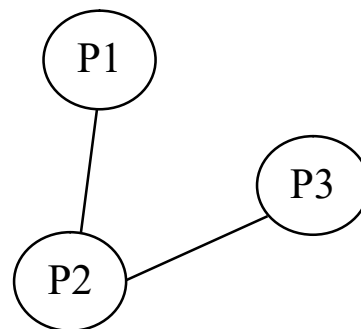
▪ Qu'est ce qu'un « *Spanning Tree* » ?

- En français : « arbre de recouvrement »
- Dans une interconnexion par pont de $\neq$ LAN, la connectivité entre les ponts peut être représentée sous forme de graphe :



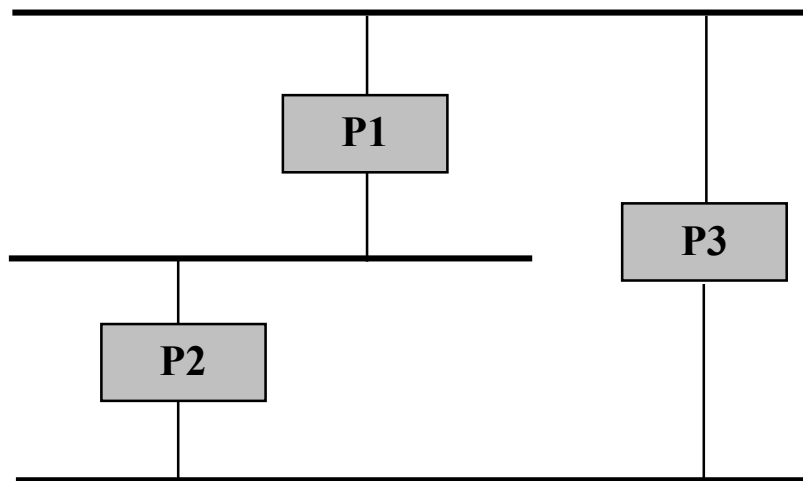
– *Spanning Tree*

- sous graphe du graphe précédent :
 - couvrant tous les nœuds
 - mais ne possédant pas de boucle



▪ Algorithme du Spanning Tree (Perlman 19xx)

- Solution appliqué pour l'interco. de niveau 2 de réseaux Ethernet
- Algo distribué entre les ponts
 - implémenté dans chaque pont
 - dont l'effet consiste à « inhiber » certains ports des $\neq$ ponts
 - de sorte que les trames ne puissent pas parcourir de boucle existant physiquement entre les ponts
- Elire un pont particulier = **racine**
- Faire en sorte que pour chaque pont :
 - il n'existe qu'un seul chemin entre lui-même et la racine



■ Protocole

- Repose sur l'échange entre les ponts de Bridge-PDU (B-PDU)
- Nécessite l'attribution d'un identificateur :
 - à chaque pont : **id_pont**
 - à chaque port d'un pont : **id_port**
- A pour objectifs
 - 1] que tous les ponts prennent connaissance de l'identificateur du pont de plus petit id_pont = **racine**
 - 2] que chaque pont détermine lequel de **ses** ports **est le plus proche** de la racine selon la règle suivante :
 - un port p1 d'un pont P est plus proche de la racine R qu'un port p2 de P si :
 - la distance en nb de pont à traverser entre P et R via p1 est inférieure à la distance entre P et R via p2
 - à égalité de distance, le port de plus petit id_port est considéré comme le plus proche
 - 3] que sur chaque LAN hormis celui (ceux) du pont racine R, chaque pont détermine s'il est le pont **le plus proche** de R :
 - un pont P1 est plus proche de la racine R qu'un pont P2 si :
 - la distance en nb de pont à traverser entre P1 et R est inférieure à la distance entre P2 et R
 - à égalité de distance, le pont de plus petit id_pont est considéré comme le plus proche

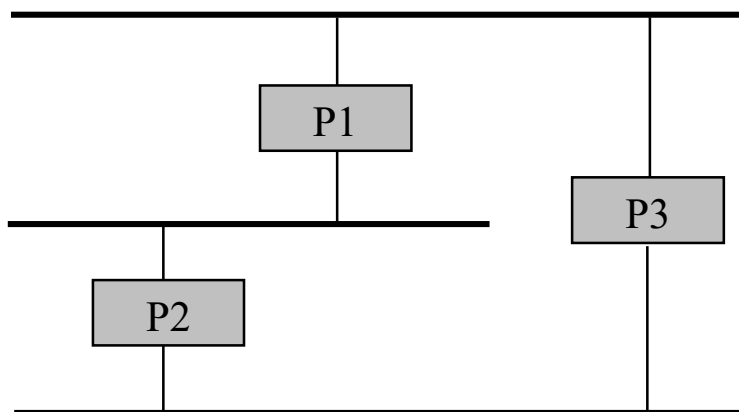
▪ **Une fois ces objectifs atteints**

- Soit « **R** » le pont **racine**
- Pour chaque pont :
 - soit « **p** » le port le plus **proche** de R
- Pour chaque LAN :
 - soit « **d** » le port de rattachement du pont le plus proche de R

⇔ pont « **désigné** » du LAN

- Alors :
 - conservation
 - de tous les ports de « **R** »
 - du port « **p** » de chaque pont
 - du port « **d** » de chaque LAN auquel R n'appartient pas
- « inhibition » de tous les autres ports

⇔ par un port inhibé, un pont ne peut ni recevoir, ni relayer une trame



▪ **Format des B-PDU : [id_R, coût, id_E, id_port]**

- **id_R** = identificateur supposé du pont racine par le pont émetteur du B-PDU
- **coût** = distance entre le pont émetteur et la racine supposée
- **id_E** = identificateur du pont émetteur
- **id_port** = ident. du port sur lequel le B-PDU est émis

▪ **Chaque B-PDU définit une « configuration » comparable aux autres de la façon suivante :**

$$[\text{id_R1}, \text{coût1}, \text{id_E1}, \text{id_port1}] < [\text{id_R2}, \text{coût2}, \text{id_E2}, \text{id_port2}]$$

- si :
 - $\text{id_R1} < \text{id_R2}$
- ou bien si :
 - $\text{id_R1} = \text{id_R2}$ et $\text{coût1} < \text{coût2}$
- ou bien si :
 - $\text{id_R1} = \text{id_R2}$, $\text{coût1} = \text{coût2}$ et $\text{id_E1} < \text{id_E2}$
- ou bien si :
 - $\text{id_R1} = \text{id_R2}$, $\text{coût1} = \text{coût2}$, $\text{id_E1} = \text{id_E2}$ et $\text{id_port1} < \text{id_port2}$

III. Les réseaux locaux virtuels (VLAN)

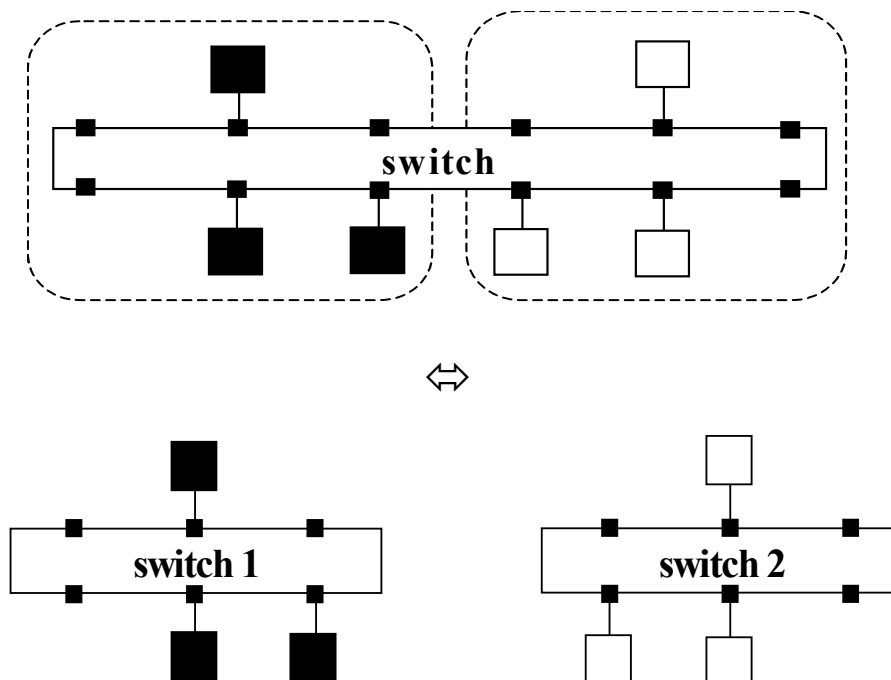
1. Le concept de VLAN

2. Configuration d'un VLAN

1. Le concept de VLAN

▪ VLAN = Virtual LAN

- Solution permettant de répartir les machines d'un même switch Ethernet ou d'une interconnexion de switch Ethernet
 - en plusieurs LAN « logiques » (ou virtuels → VLAN)
 - indépendants les uns des autres, i.e. :
 - que l'on ne souhaite pas interconnecter
 - ou qui ne pourront communiquer que par le biais d'un routeur (ou d'un proxy applicatif, d'un NAT, ... → cf. suite du cours)
 - l'objectif étant :
 - d'interdire ou d'autoriser sous contrôle l'accès à certains services
 - de distinguer plusieurs domaines de broadcast, ...
- **Ex** : constitution de 2 VLAN sur un unique switch Ethernet



2. Configuration d'un VLAN

▪ Plusieurs façons de configurer un VLAN

⇔ définir l'appartenance d'un hôte à un VLAN

1] Par port physique des switch Ethernet ⇔ méthode la + simple

– Principe

- A chaque port du switch est associé un n° de VLAN
 - opération effectuée par l'administrateur du switch (root)
 - par défaut : tous les ports sont sur le même VLAN = VLAN 0
- Sur soumission par un hôte d'une trame T sur le port p, le switch :
 - commute T vers le port permettant d'accéder au destinataire de la trame si @ eth_{dest} = @ unicast
 - à condition que son port appartienne au même VLAN que celui de p
 - commute T vers tous les ports **du VLAN de p** si @ eth_{dest} = @ broadcast

– Ex : configuration de 2 VLAN distinguant 2 réseaux IP

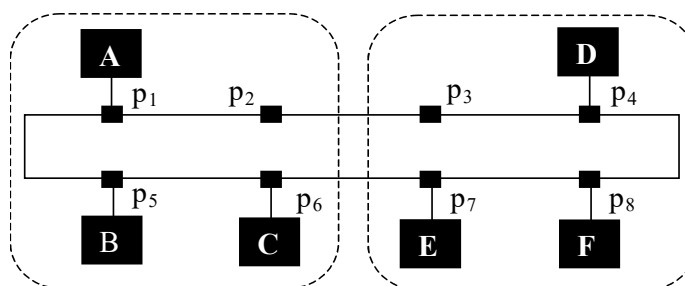


Table du switch

port	VLAN

– Rmq

- Si déplacement d'une machine d'un port à un autre :
 - obligation de reconfigurer le VLAN
 - trou potentiel en terme de sécurité

2] Par adresse MAC

- Méthode plus souple que la précédente car elle évite de reconfigurer les VLAN en cas de déplacement d'une machine
- Principe
 - A chaque @MAC est associée un n° de VLAN (administrateur)
 - Sur soumission par un hôte d'une trame T sur le port p, le switch :
 - 1) détermine le n° de VLAN correspondant à l'@ MAC_{src} de T et si besoin : met à jour le n° de VLAN de p
 - 2) commute T :
 - vers tous les ports du VLAN de p si @ eth_{dest} = @ broadcast
 - vers le port du VLAN de p permettant d'accéder au destinataire de la trame si @ eth_{dest} = @ unicast
- Ex

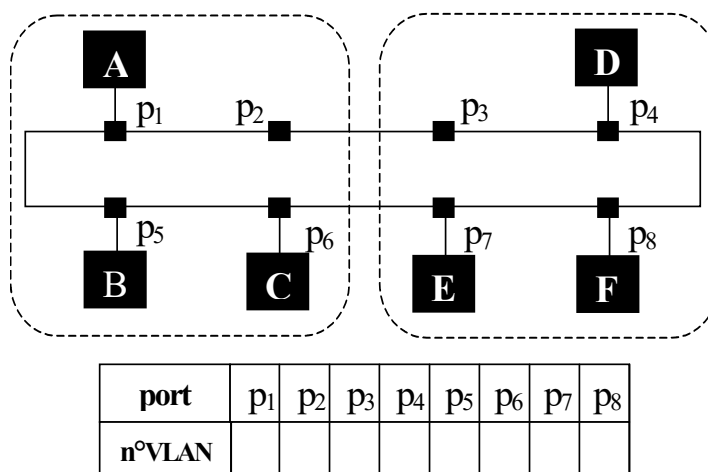


Table du switch

@MAC	n°VLAN

- Rmq
 - Lourd en termes de configuration si grand nombre de machines

3] Par adresse IP (et plus généralement adresse de niveau 3)

- Principe analogue au précédent
 - A chaque @ de (sous) réseau IP est associée un n° de VLAN
 - Sur soumission par un hôte d'une trame T sur le port p, le switch :
 - 1) détermine le n° de VLAN correspondant au préfixe de l'@ IP_{src}
 - * du paquet IP encapsulé dans T
 - * et si besoin : met à jour le n° de VLAN de p
 - 2) commute T :
 - vers tous les ports du VLAN de p si @ eth_{dest} = @ broadcast
 - vers le port du VLAN de p permettant d'accéder au destinataire de la trame si @ eth_{dest} = @ unicast
- Ex

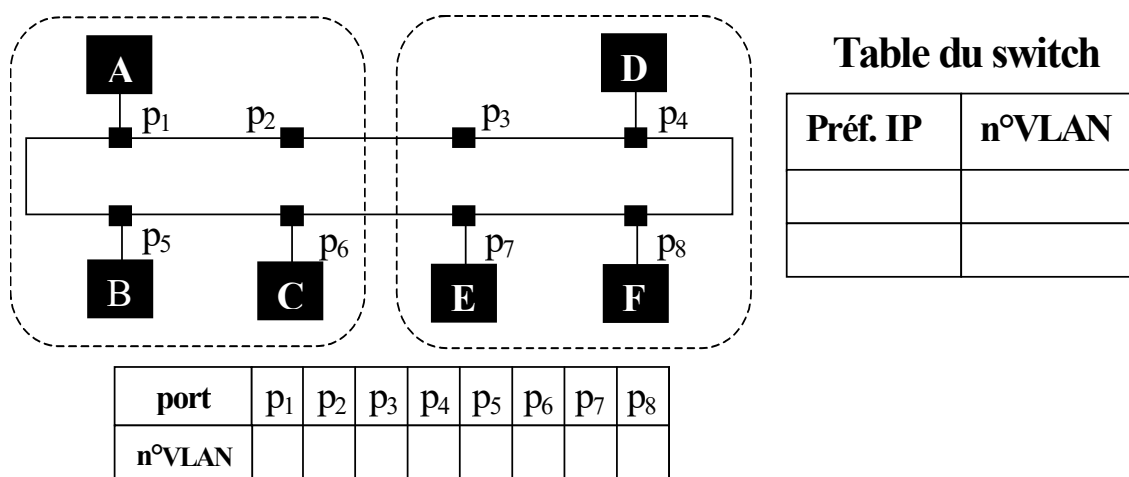


Table du switch

Préf. IP	n°VLAN

- Rmq
 - Même s'il y a consultation des adresses IP
 - le mécanisme « reste de niveau 2 » : aucune fonction de routage

4] Par protocole de niveau 3 (IPv4, IPv6, ...)

- Principe analogue au précédent
 - A chaque protocole de niveau 3 (IPv4, IPv6, ...) est associé un n° de VLAN (toujours par l'administrateur du switch)
 - Sur soumission par un hôte d'une trame T sur le port p, le switch :
 - 1) détermine le n° de VLAN correspondant au protocole encapsulé dans T → via le champ « type » de la trame
 - et si besoin : met à jour le n° de VLAN de p
 - 2) commute T :
 - vers tous les ports du VLAN de p si @ eth_{dest} = @ broadcast
 - vers le port du VLAN de p permettant d'accéder au destinataire de la trame si @ eth_{dest} = @ unicast
- Ex

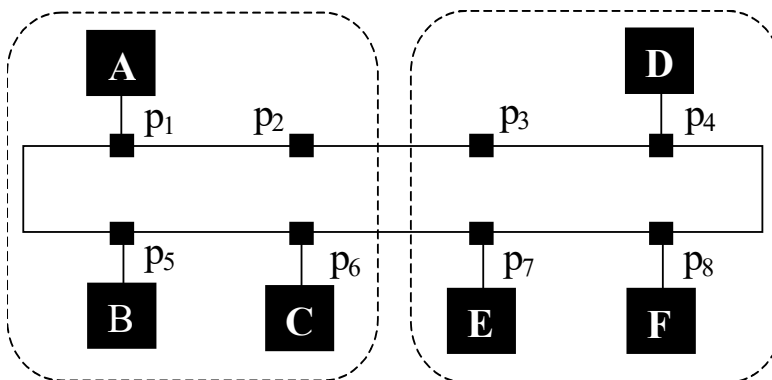


Table du switch

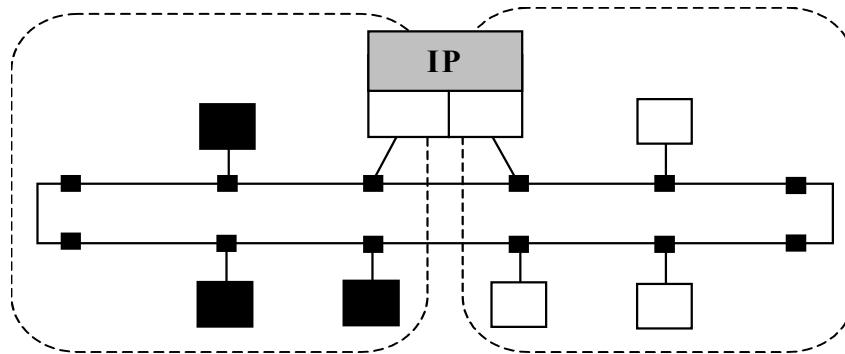
Proto.	n°VLAN

port	p ₁	p ₂	p ₃	p ₄	p ₅	p ₆	p ₇	p ₈
n°VLAN								

▪ Rmq 1

- Comment autoriser la communication entre deux machines appartenant à des VLAN distinguant 2 ou > 2 (sous) réseaux IP ?

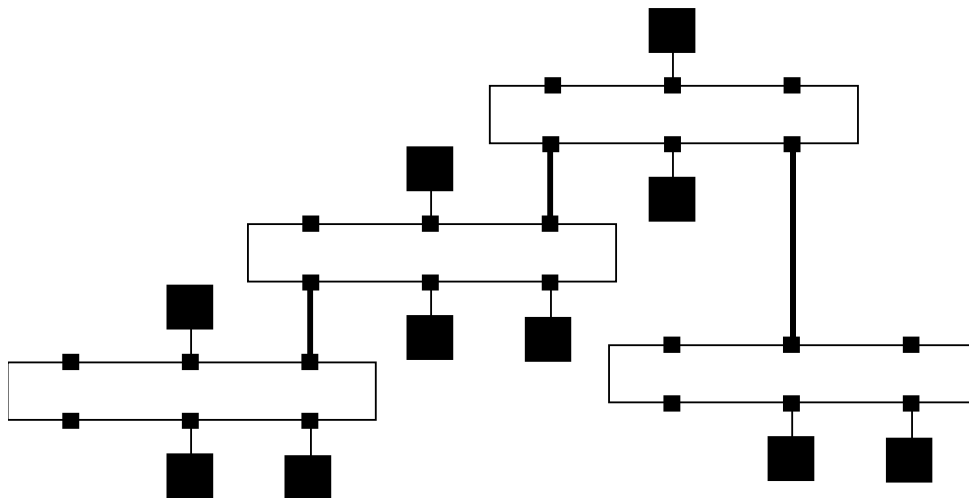
- **Réponse** = par le biais d'un routeur IP ! => Pourquoi ?



- Possibilité de configurer une machine (PC Linux par ex) en routeur
 - alors qu'elle n'est configurée qu'à un seul port du switch !
 - nécessite la mise en œuvre de la norme Ethernet 802.1q entre la machine « routeur » et le switch Ethernet : cf . TP n°2
- Ceci étant : un routeur physique ou une machine routeur sont-ils indispensables ... ?
 - en d'autres termes : le switch ne peut-il pas les émuler ?!
 - ⇔ switch Ethernet appelés « commutateur / routeur »
ou « com. de niveau 2 et 3 » (cf. TP n°2)
- Intérêt de la solution VLAN vis à vis d'une interconnexion classique de réseaux IP proches
 - moins de matériel
 - coexistence possible de ≠ techno d'interconnexion (niveau 3)
 - ...

▪ Rmq 2

- Configuration des VLAN
 - Manuelle par l'administrateur du switch
 - Facile si un seul commutateur
 - Moins évidente si plusieurs switch en cascade



- Développement de solutions de configuration :
 - « semi automatique »
 - Via des logiciels basés sur l'utilisation de SNMP
 - Logiciels offrant une IHM graphique à l'utilisateur
 - « automatique »
 - Fait suite à un projet de norme pour les VLAN : norme 802.1Q
 - Utilise le protocole GVRP défini dans cette norme
 - GARP VLAN Registration Protocol

IV. Interconnexion par routeur IP

1. Introduction

2. IP : Internet Protocol

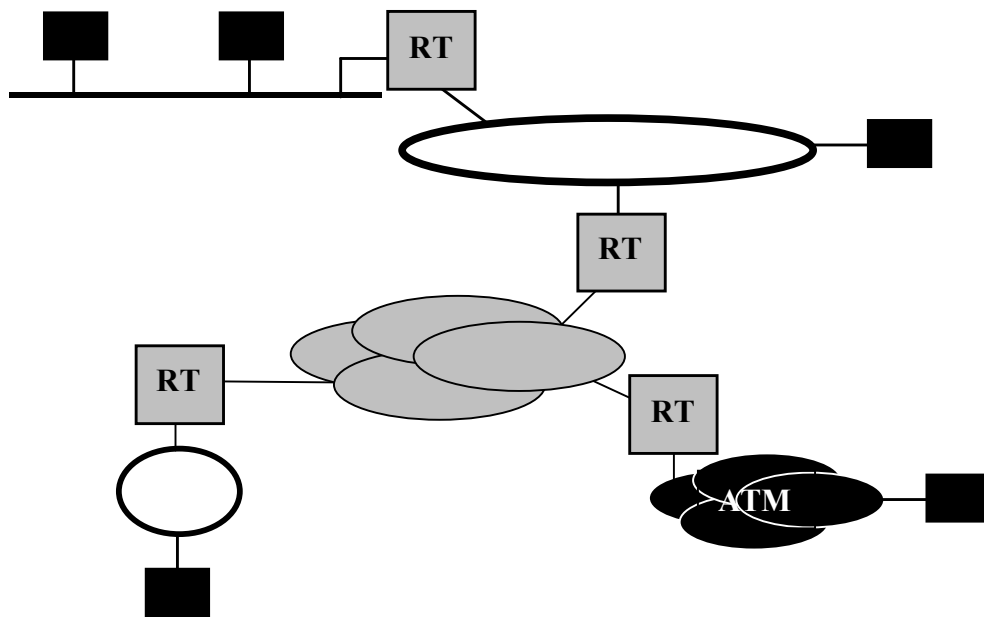
3. Les protocoles de routage (introduction)

4. ARP, RARP/DHCP, proxy ARP et ICMP

1. Introduction

▪ Objectif

- permettre l'interconnexion de réseaux Φ de tout type
 - quelles que soient leurs topologies et architectures de niveau 1 et 2



▪ Dans ce type d'interconnexion de réseaux (*internet*) :

- Equipements d'interconnexion = **routeurs (RT)**
 - **Rôle** = permettre la communication entre machines $\in$ réseaux physiques différents
 - en prenant en compte les limites des ponts transparents, cad :
 - interco. impossible de réseaux différant à partir du niveau MAC
 - non distinction des plusieurs domaines de broadcast/multicast
 - par mise en œuvre d'un même protocole d'interconnexion
 - IP, IPX, ...

➤ Rmq

- **L'*internet* de fait : l'Internet (TCP/IP)**
- **Protocole d'interconnexion de l'Internet : IP (Internet protocol)**
 - PDU = paquet IP (niveau 3)
 - Adressage : @ IP
- **Routage au sein de l'Internet $\neq$ routage au sein des réseaux Φ :**
 - *Formats d'adresse différents*
 - @ IP vs @ MAC (niveau 2), ATM (niveau 3), ...
 - *Algorithmes de routage basés sur des techniques différentes*
 - LAN $\Rightarrow$ pas de problème de « routage »
 - Diffusion des trames sur le réseau et acceptation / rejet des trames en fonction de l'@ MAC
 - Rmq : bémol si switch Ethernet au lieu de HUB
 - **WAN et Internet** : cf. cours 3A (Techniques de commutation)
 - Commutation de circuits
 - Commutation de paquets en mode connecté (/ *circuit virtuel*)
 - Commutation de paquets en mode non connecté (/ *datagramme*)
 - Protocoles de routage différents ➔ cf. suite du cours
 - Internet : RIP, OSPF, ...
 - ...

2. IP : Internet Protocol

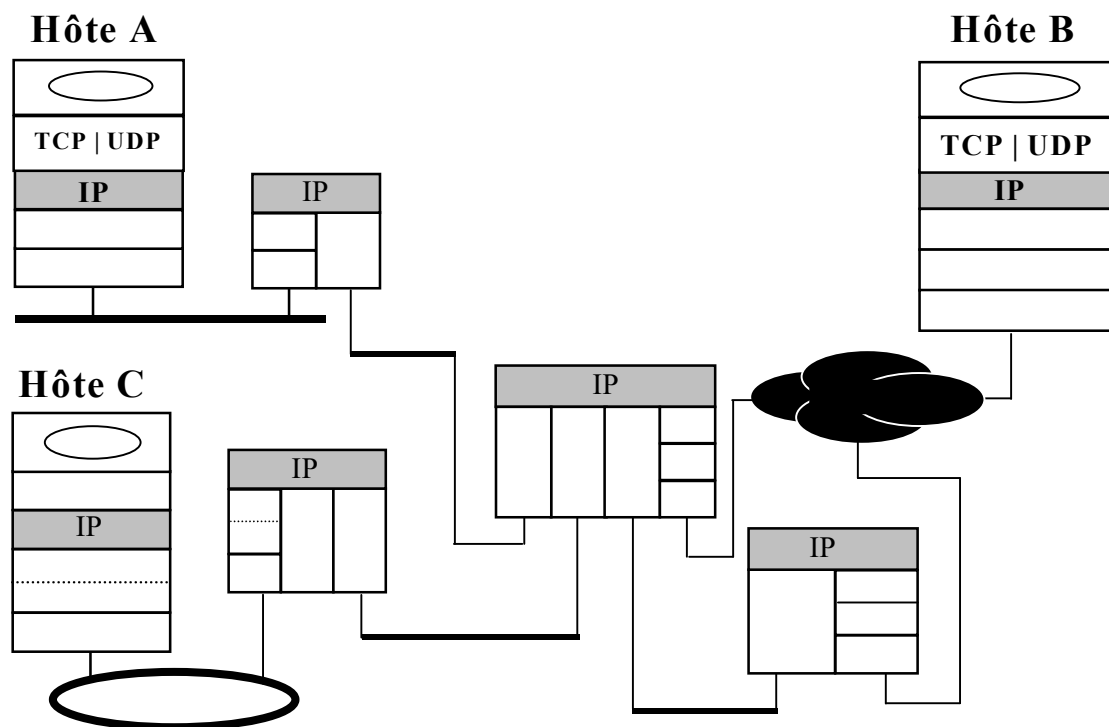
- **IP = protocole sans connexion**

→ ni reprise des pertes ni contrôle de flux

– conçu pour permettre l'échange de **paquets IP** entre hôtes distants ∈ (sous) réseaux IP éventuellement différents

- chaque paquet contenant les informations suffisantes pour être acheminé vers le(s) hôte(s) destinataire(s) :

indépendamment des autres paquets IP !



➤ Rmq

- **Bémol lié à l'existence du protocole ICMP**

– Internet Control Message Protocol → cf. suite du cours

2.1. Service et fonctions

▪ Modèle de service

– Service sans connexion assurant :

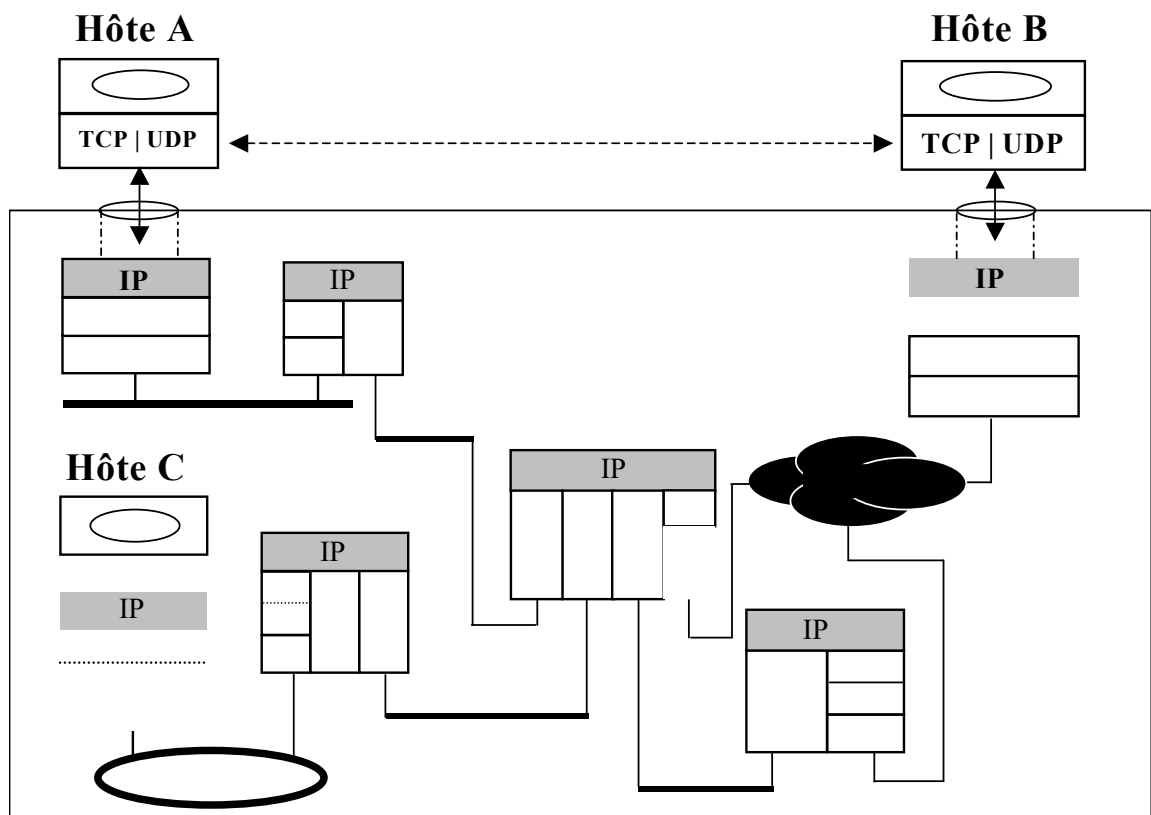
- l'acheminement au travers de l'Internet et la délivrance des données utilisateur entre 2 (ou plusieurs) nœuds d'extrémité

**sans garantie de délivrance ni de séquençement
de ces données !**



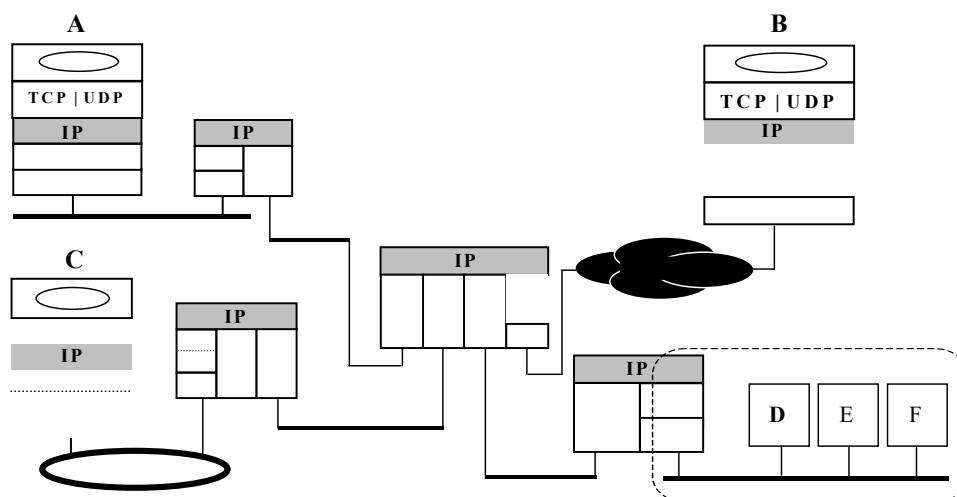
service de « Best Effort » (sans garantie de QoS)

– Ex

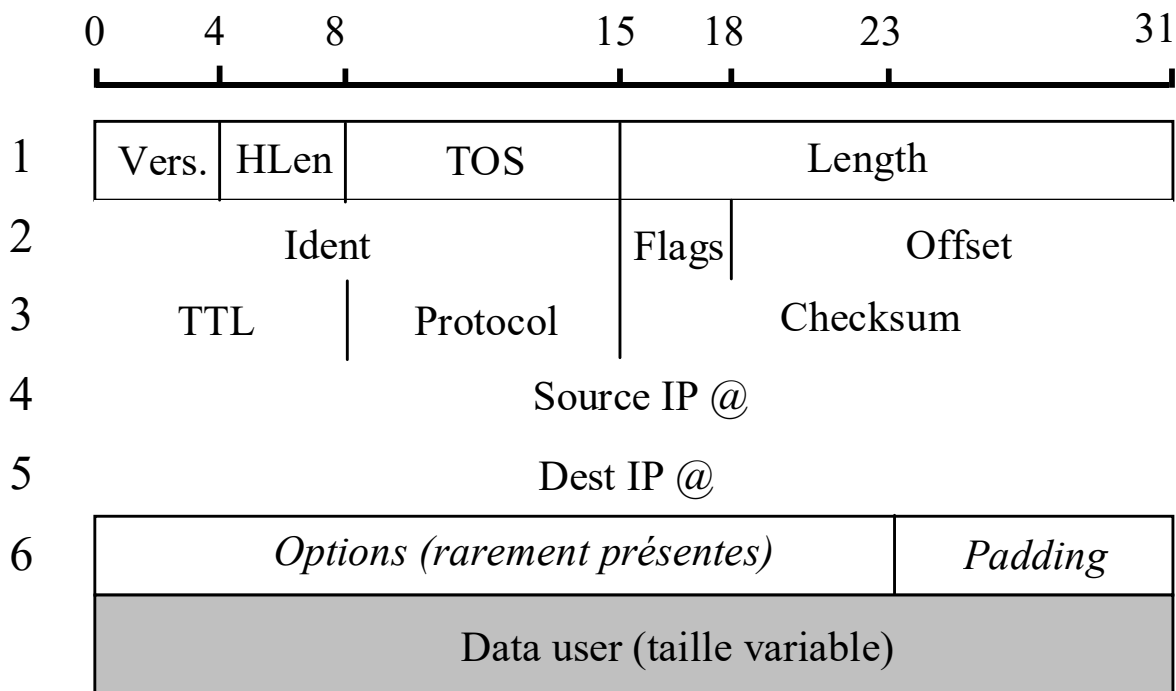


■ Fonctions principales

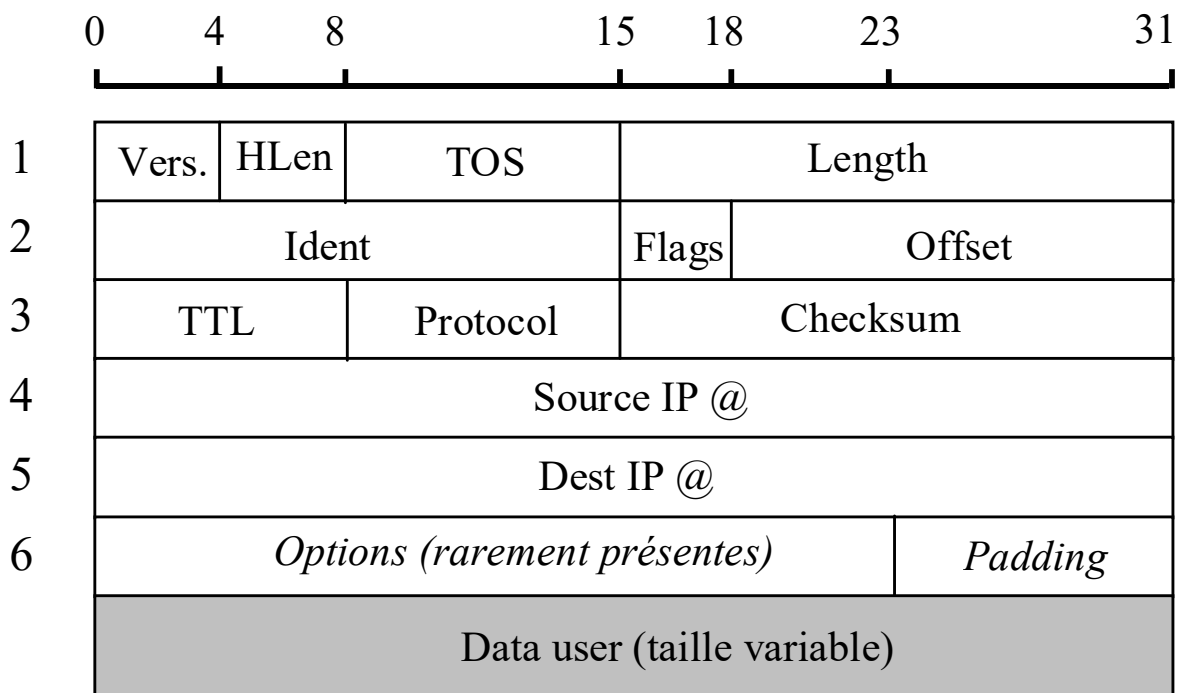
- Routage des paquets d'hôte à hôte en mode « datagramme »
- Fragmentation et réassemblage
 - des paquets IP de taille > MTU des réseaux traversés
 - MTU = unité maximale de transmission
- MUX et DMUX de canaux IP « statiques »
 - i.e. dédiés à des protocoles de niveau supérieurs : TCP, UDP, ...
- Rmq
 - Transmission possible d'un paquet IP **en mode *broadcast***
 - à toutes les machines d'un même réseau IP
 - ... mais pas dans « tout l'Internet » ! ➔ pourquoi ??
 - Transmission possible d'un paquet IP **en mode *multicast***
 - émission du paquet IP d'un hôte vers plusieurs n'appartenant pas nécessairement au même réseau Φ ou au même réseau IP
 - évolution datant de 1992 (version de base) ➔ cf. suite du cours



2.2. Format des paquets IP



- **Vers.** : n° de version (actuellement = 4 pour IPv4)
- **Hlen** : longueur de l'entête en nb de mots de 32 bits
 - Hlen = 5 si aucune option
- **TOS** (Type of Service)
 - défini pour permettre un traitement ≠ des paquets au niveau des routeurs ; **en pratique** : jamais utilisé ... jusqu'à il y a peu ! (QoS)
- **Length** : longueur du paquet en octets (entête incluse)
- **TTL** (Time to Live) : temps de vie (en nb max de routeurs traversés) des paquets dans le réseau

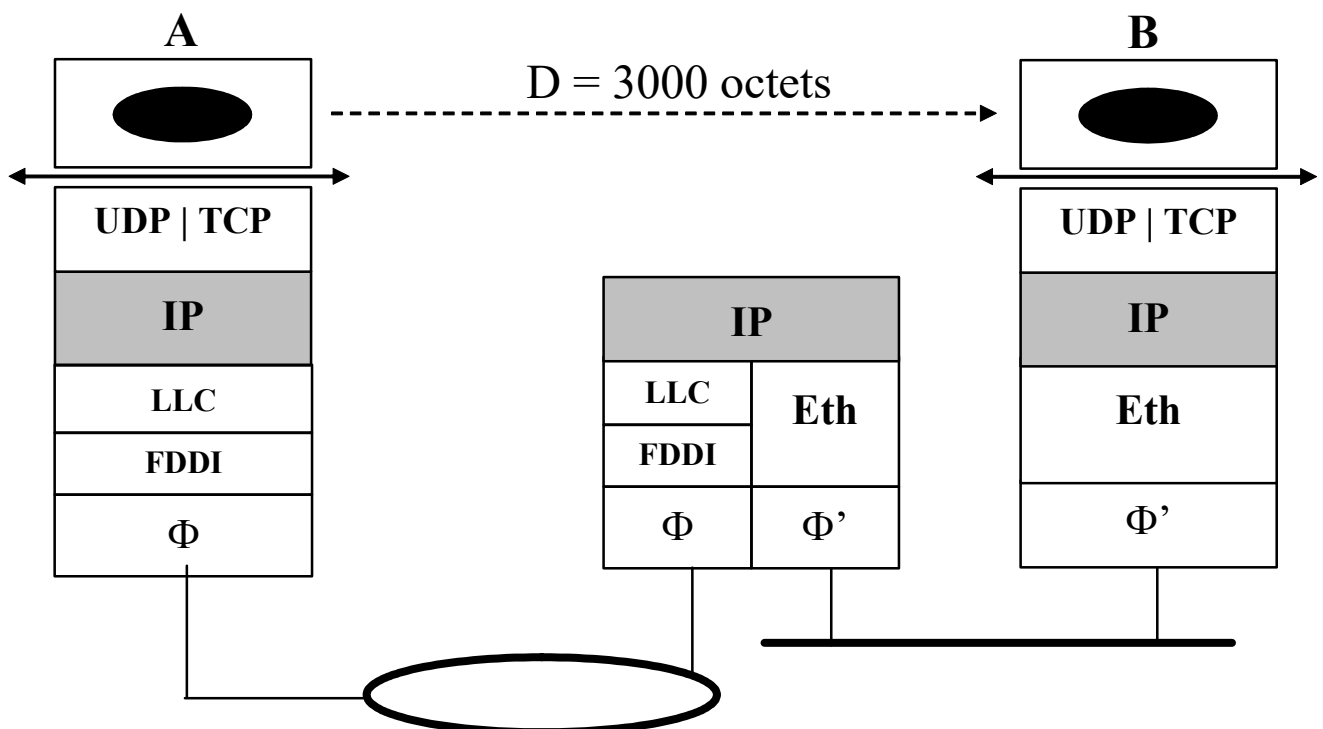


- **Protocol** : identificateur du protocole de niveau supérieur
 - TCP = 6, UDP = 17, ...
- **Checksum** : contrôle d'erreur effectué sur **l'entête** du paquet
- **Ident** : identificateur du paquet (cf. fragmentation/réassemblage)
- **Flags et Offset** : cf. fragmentation/réassemblage
- **Dest et Src @** : @ IP des hôtes destinataire et source
- **Options** : champs optionnel interprété (si présent) par les utilisateurs du service (généralement absent)
- **Padding** : permet d'aligner le champs options sur un multiple de 32 bits (si nécessaire)

2.3. Fragmentation et réassemblage

■ Position du problème

- Interconnecter des réseaux Φ hétérogènes pose le problème :
 - des **différences de MTU** (*Maximum Transmission Unit*)
- Rappel
 - **MTU** = taille max des données user d'une trame MAC
- Exemple
 - Interconnexion d'un LAN Ethernet et d'un LAN FDDI
 - MTU FDDI = 4400 octets / MTU Ethernet = 1500 octets
 - Emission d'une donnée D de 3000 octets de A vers B → **Pb !**



▪ Description du mécanisme

1. L'E(IP) de l'hôte **émetteur** génère des paquets IP de taille compatible avec la MTU de son réseau

2. Lorsqu'un **routeur intermédiaire** doit aiguiller vers un LAN X un paquet IP de taille supérieure à la MTU de X :

– il en répartit la **charge utile** sur autant de paquets IP que nécessaire

⇒ de sorte à ce que chacun ait une taille $\leq$ MTU de X !

- l'entête de chacun de ces paquets reste identique à celle du paquet initial excepté :

- le champ **Length** (réduit)

- le 3^{ème} bit du champ **Flag** = **M** (More Fragment)

- $M = 1$ si au moins un fragment du paquet IP initial suit le fragment courant

- $M = 0$ sinon

- le champ **Offset**

- indique le nb de blocs de 8 octets de données user (du paquet IP initial) précédant celles contenues dans le paquet courant

- puis émet ces paquets sur le LAN X

3. Le réassemblage du paquet IP initial est effectué par l'E(IP) de l'**hôte destinataire**, avant remise des données utilisateur à la couche sup.

➤ Rmq

▪ Si un fragment est perdu :

- le réassemblage du paquet IP initial est interrompu lors de la « découverte » de la perte
- ... et le paquet complet est rejeté !

▪ Tous les fragments ont même champ Ident que le paquet initial

- sécurité supplémentaire lors du réassemblage (cf exemple)

▪ Si 2^{ème} bit du champ Flag = 1 (bit DF pour *Don't Fragment*)

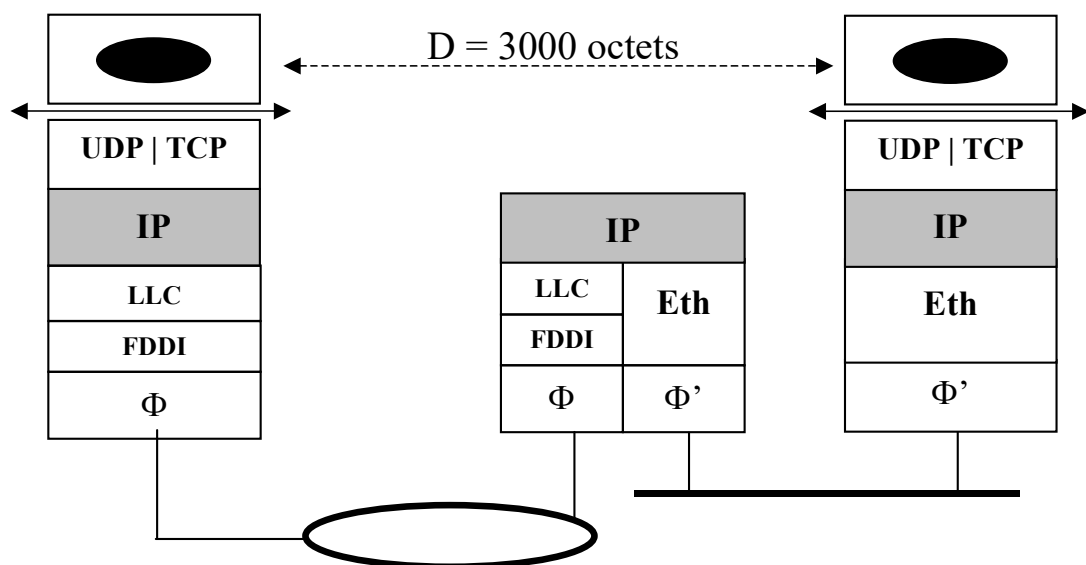
- alors : fragmentation interdite !

- un paquet IP trop grand dont le bit DF = 1 est rejeté et l'émetteur se voit notifier le rejet via un paquet ICMP

⇒ utilisé pour découvrir la + petite MTU sur le chemin (cf exemple)

▪ Exemple de fragmentation / réassemblage

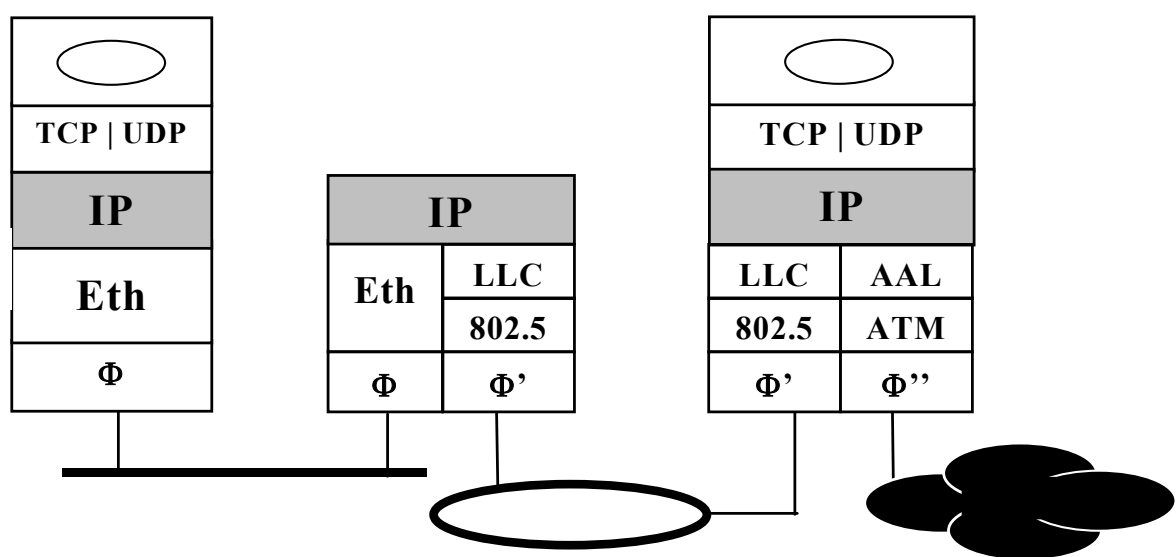
- Interconnexion d'un LAN Ethernet et d'un LAN FDDI



2.4. Adressage et algorithme de routage des paquets IP

▪ Rappel

- Toute machine de l'Internet possède au moins 2 adresses :
 - l'une l'identifiant de façon unique sur son réseau physique
 - ⇔ **adresse matérielle codée en dur sur la carte réseau**
 - l'autre l'identifiant de façon unique dans l'Internet adresse IP
 - ⇔ adresse logicielle attribuée lors de la configuration de la (carte réseau de la) machine
- Une @ IP désigne une seule machine (hôte ou routeur) mais une machine peut avoir plusieurs @ IP
 - cas des routeurs
 - cas + général d'une machine connectée à 2 (ou > 2) réseaux IP accessibles l'un et l'autre par TCP/IP



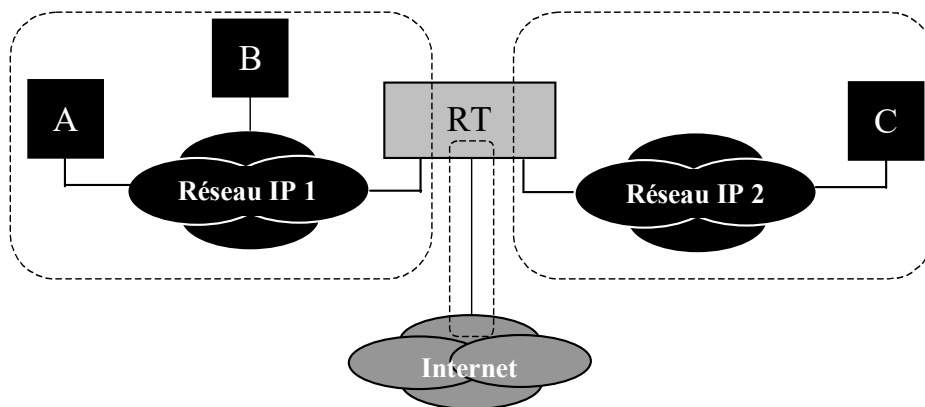
♦ Adressage global et algorithme de routage correspondant

1. Format d'une adresse IP (rappel)

▪ Dans l'Internet IP version 4 (IPv4 ou plus classiquement IP) :

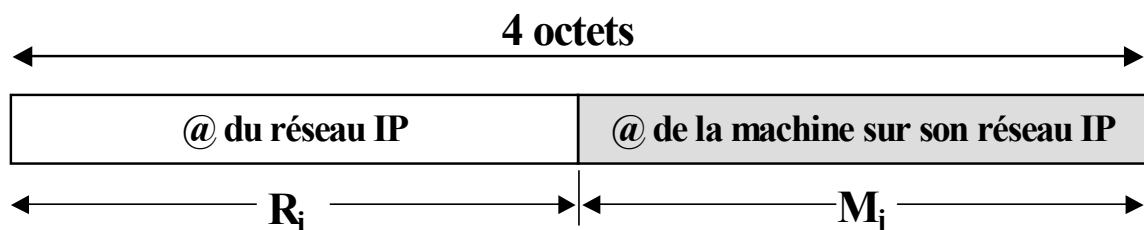
1) Codage des adresses IP sur 4 octets (notées $R_i . M_j$)

- identifiant de façon unique :
 - le **réseau IP** auquel appartient la machine (hôte ou routeur), parmi tous les réseaux IP de l'Internet
→ partie R_i de l'adresse
 - la **machine** parmi toutes les machines de son réseau IP
→ partie M_j de l'adresse



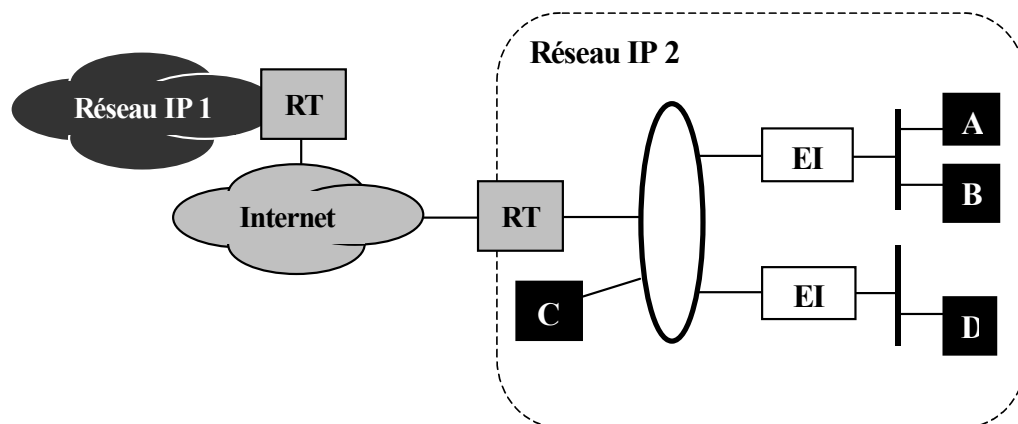
2) Initialement : répartition des @ IP en « classes d'adresse »

- différant selon le nombre de bits réservés au codage de R_i par rapport à ceux réservés au codage de M_j



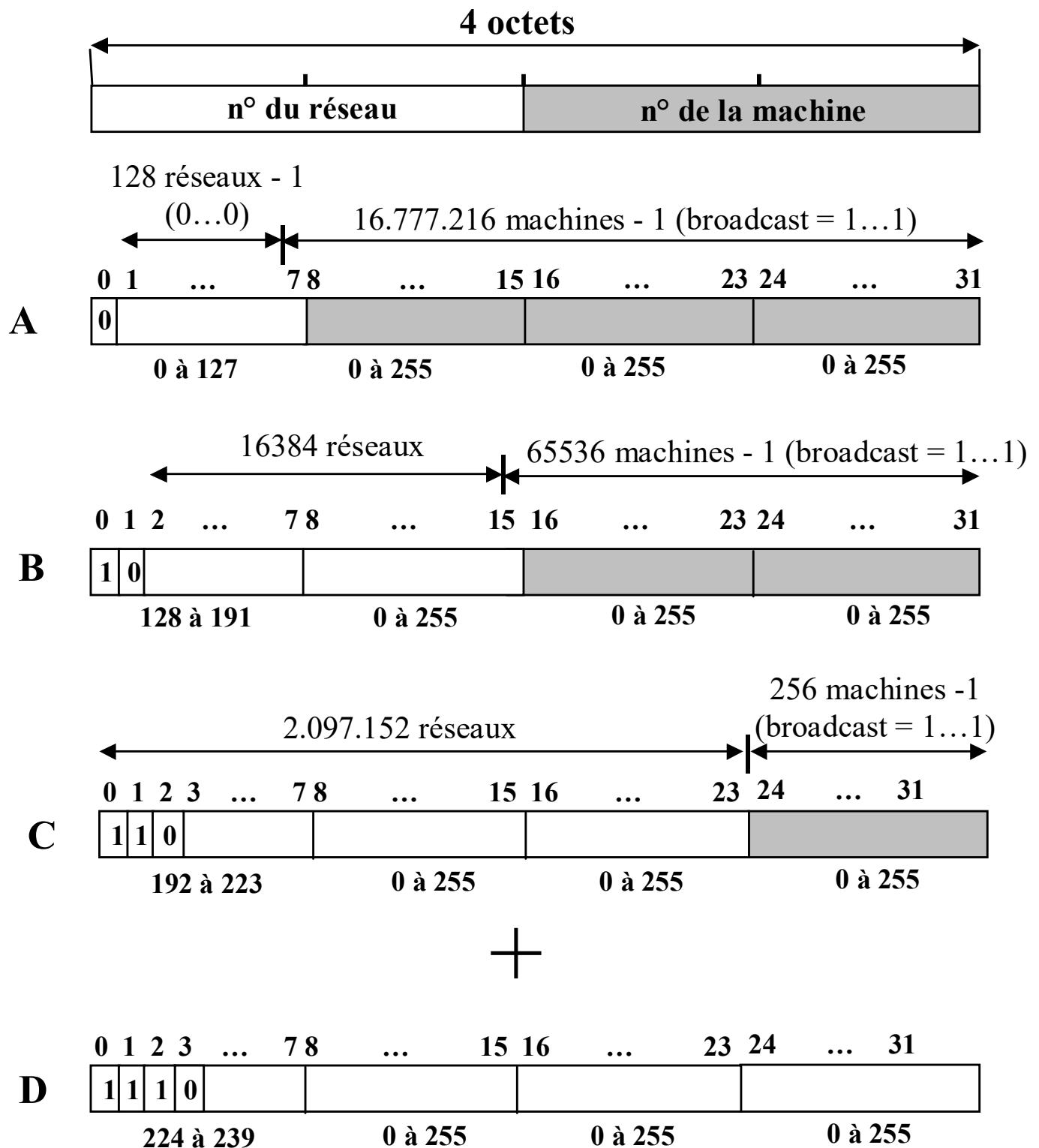
➤ Rmq

- **On représente toujours chacun des 4 octets (Ø) d'une @ IP sous forme décimale : 1^{er} Ø . 2^e Ø . 3^e Ø . 4^e Ø**
 - le 1^{er} (si classe A), les deux 1^{er} (si classe B) ou les trois 1^{er} (si classe C) désignant :
 - l'**@ du réseau IP** (noté $R_i.0$) auquel appartient la machine
 - Ex
 - @ IP de la machine *droopy* (LAAS) : 140.93.192.25
 - partie $R_i = 140.93$ – partie $M_j = 192.25$
 - @ IP de (feu ...) la machine *verlaine* (INSA) : 193.50.169.11
 - partie $R_i = 193.50.169$ – partie $M_j = 11$
- **Réseau IP $\Leftrightarrow$ réseau Φ ou une interconnexion de réseaux Φ ayant la MEME adresse de réseau IP ($R_i.0$)**
 - dans le 2^{ème} cas, les équipements d'interconnexion (EI) sont :
 - soit des répéteurs (/ hub) ou des ponts (/ switch Ethernet)
 - soit éventuellement des routeurs si *subnetting* (cf. 4^{ème} année)



▪ Les classes d'adresse IP => adresses IP dites « publiques »

– 4 classes majeures : A, B, C + classe D (adresses multicast)



➤ Rmq

- **Les adresses de réseau IP publiques (de classe A, B ou C) sont uniques à l'échelle de l'échelle l'Internet**
 - Elles ne sont attribuées qu'une seule fois par un organisme international = IANA (*Internet Assigned Numbers Authority*)
 - Plus globalement, l'IANA supervise :
 - l'allocation des adresses de réseau IP
 - l'allocation des numéros de systèmes autonomes (1 par opérateur)
 - cf. cours Interconnexion avancée de 4IR/SC
 - la gestion de la « zone racine » dans les Domain Name System
 - DNS : service de l'Internet qui permet de mettre en correspondance un nom logique et une adresse IP
 - cf. cours App. Internet du second semestre de 3A
- **Schéma d'adressage rapidement identifié comme « non résistant au facteur d'échelle » au regard de la croissance de l'Internet**

NB : en anglais : « not scalable »

➔ 2 approches de solutions envisagées :

- IPv6 : cf. cours 4IR / SC - Interconnexion avancée
- **Adressage « sans classe »** (ou adressage privé)

▪ L'adressage sans classe

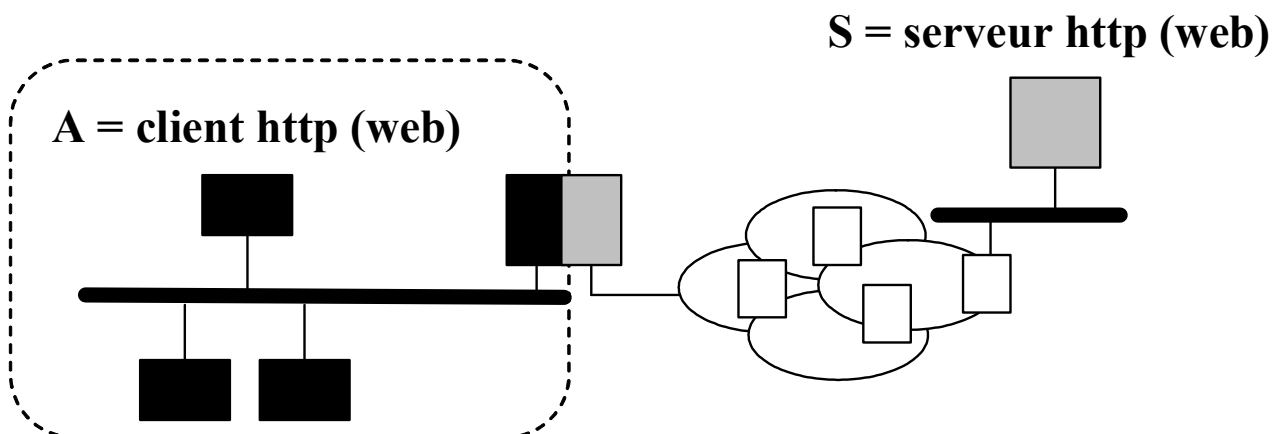
Face à la pénurie à venir des @ publiques, le choix a rapidement été fait de définir le **concept « d'adresses privée »** :

1] Les @ IP privées sont issues de 3 plages d'@ IP (RFC n° 1918) :

- toutes les @ de classe A de : 10.0.0.0 à 10.255.255.255
- toutes les @ de classe B de : 172.16.0.0 à 172.31.255.255
- toutes les @ de classe C de : 192.168.0.0 à 192.168.255.255

2] les @ IP privées sont utilisables par qui veut, mais avec une restriction importante :

- Une @ IP privée est « **non routable** », i.e. les @ privées ne sont pas reconnues par les routeurs
 - un paquet IP d'@IP **source ou destination** privée sera rejeté par tout routeur de l'Internet
 - NB : Inconvénient et éléments de solution
 - Comment communiquer avec l'extérieur ?!
 - Solution : NAT, proxy applicatif, VPN+NAT ➔ cf. section V.



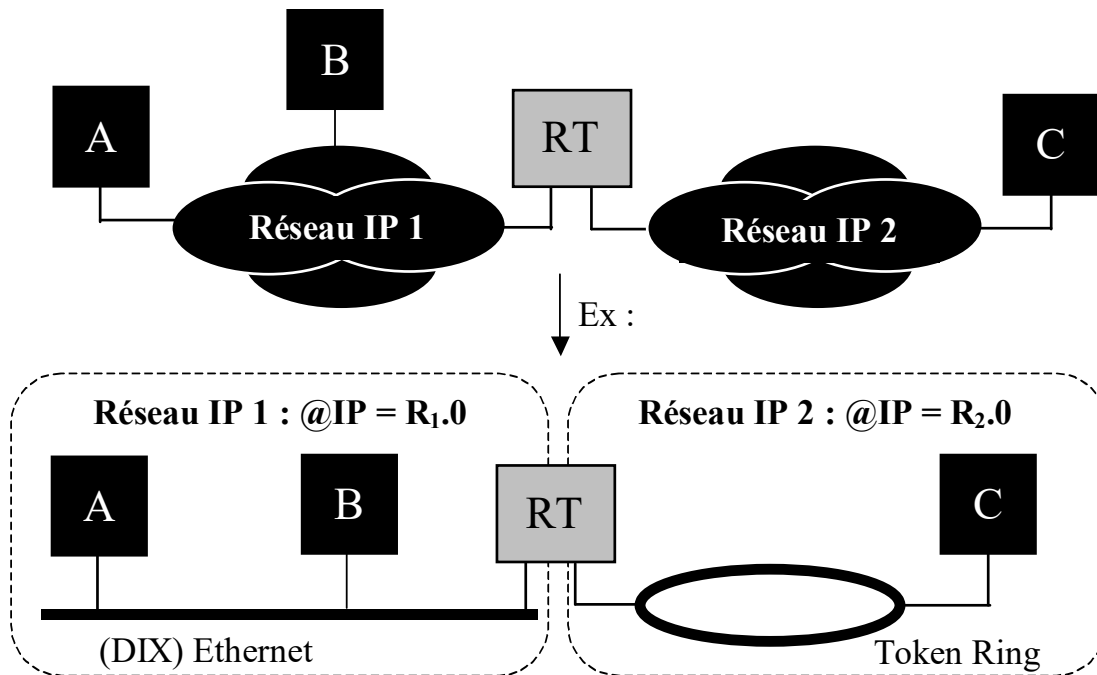
3] Dans l'@ssage privé, le concept de classe (A, B, C) n'existe plus

- L'administrateur peut placer où il le souhaite la frontière entre les parties Réseau IP (R_i) et Machine (M_i)
 - i.e. **sans avoir à respecter les frontières initialement imposées pour les adresses de classe A, B ou C**
- **Ex 1** : 192.168.0.0 peut être une adresse de réseau IP
 - en choisissant 16 bit (et non 24) pour adresser le réseau
 - on dit alors que le « **masque** » **du réseau** est sur 16 bit
 - Masque = séquence de 32 bits dont la valeur vaut :
 - 1 pour tous les bits de codage de la partie réseau IP (R_i)
 - 0 pour tous les bits de codage de la partie machine (R_j)
 - Ici : Masque = 255.255.0.0
 - On écrit aussi : @ réseau IP = 192.168.0.0 / **16**
 - 16 désignant le nb de bits réservés au codage de la partie R_i
 - NB : le masque sera utilisé dans l'algo. de routage (cf. ci-après)
- **Ex 2** : soit une adresse IP privée = 172.16.1.9 / 28
 - Quel est le masque de son réseau IP ?
 - Quelle est l'adresse de la partie réseau IP (R_i) ?
 - Quelle est l'adresse de la partie machine (M_j) ?

2. Algorithme de routage

1) Principe via un exemple

- Soit l'interconnexion de réseaux IP suivante



- Soit à router d'un paquet IP « P »

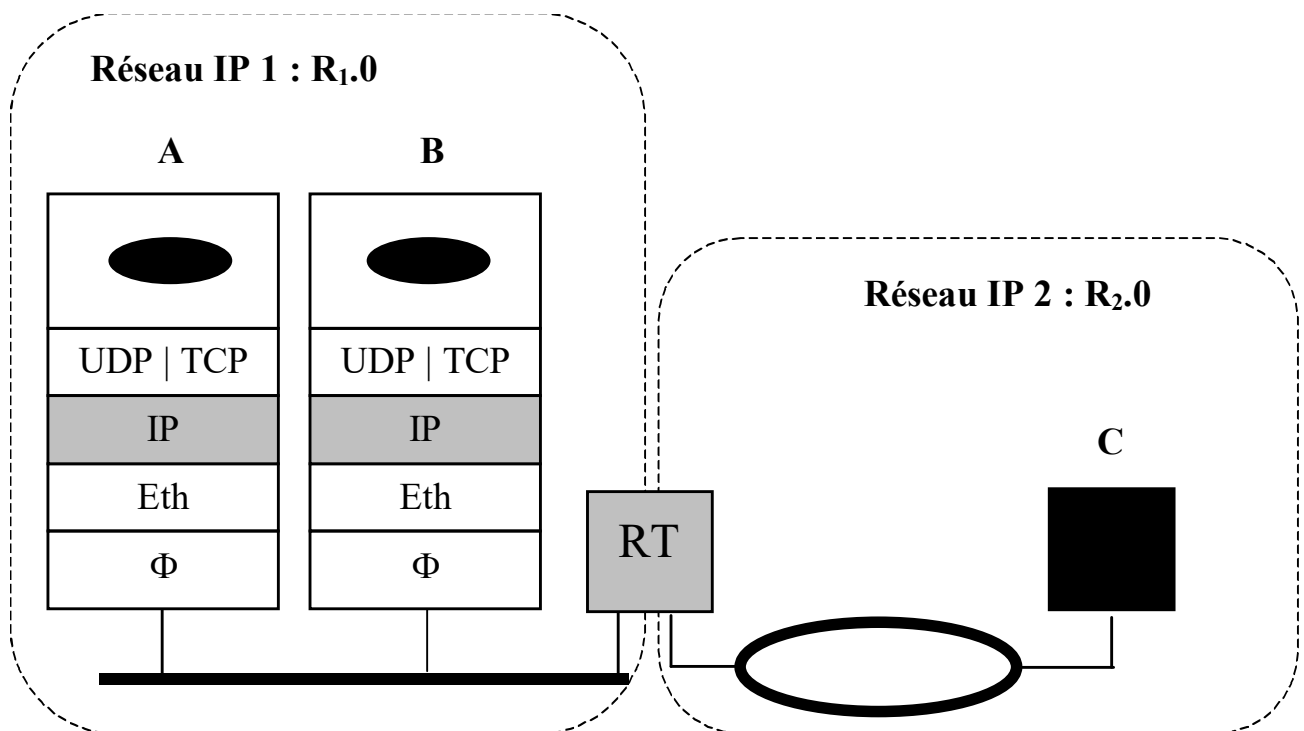
- émis par A à destination (au sens @ IP) de B ou C
- sachant que :

- $@IP_A = R_1.M_A / @MAC_A = e_A$
- $@IP_B = R_1.M_B / @MAC_B = e_B$
- $@IP_C = R_2.M_C / @MAC_C = tr_C$
- $@IP_{RT}$ sur le réseau IP $R_1 = R_1.M_{RT} / @MAC_{RT}$ sur son réseau physique (Ethernet) = e_{RT}
- $@IP_{RT}$ sur le réseau IP $R_2 = R_2.M_{RT} / @MAC_{RT}$ sur son réseau physique (Token Ring) = tr_{RT}

- Si P est émis à destination de B (i.e. $P.@IP_{dest} = R_1.M_B$)



Emission d'un paquet IP à destination d'une machine appartenant au même réseau IP que la machine émettrice



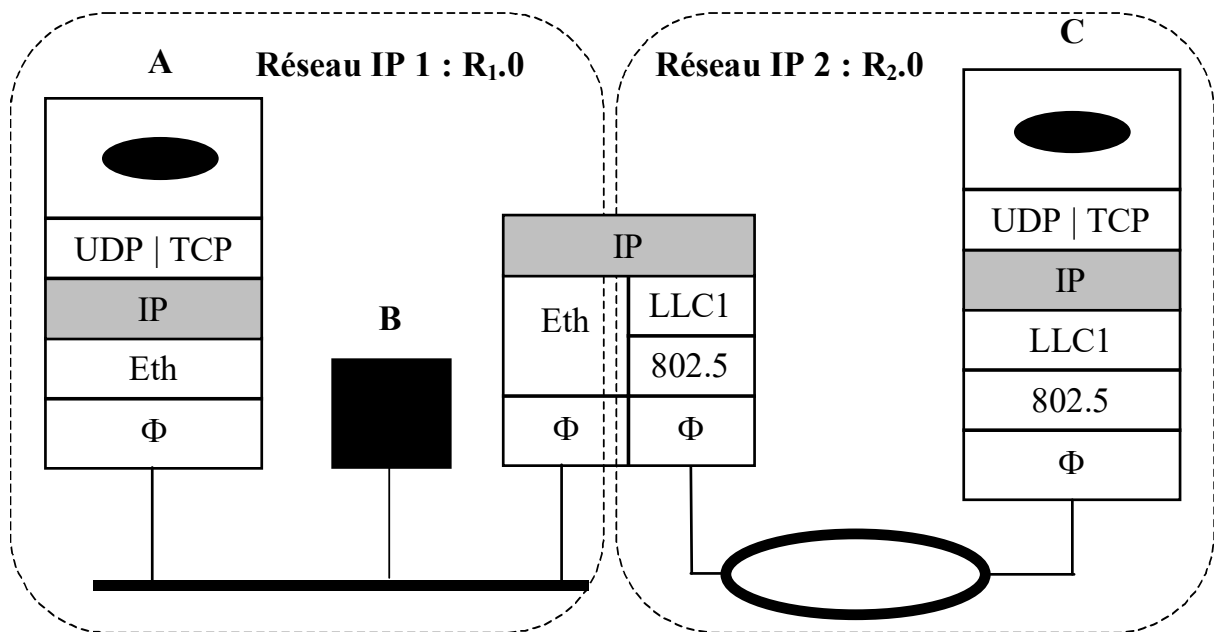
– Alors

- Encapsulation de P dans une trame Ethernet T émise à destination (au sens @ Ethernet) de B
 - acceptation de T par B car $e_B = T.@Eth_{dest}$
 - rejet de T par RT car $e_{RT} \neq T.@Eth_{dest}$

- Si P est émis à destination de C (i.e. $P.@IP_{dest} = R_2.M_C$)



Emission d'un paquet IP à destination d'une machine n'appartenant **pas** au même réseau IP que la machine émettrice



– Alors

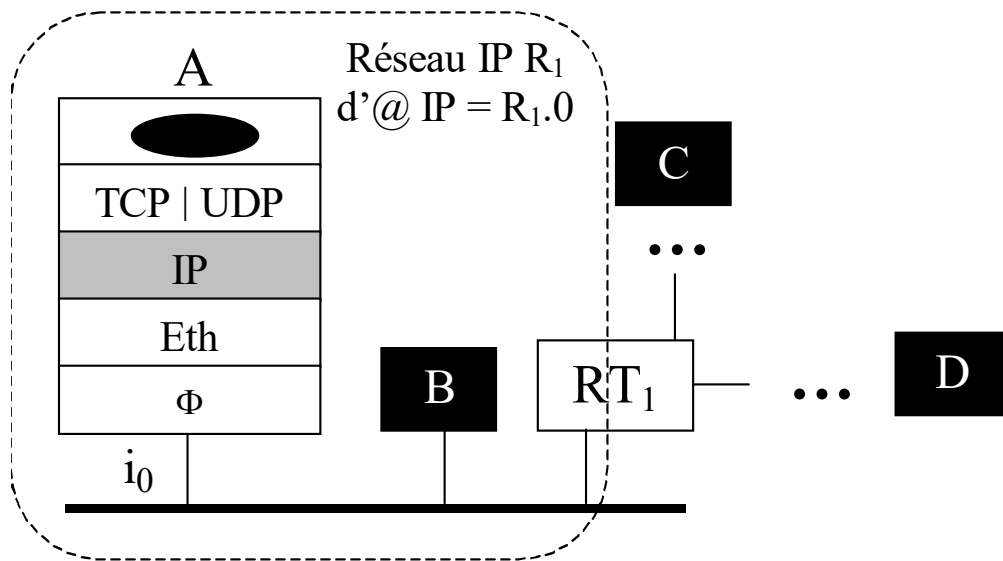
- Encapsulation de P dans une trame Ethernet T émise à destination (au sens @ Ethernet) de RT
 - rejet de T par B car $e_B \neq T.@Eth_{dest}$
 - acceptation de T par RT car $e_{RT} = T.@Eth_{dest}$
- Puis au niveau de RT :
 - encapsulation de P dans une trame Token Ring 802.5 T' (via LLC1) émise à destination (au sens @ Token Ring) de C
 - acceptation de T' par C car $tr_C = T'.@TokRing_{dest}$

2] Description de l'algorithme dans le cas général

- **Algorithme exécuté par un hôte (ici A) pour émettre un paquet IP « P » à destination d'un hôte H d'@ IP = $R_p.M_q$**

– Hyp

- le réseau IP R_1 se réduit à un LAN Ethernet (mais pourrait être une interconnexion de niveau 2 de plusieurs LAN)



– 2 cas de figure possibles

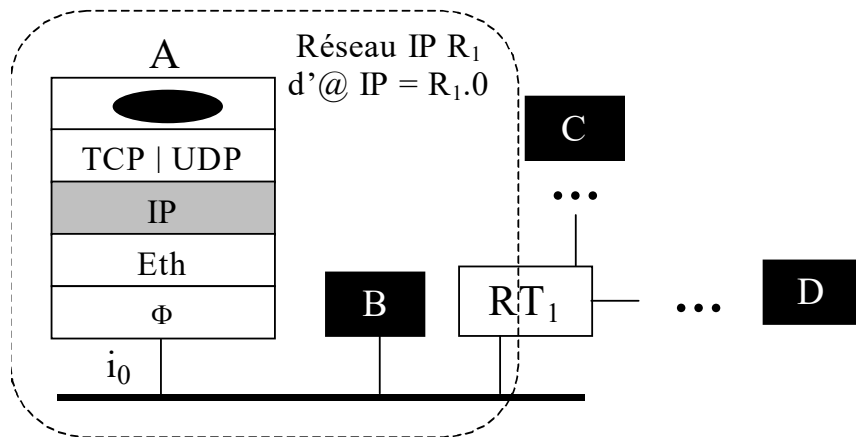
- $H \in$ même réseau IP que A (d'@ $R_1.0$)
 - P peut alors être transmis « directement » à H
 - ici : via une trame Ethernet émise à dest. de H
- $H \notin$ même réseau IP que A (d'@ $R_1.0$)
 - P doit transiter par un routeur intermédiaire du réseau IP $R_1.0$ (en l'occurrence RT₁) permettant de « sortir » du réseau IP R_1
 - ici : via une trame Ethernet émise à dest. de RT₁

- Pour déterminer le bon cas de figure :
 - l'entité IP de A consulte une « **table de routage** » **locale à A**
 - = tableau à n lignes et 3 colonnes principales
- chaque ligne comportant :
 - l'@ IP d'un réseau IP à atteindre (R.0)
 - l'@ IP d'un routeur du même réseau IP que A
- vers lequel faire transiter les paquets ayant à **sortir** du réseau R₁.0 pour se rapprocher du réseau d'adresse R.0
 - l'interface Φ **locale** à emprunter (car potentiellement plusieurs)

@ d'un réseau IP à atteindre	@ IP du prochain routeur intermédiaire	Interface Φ locale

- cas particuliers
 - si R.0 = R₁.0
- alors : l'@ IP indiquée dans la 2^{ème} colonne n'est pas à considérer puisque le destinataire est directement atteignable
- par convention : l'@ IP de A est copiée à cet endroit
 - existence obligatoire d'une entrée « *par défaut* »
 - existence obligatoire de l'entrée de « *loopback* » (127.0.0.1) pour la communication entre processus d'une même machine

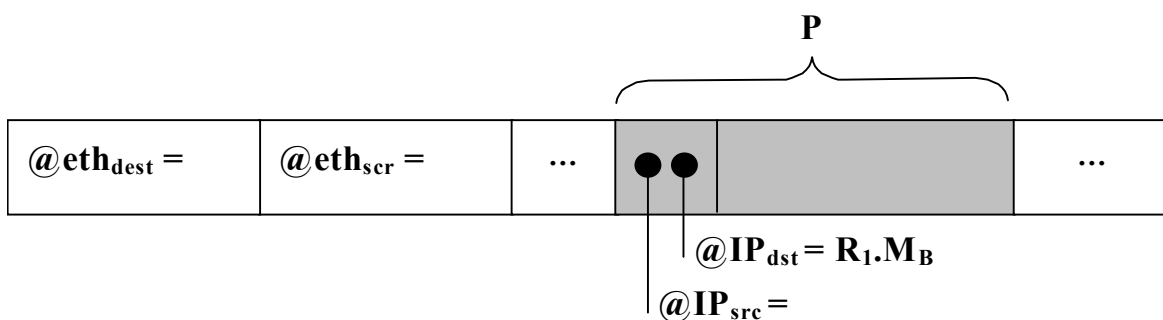
➤ **Ex** : Table de routage **minimale** de A



@ IP du réseau à atteindre	@ IP du prochain routeur intermédiaire	Interface Φ locale
127.0.0.1	127.0.0.1	lo ₀

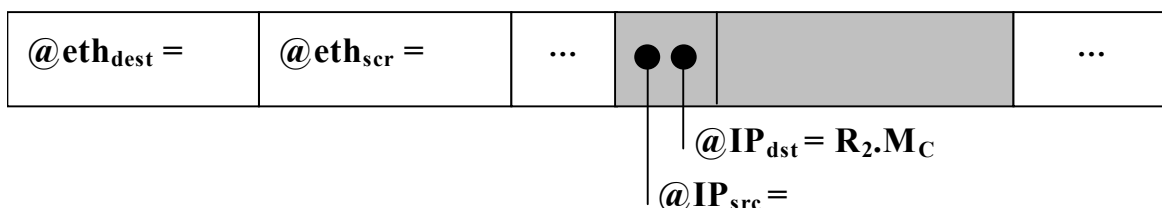
▪ **cas 1** : $@IP_{\text{dest}}$ du paquet P = $@IP_B (= R_1.M_B)$

– trame Ethernet générée sur le réseau :



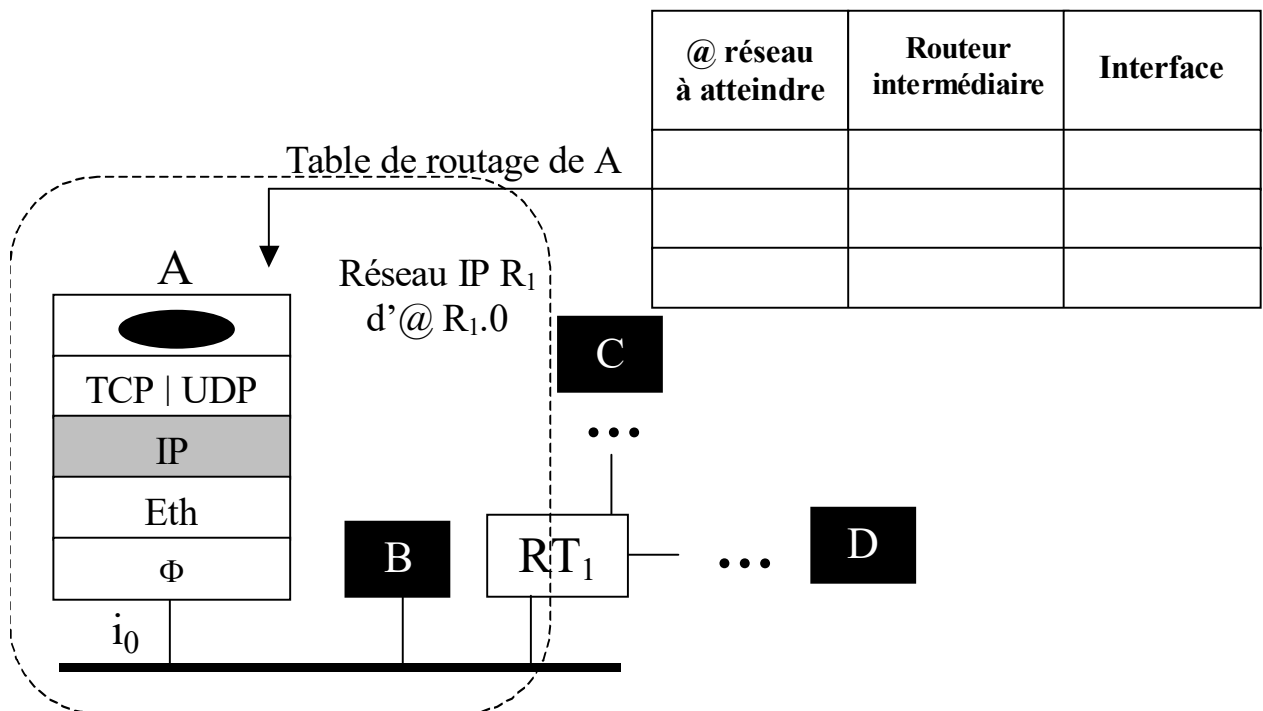
▪ **cas 2** : $@IP_{\text{dest}}$ du paquet P = $@IP_C (R_2.M_C)$

– trame Ethernet générée sur le réseau :



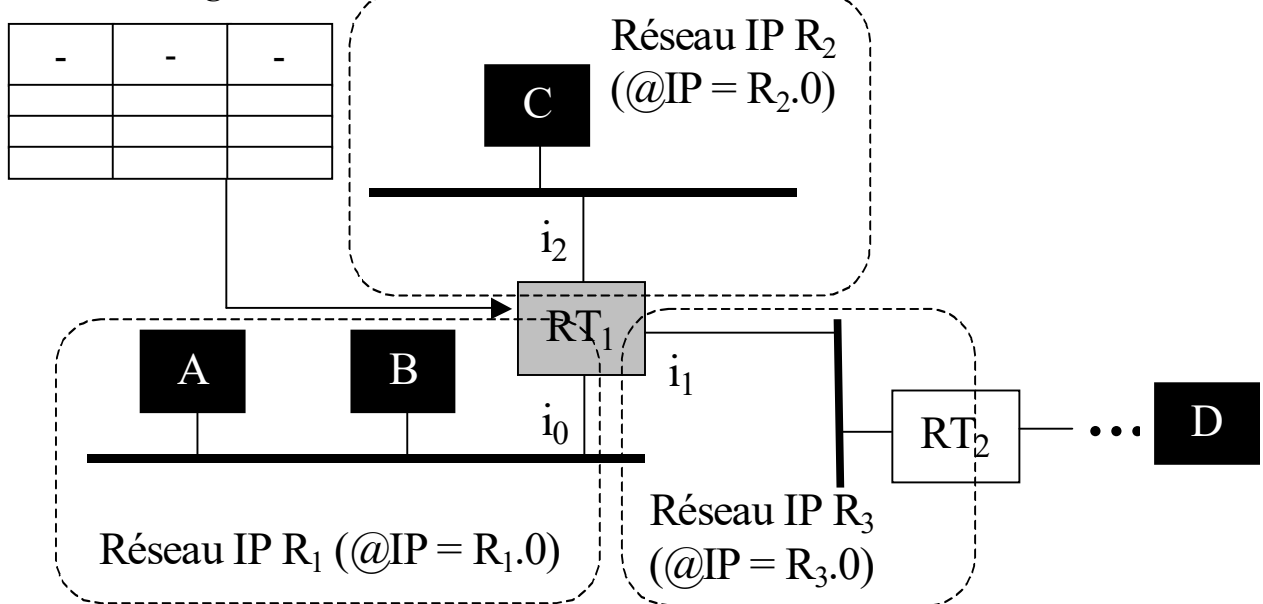
– Algorithme d'utilisation de la table

- Pour émettre le paquet P d'@ IP_{dest} = R_p.M_q, l'E(IP) de A :
 - consulte ligne à ligne sa table de routage
 - s'arrête sur l'entrée identifiant comme @ de réseau IP à atteindre :
 - soit R_p.0
 - soit *default* si aucune entrée n'identifie explicitement R_p.0
 - transmet le paquet via une trame (ici Ethernet) sur l'interface Φ associée à l'entrée sélectionnée
 - soit directement à destination (@ Eth) du hôte destinataire
 - soit à destination (@ Eth) du prochain routeur intermédiaire associé à l'entrée sélectionnée



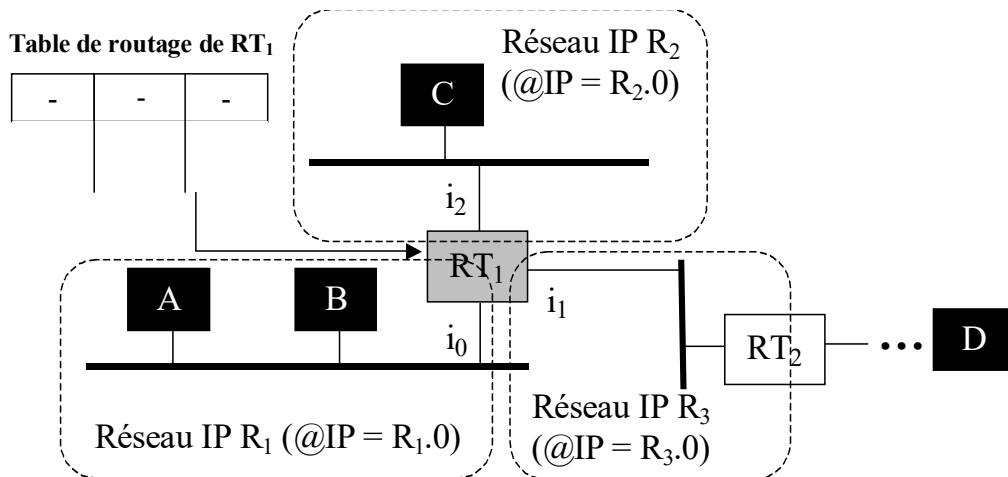
- **Algo exécuté par un routeur** (ici RT_1) recevant sur l'une de ses interfaces (ici i_0) un paquet IP « P » d'@ destination = $R_p.M_q$

Table de routage de RT_1



- Recevant P sur l'interface i_0 , l'E(IP) de RT_1 :
 - consulte ligne à ligne sa table de routage (**table locale à RT_1**)
 - s'arrête sur l'entrée identifiant comme @ de réseau IP à atteindre :
 - soit $R_p.0$
 - soit *default* si aucune entrée n'identifie explicitement $R_p.0$
 - transmet P via une trame Ethernet sur l'interface associée à l'entrée retenue :
 - soit directement à destination (@ Eth) du hôte destinataire
 - si celui-ci est situé sur **l'un des autres réseaux IP** d'appartenance du routeur
 - soit à destination (@ Eth) du prochain routeur intermédiaire associé à l'entrée sélectionnée

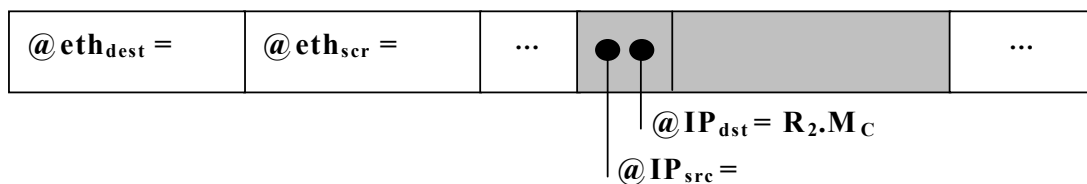
➤ **Ex** : Table de routage **minimale** de RT_1



@ IP du réseau à atteindre	@ IP du prochain routeur intermédiaire	Interface Φ
127.0.0.1	127.0.0.1	lo ₀

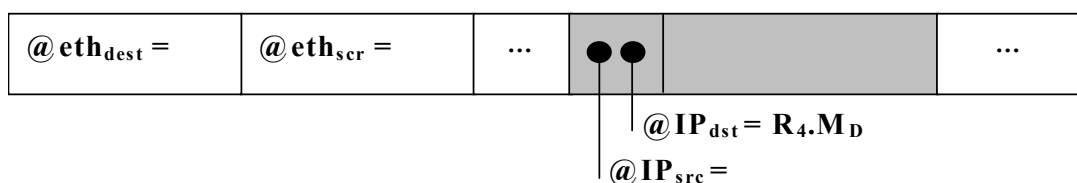
▪ **cas 1** : $@IP_{\text{dest}}$ du paquet $P = @IP_C (= R_2.M_C)$

– trame Ethernet générée sur le réseau par RT_1 :

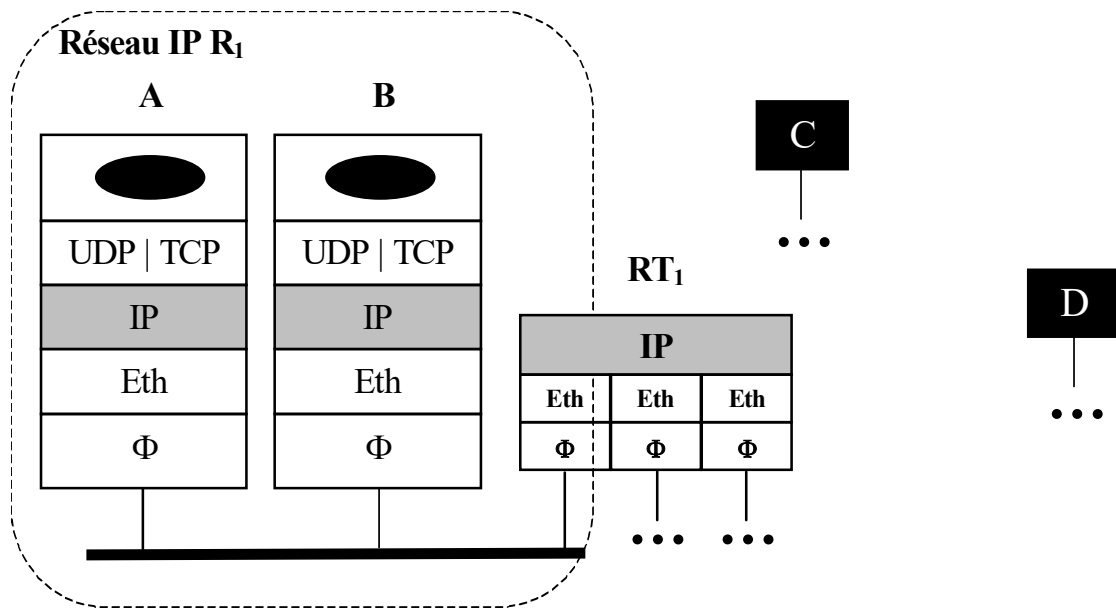


▪ **cas 2** : $@IP_{\text{dest}}$ du paquet $P = @IP_D (R_4.M_D)$

– trame Ethernet générée sur le réseau par RT_1 :



➤ **Rmq importante**



▪ **Lors de l'émission par A du paquet IP P à destination :**

- soit de B (cas 1)
- soit de C ou D via le routeur RT₁ (cas 2)

la trame Ethernet encapsulant P doit être émise à destination :

- soit de B dans le cas 1 → $@Eth_{dest} = @Eth_B$
- soit de RT₁ dans le cas 2 → $@Eth_{dest} = @Eth_{RT1}$

▪ **Sachant que le programme applicatif émetteur (ou initiateur de la connexion si TCP) désigne le programme récepteur par :**

- un n° de port + une adresse IP

➔ **la question qui se pose alors est :**

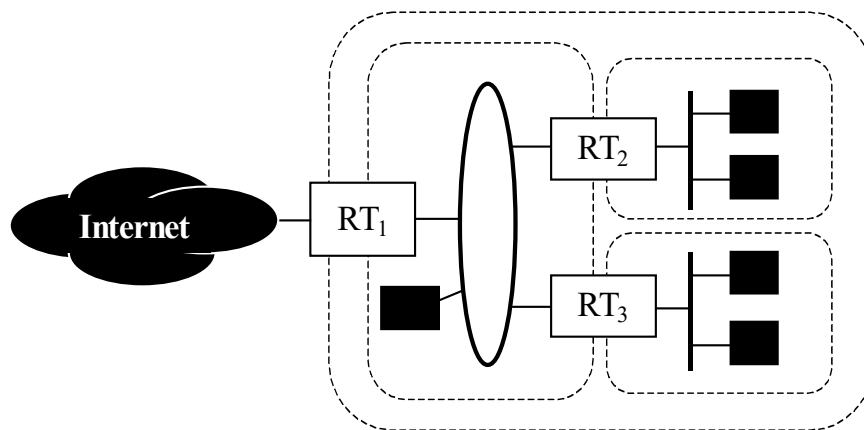
- d'où provient l'@ Eth de destination de la trame encapsulant P ??

➔ cf suite du cours pour la réponse (ARP)

2.5. Le routage au sein de sous-réseaux IP (*Subnetting*)

◆ Notions de sous réseau IP (*subnet IP*) et de *subnetting*

- Soit un site possédant une **unique** adresse de réseau IP et dans lequel on souhaite installer deux routeurs internes
 - **Question** = comment éviter de devoir attribuer une @ de réseau IP à chaque réseau situé de part et d'autre d'un routeur interne ?



▪ Idée

- 1] considérer que les réseaux Φ d'un même réseau IP situés de part et d'autre d'un routeur **INTERNE** au site
 - font partie de « sous réseaux IP » différents identifiables localement par une « @ de sous-réseau IP »
 - Rmq : localement $\Leftrightarrow$ uniquement au sein du réseau IP
- 2] gérer le routage des paquets IP entre ces sous-réseaux comme s'il s'agissait de réseaux IP à part entière

et ce, tout en manipulant **une seule @ de réseau IP** :

- celle du site (classe A, B ou C), seule à être reconnue par les routeurs externes aux sites

$\Leftrightarrow$ gérer du « **subnetting** » au sein du réseau IP

♦ Identification d'un sous réseau (au sens « adressage d'un SR »)

▪ Constat

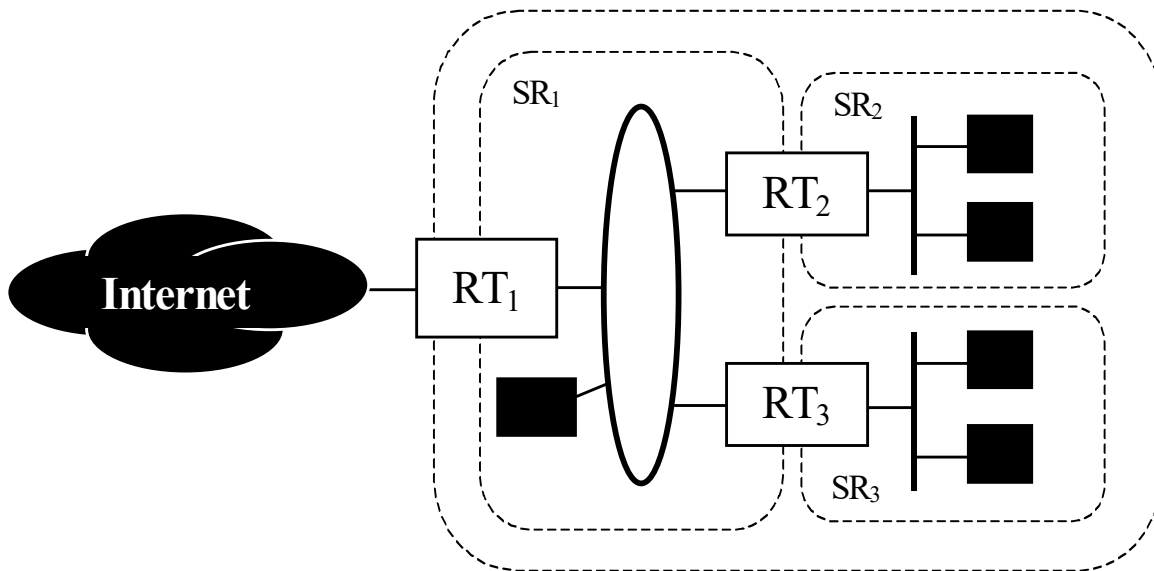
- Format d'une @ IP classique :
 - $R_i.M_j$ avec gestion locale de la partie M_j
- Or :
 - le nombre de bits de codage de M_j est souvent **SUPERIEUR** au nb nécessaire pour identifier toutes les machines du réseau
 - ex : 140 . 93 . 25 . 32

▪ Pour construire une @ de sous-réseau (SR)

- **Idee** = réduire l'espace d'adressage des machines
 - et affecter les bits « économisés » à l'identification des SR
- Concrètement :
 - l'@IP du réseau = R . 0, avec :
 - 16 ou 8 bits réservés au codage des machines (classes B / C)
 - devient :
 - @IP du réseau = R . SR . 0 avec :
 - x bits réservés au codage des sous réseaux
 - 16-x ou 8-x bits réservés au codage des machines selon que le réseau est de classe B ou C

➤ **Ex**

- Soit un réseau de classe B (140.93.0.0) que l'on souhaite diviser en 3 sous-réseaux comportant au plus 1000 machines chacun



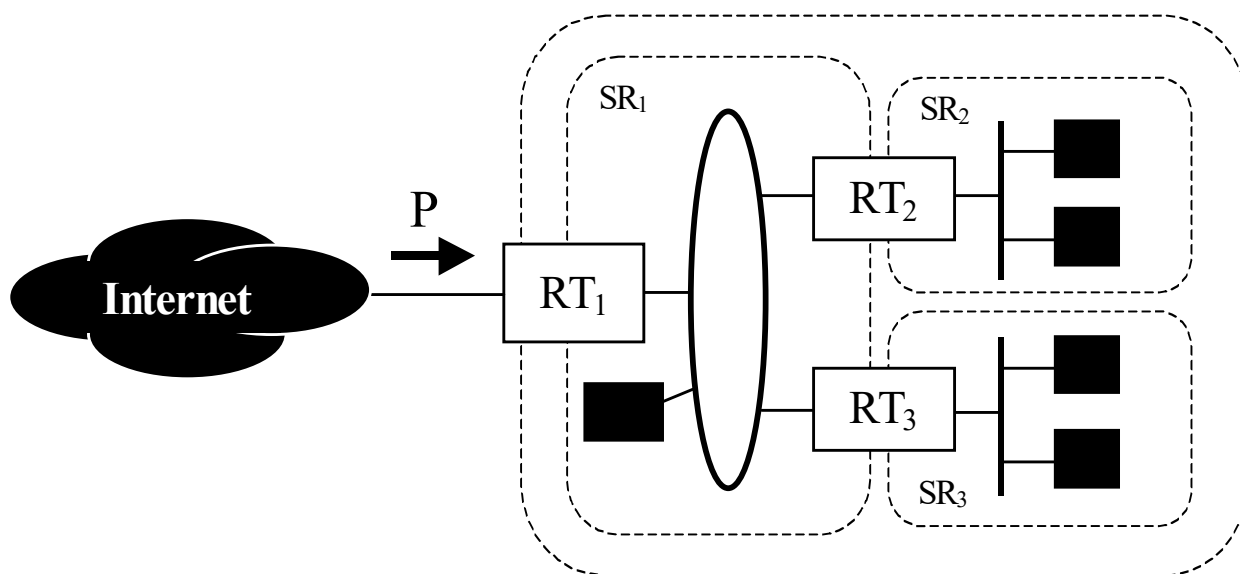
- Proposer l'affectation d'une adresse de sous réseau à chacun des sous-réseaux SR₁, SR₂ et SR₃
- Proposer un schéma d'adressage des machines de chacun des sous-réseaux SR₁, SR₂ et SR₃

◆ Masque de sous-réseau

■ Problème

- Soit un réseau IP d'adresse R.0
- Recevant un paquet IP « P » de l'extérieur
 - comment RT₁ déduit-il s'il doit :
 - transmettre directement P sur SR₁
 - ou aiguiller P vers SR₂ ou SR₃ en passant par RT₂ ou RT₃

???



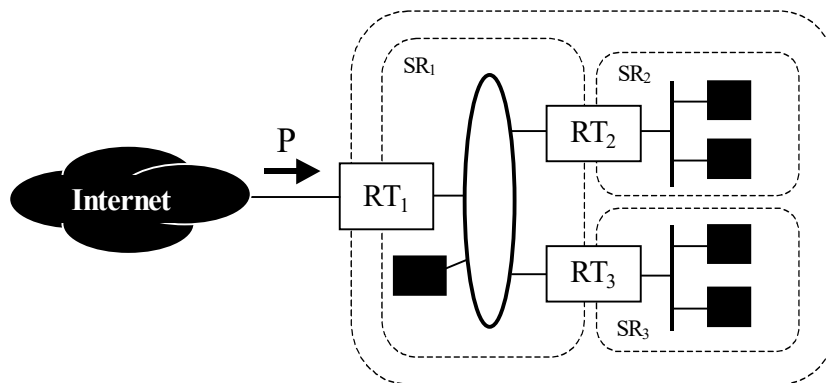
■ Solution

- repose sur l'attribution d'un « **masque de sous-réseau** » à chaque sous réseau SR_i permettant de conclure par oui / non à la question :
 - l'@IP_{dest} de P identifie-t-elle une machine du sous réseau SR_i ?

▪ Masque d'un sous-réseau

- Séquence de 32 bits ayant pour valeurs :
 - 1 pour tous les bits de codage des @ de réseau et de sous réseau IP
 - 0 pour tous les bits de codage de l'@ machine
- **Pour déterminer si une @ IP désigne une machine d'un sous-réseau SR donné** (i.e. dont on connaît l'adresse et le masque)
 - effectuer un « ET » logique entre le masque de SR et cette @
 - si le résultat du « ET » = @ du sous réseau SR
 - alors : l'@ IP désigne une machine de SR, sinon : non

➤ Ex (reprise de l'ex. précédent)



- Donner le masque des sous réseaux SR_1 , SR_2 et SR_3 précédents
- Soit 140.93.11.73 l'@IP_{dest} du paquet P reçu de l'extérieur par RT₁
 - 140.93.11.73 identifie-t-elle une machine de SR_1 , SR_2 ou SR_3 ?

♦ Algorithme de routage de sous-réseau (*subnetting*)

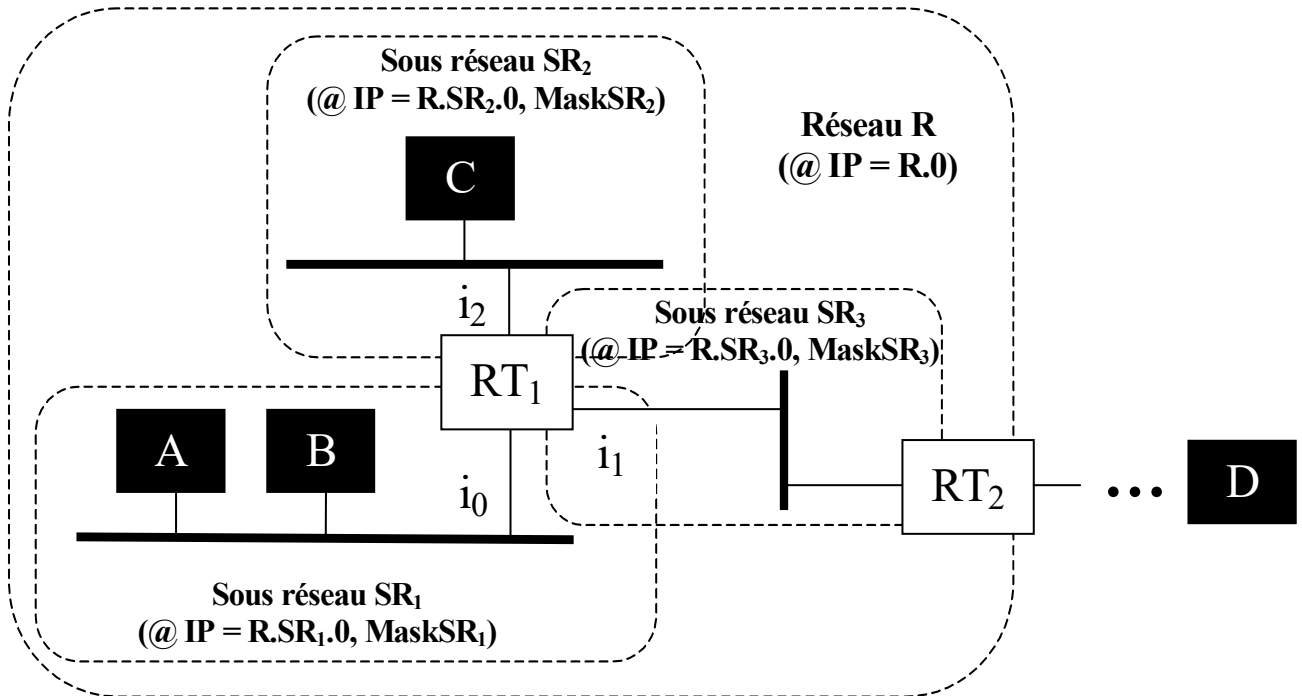
■ Besoins supplémentaires par rapport à l'algorithme initial

- Chaque hôte doit connaître :
 - le masque du sous-réseau auquel il est connecté
- Chaque routeur doit connaître :
 - le masque de chacun des sous-réseaux auxquels il est connecté
 - + celui des autres sous-réseaux mentionnés dans sa table de routage

■ Rmq IMPORTANTE

- Lors de la configuration d'une interface physique
 - nécessité de spécifier le masque du sous-réseau à l'aide de la commande « **ifconfig** »
 - permet de faire correspondre une adresse IP et un masque à une adresse physique → cf TP et « **man ifconfig** »
 - S'il n'y a pas de sous-réseau (cas fréquent !)
 - on attribue comme valeur de masque :
 - 255.255.0.0 si réseau de classe B
 - 255.255.255.0 si réseau de classe C
- ⇔ aucun bit réservé à l'identification des sous-réseaux
- Autrement dit
 - Sous réseau ou pas, il y a toujours un masque et on applique l'algorithme de routage incluant la manipulation des masques

- Soit à router un paquet IP « P » émis par le hôte A du réseau IP R à destination d'un hôte H = B, C ou D



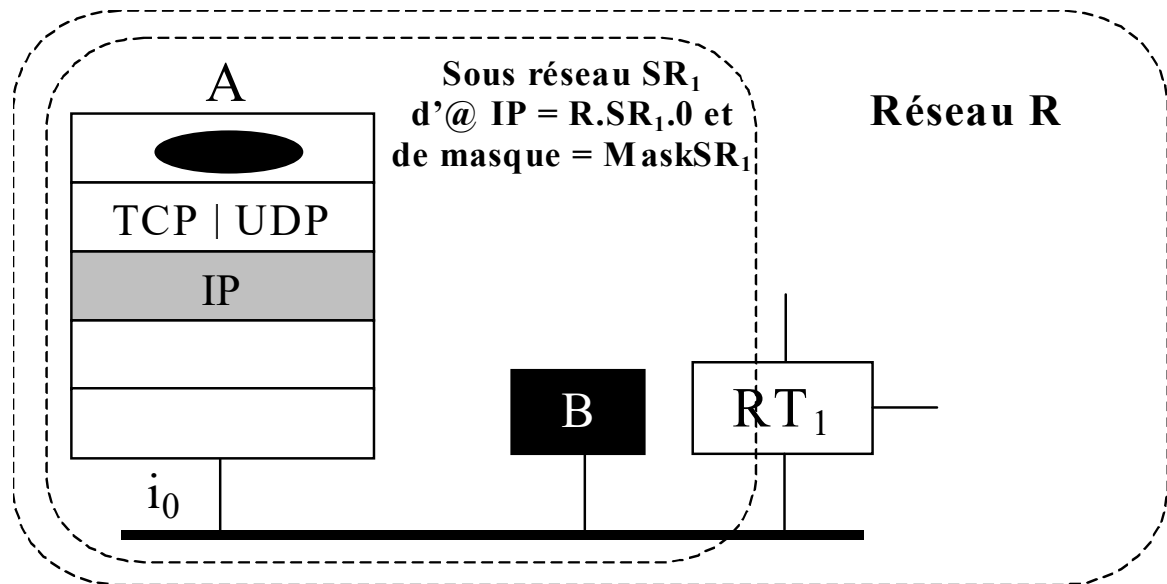
– Hyp

- Le réseau R est constitué de 3 sous-réseaux SR₁, SR₂ et SR₃
- A, B, C ∈ réseau IP R (@IP = R.0)
- D ∉ réseau IP R
- A et B ∈ même sous réseau IP SR₁ (@IP = R.SR₁.0)
- C ∈ sous réseau IP SR₂ (@IP = R.SR₂.0)
- RT₁ ∈ sous réseau IP SR₁, SR₂ et SR₃
- RT₂ ∈ sous réseau IP SR₃

1] Algo exécuté par un hôte (ici A) pour émettre un paquet IP (ici P) à destination d'un hôte H d'@ IP = $R_p.M_q$

▪ **Hyp**

- A $\in$ sous réseau SR_1 de masque $Mask_{SR_1}$



- 2 cas de figure possibles :

- $H \in SR_1$ = sous-réseau IP de A

- P peut alors être transmis « directement » à H

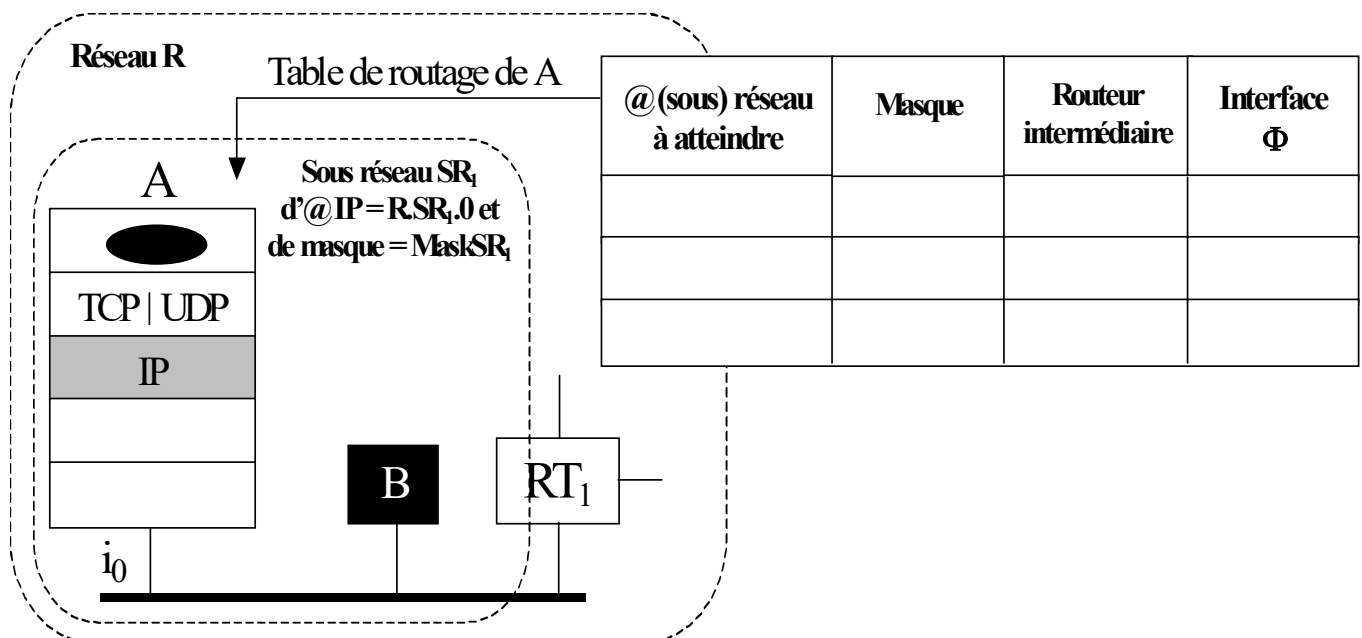
- ici : via une trame Ethernet émise à destination de H

- $H \notin SR_1$

- P doit transiter par un routeur intermédiaire (ici RT_1) permettant de « sortir » du sous-réseau SR_1

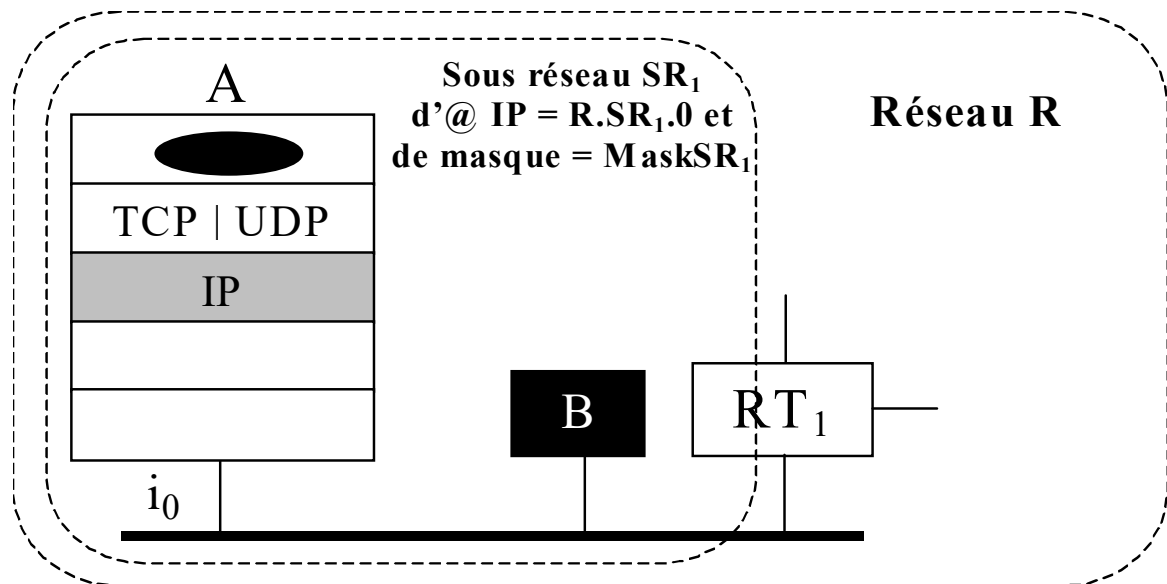
- ici : via une trame Ethernet émise à destination de RT_1

- Pour déterminer le bon cas de figure, l'E(IP) de A :
 - consulte ligne à ligne sa table de routage
 - s'arrête sur l'entrée ayant comme @ de (sous) réseau IP à atteindre :
 - soit $[R_p.M_q \text{ ET Mask}]$
 - où Mask désigne le masque associé à l'entrée
 - soit **default** si aucune des entrées n'identifie explicitement $[R_p.M_q \text{ ET Mask}_{SR}]$ comme @ de (sous) réseau IP à atteindre
 - transmet P via une trame Ethernet sur l'interface Φ associée à l'entrée sélectionnée
 - soit directement à destination (@ Eth) du hôte destinataire
 - soit à destination du prochain routeur intermédiaire associé à l'entrée sélectionnée



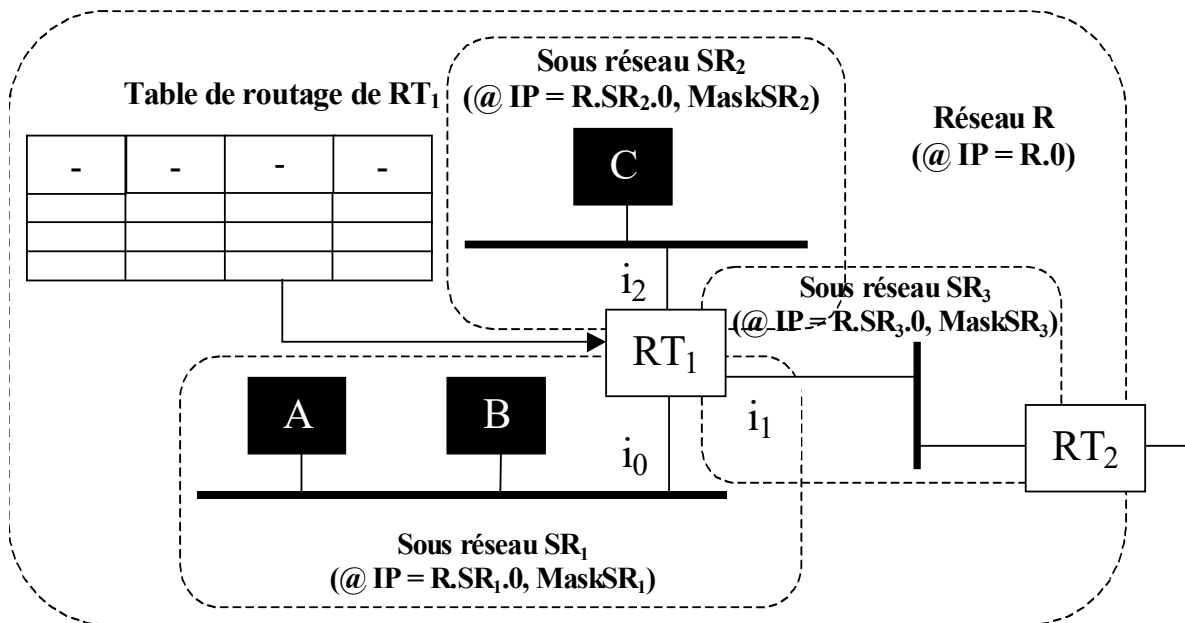
➤ Ex

▪ Table de routage minimale de A



@ IP du réseau (SR) à atteindre	Masque	@ IP du prochain routeur	Interface Φ locale
$R.SR_1.0$	$MaskSR_1$	$R.SR_1.M_A$	i_0
default	-	$R.SR_1.M_{RT1}$	i_0
127.0.0.1	-	127.0.0.1	lo_0

2] Algo exécuté par un routeur RT (ici RT_1) recevant sur l'une de ces interfaces (ici i_0) un paquet IP (ici P) d'@ dest = $R_p.M_q$

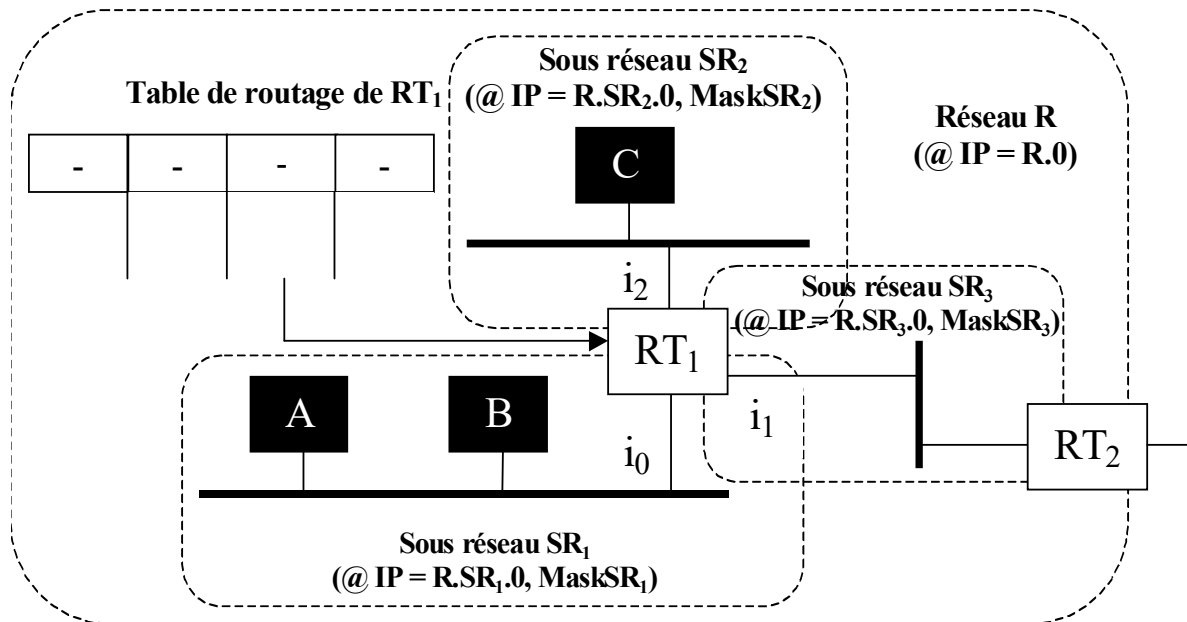


▪ Recevant P sur l'interface i_0 , l'E(IP) de RT_1 :

- consulte ligne à ligne sa table de routage
- s'arrête sur l'entrée ayant comme @ de (sous) réseau IP à atteindre :
 - soit $[R_p.M_q \text{ ET Mask}]$
 - où Mask désigne le masque associé à l'entrée
 - soit **default** si aucune des entrées n'identifie explicitement $[R_p.M_q \text{ ET Mask}_{SR}]$ comme @ de (sous) réseau IP à atteindre
- transmet P sur l'interface associée à l'entrée sélectionnée :
 - soit directement à destination du hôte destinataire
 - soit à destination du prochain routeur intermédiaire associé à l'entrée sélectionnée (ici : RT_2 sur SR_3)

➤ **Ex**

▪ **Table de routage minimale de RT₁**



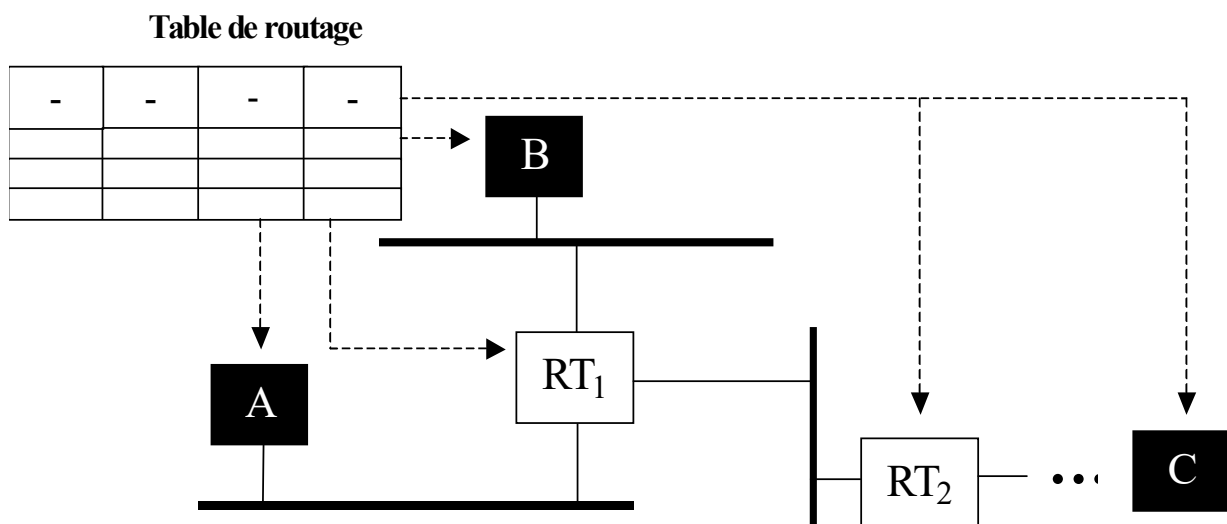
@ IP du réseau (SR) à atteindre	Masque	@ IP du prochain routeur intermédiaire	Interface Φ locale
R.SR ₁ .0	MaskSR ₁	R.M _{RT1} sur SR ₁	<i>i</i> ₀
R.SR ₂ .0	MaskSR ₂	R.M _{RT1} sur SR ₂	<i>i</i> ₂
R.SR ₃ .0	MaskSR ₃	R.M _{RT1} sur SR ₃	<i>i</i> ₁
default	-	R.M _{RT2} sur SR ₃	<i>i</i> ₁
127.0.0.1	-	127.0.0.1	lo ₀

3. Protocoles de routage (courte introduction)

3.1. Introduction

▪ L'algorithme de routage des paquets IP

- repose sur la consultation d'une **table de routage**
 - **locale à un hôte** pour traiter un paquet IP « sortant »
 - **locale à un routeur** pour traiter un paquet IP « entrant »
- permet l'acheminement des paquets IP de routeur en routeur depuis le hôte émetteur jusqu'au hôte récepteur

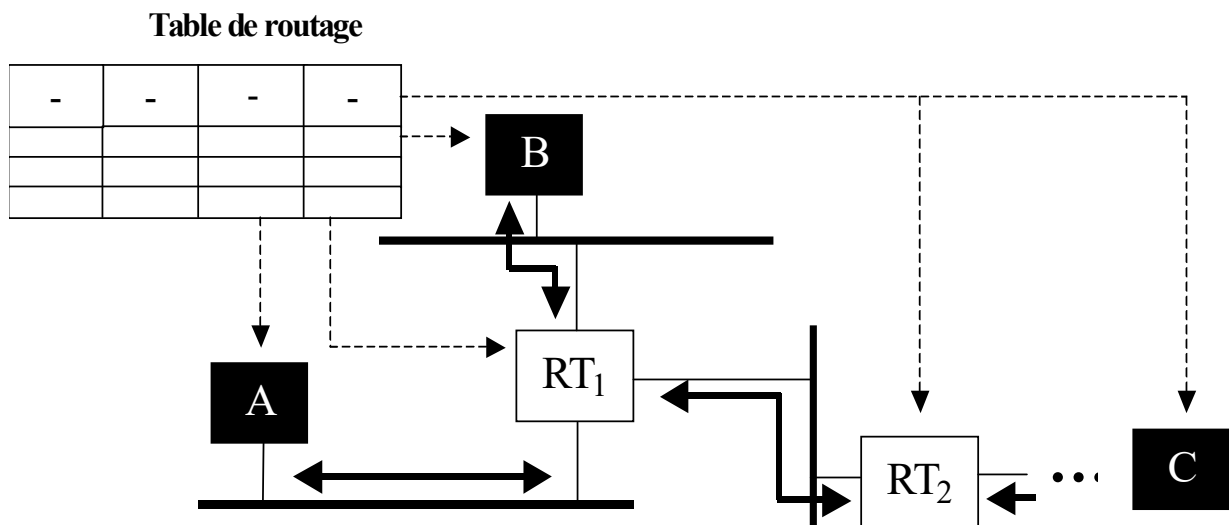


▪ Question

- Comment ces tables de routage sont-elles mises à jour de sorte qu'au bout de compte :
 - tout paquet IP émis soit reçu par l'hôte destinataire ?!
 - et ne erre pas indéfiniment dans l'Internet ...

■ Réponse

- Fournie par les « **protocoles de routage** »
 - dont l'objectif est de mettre à jour **dynamiquement** les tables de routage des hôtes et des routeurs
 - en fonction de l'évolution des réseaux (apparition, panne, ...)
 - basés sur l'échange de PDU contenant des « info de routage »
 - réseaux atteignables en passant par le RT émetteur du PDU, ...
 - dont la mise en œuvre implique :
 - soient les routeurs seulement, soient les routeurs et les hôtes



▪ La notion de système autonome

– Internet

- Interconnexion d'un ou plusieurs réseaux IP au sein d'un site
- Mais aussi :

– l'interconnexion de ces sites via :

- une liaison spécialisée (LS)
- un backbone (ATM par ex.)
- ...

– On désigne par « **système autonome** » (AS : *Autonomous System*)

- toute région de l'Internet sous l'autorité administrative d'une seule et même entité :

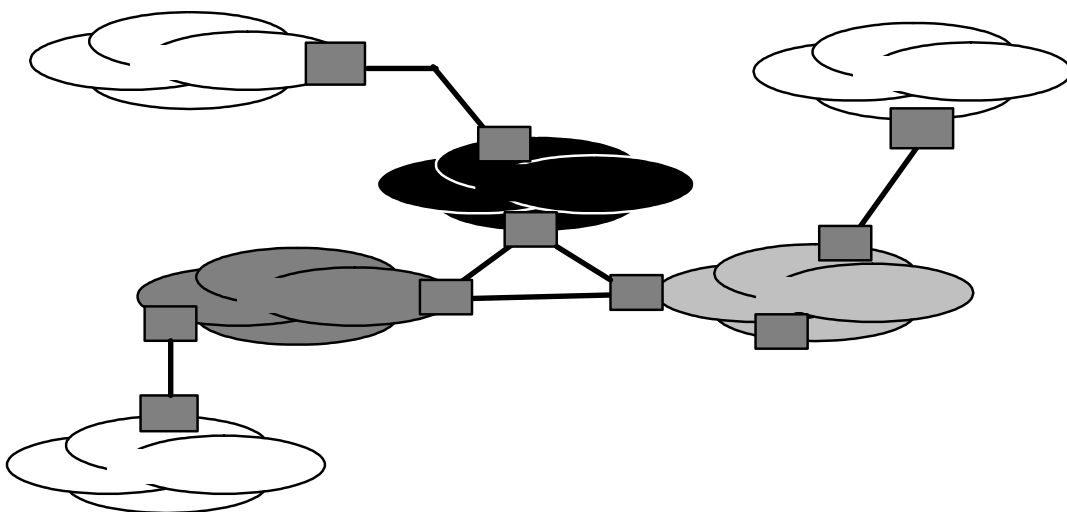
– sites d'extrémité tels que : INSA, LAAS, ...

- utilisateurs de leurs propres services Internet
- Rmq importante : ils ne représentent pas le cas majoritaire !

– opérateurs tels que : AOL, SPRINT, ..., potentiellement :

- fournisseurs de services Internet (ISP) à des clients
- fournisseurs d'accès à des ISP (satellite par ex)
- ...

- **En conséquence, on distingue 2 types de protocole de routage**
 - les protocoles de routage « **intra domaine** » (intra AS)
 - sous la responsabilité unique de l'autorité qui gère le domaine
 - échange d'info de routage **au sein du domaine** uniquement
 - par le biais de protocole de routage intra-domaine
 - IGP : Interior Gateway Protocol
 - ex : RIP, OSPF, ...
 - les protocoles de routage « **inter domaine** » (inter AS)
 - échange d'info de routage entre les $\neq$ domaines via leur(s) routeur(s) « de bordure »
 - par le biais de protocole de routage inter-domaine :
 - EGP : Exterior Gateway Protocol
 - ex : BGP, ...

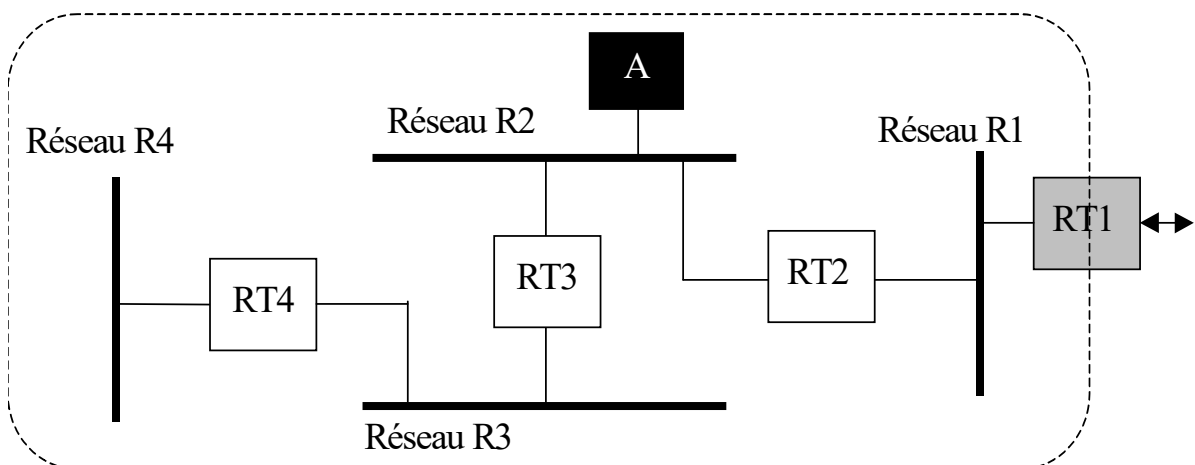


3.2. Le protocole de routage intra domaine RIP

♦ Introduction

- **On distingue 2 familles de protocoles intra domaines, fondés :**
 - soit l’algo du « vecteur de distance » (*Distance Vector*)
 - soit sur l’algo de l’« état de liaison » (*Link State*)
- **Principe des protocoles de type « Distance Vector »**
 - repose sur la diffusion par chaque routeur, sur chacun de ses (sous) réseaux IP d’appartenance, d’une partie de sa table de routage
 - en l’occurrence, pour chaque entrée de la table :

« réseau atteignable, coût pour atteindre ce réseau en passant par moi »
 - les diffusions étant périodiques ou bien faisant suite à une modification de la table
 - routeurs et hôtes d’un même (sous) réseau IP mettent à jour leur table de routage en fonction des info qu’ils récupèrent

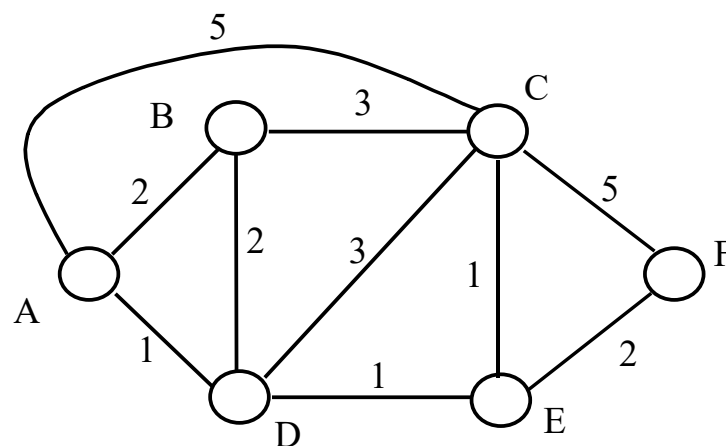


▪ Principe des protocoles de type « Link State »

- A l'opposé du routage par vecteur de distance
 - le routage par état de liaison repose sur la connaissance par tous les routeurs de la « **carte** » du réseau

⇔ base de données représentative :

- de tous les liens entre 2 routeurs adjacents qcq du réseau
- du coût de chacune de ces liens



- Sur ces bases, chaque routeur « choisit » le meilleur chemin pour aller d'un point à un autre
 - par application de l'algo de Dijkstra

◆ Principe du protocole RIP : Routing Information Protocol

■ RIP = protocole intra domaine le plus simple

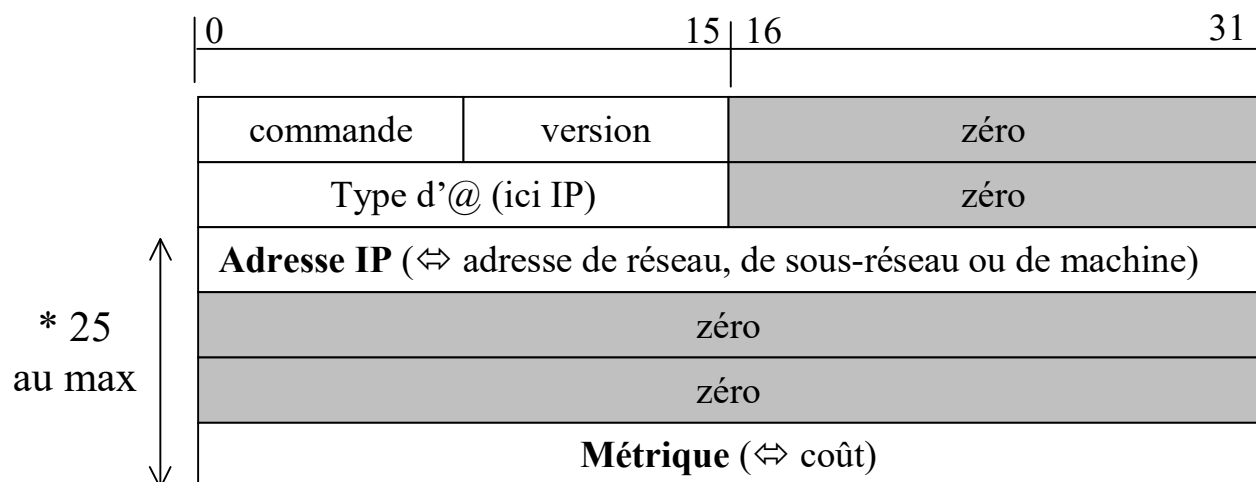
- règles très proches de l'algo du vecteur de distance

■ Implémentation de RIP = pgme « *routed* »

- pgme applicatif qui utilise UDP pour véhiculer ses PDU (port 520)
- cf. TP n°2 « routage dynamique entre réseaux IP proches »

■ Principes généraux

- Périodiquement, chaque routeur diffuse sur chacun de ses réseaux IP d'appartenance un PDU RIP contenant :
 - la liste des réseaux qu'il peut atteindre
 - le coût en nombre de routeurs à traverser (y compris lui-même) pour aller à chacun de ces réseaux (valeur max = 15, 16 ⇔ infini)



▪ Principes généraux (suite)

- 1) Sur réception d'un PDU RIP sur l'une de ses interfaces
 - Toute machine (hôte ou routeur) met à jour sa table de routage suivant les règles suivantes :
 - a) Si le PDU comporte une info
sur un réseau non répertorié dans sa table
 - accessible à **un coût non infini**, alors :
 - rajout de l'entrée correspondante dans la table
 - activation du timer associé à l'entrée
 - b) Si le PDU comporte une info
sur un réseau déjà répertorié dans sa table
 - accessible à **un coût moindre par un autre routeur** que celui mentionné dans la table, alors :
 - remplacement de l'entrée existante par la nouvelle entrée
 - activation du timer associé à la nouvelle entrée
 - accessible **par le même routeur** que celui mentionné dans la table, alors :
 - modification du coût de l'entrée existante ($\forall$ sa valeur)
 - activation du timer associé à l'entrée
 - c) En dehors des cas précédents, le PDU est rejeté

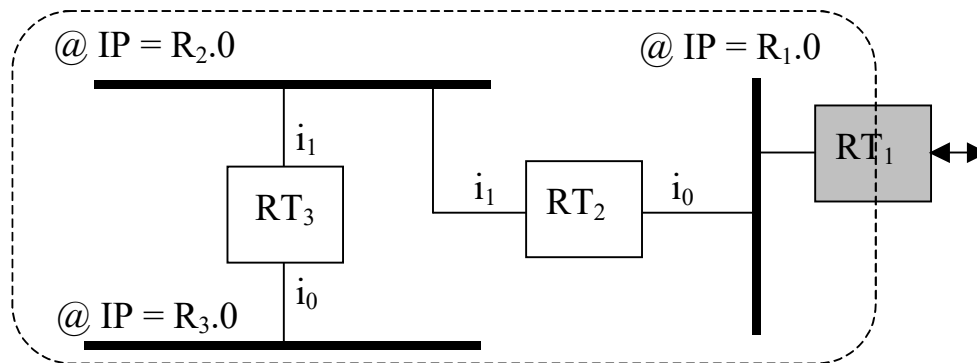
- 2) Sur expiration du timer associé une entrée
 - Toute machine (**hôte ou routeur**) met à jour sa table de routage
 - en positionnant à 16 ($\Leftrightarrow$ infini) le coût associé à l'entrée

- 3) Tout routeur qui met à jour sa table de routage
 - doit ensuite générer sur chacune de ses interfaces un PDU RIP donnant le nouvel état de la table

- 4) In fine, l'algorithme converge
 - i.e. seuls des PDU RIP « périodiques » sont émis par les routeurs
 - mais n'entraînant pas de mise à jour

➤ Ex

- A $t = t_0$



– Hyp

- Table de routage de RT_2

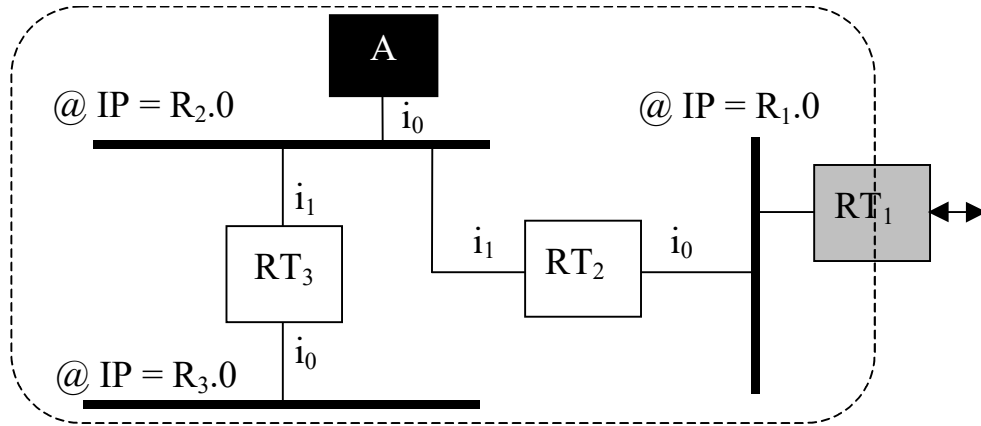
Réseau IP	Prochain Routeur	Interface	Coût
$R_1.0$	@IP de RT_2 sur $R_1.0$	i_0	0
$R_2.0$	@IP de RT_2 sur $R_2.0$	i_1	0
$R_3.0$	@IP de RT_3 sur $R_2.0$	i_1	1
Default	@IP de RT_1 sur $R_1.0$	i_0	1

- Table de routage de RT_3

Réseau IP	Prochain Routeur	Interface	Coût
$R_1.0$	@IP de RT_2 sur $R_2.0$	i_1	1
$R_2.0$	@IP de RT_3 sur $R_2.0$	i_1	0
$R_3.0$	@IP de RT_3 sur $R_3.0$	i_0	0
Default	@IP de RT_2 sur $R_2.0$	i_1	2

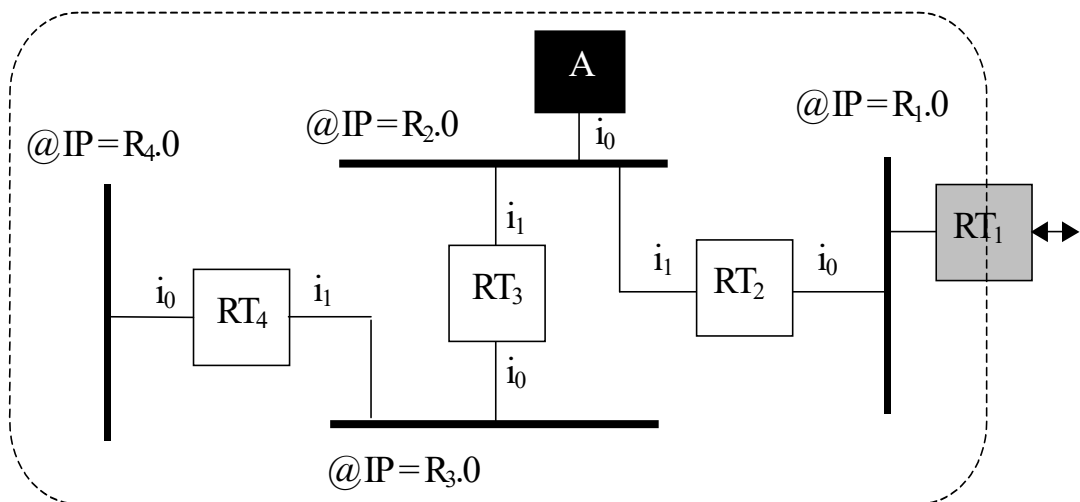
- **A t = t₁**

- A est connectée au réseau R₂.0



- **A t = t₂**

- R₄ est connecté au réseau R₃ par le biais de RT₄

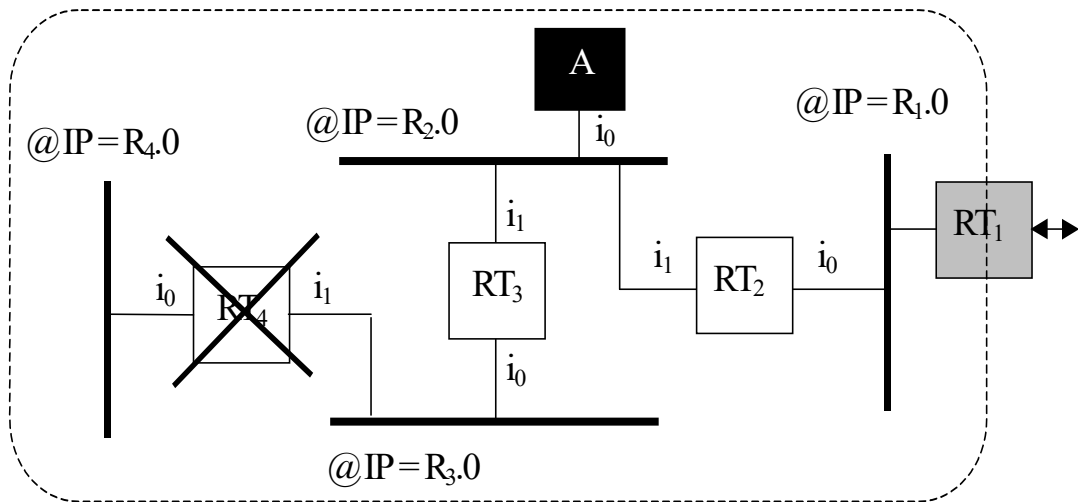


- **Question**

- Décrire, en conséquence de la réception de PDU RIP, l'évolution des tables de routage des machines du réseau (hormis RT₁) à t₁ puis à t₂

- **Cas particuliers**

- $\exists$ des circonstances amenant l’algo à converger très lentement
- Ex :

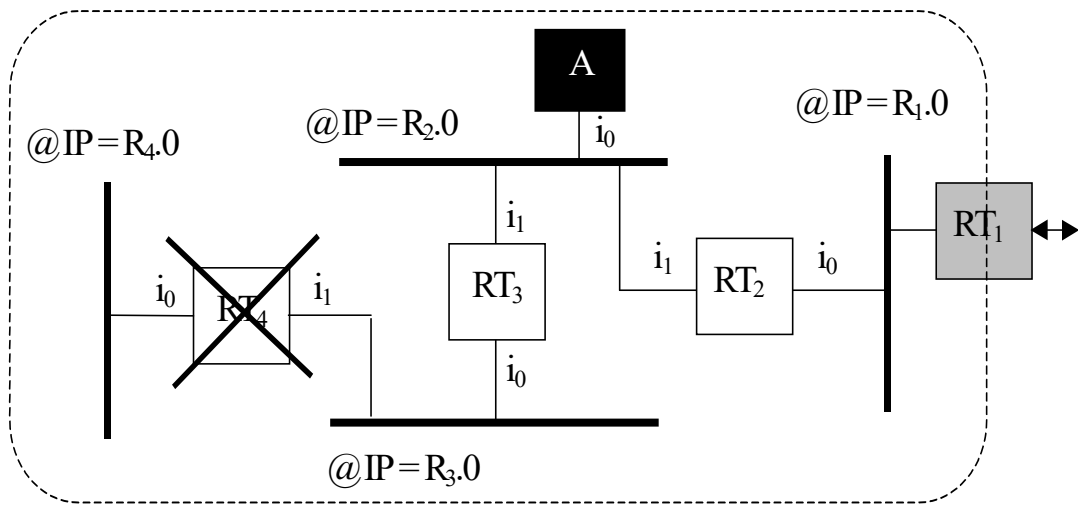


- **Hyp** : à $t > t_2$ et après stabilisation des tables de RT_2 , RT_3 et A :
 - RT_4 tombe en panne
- Conséquences
 - L'entrée de la table de RT_3 associée au réseau $R_{4.0}$ expire
 - car non rafraîchie par RT_4 et rafraîchissement de RT_2 rejeté
 - d'où :
 - modification de la table de routage de RT_3 : coût correspondant à l'entrée concernée $\leftarrow 16$
 - émission d'un PDU RIP indiquant la modification et mise à jour de tables de A et RT_2 en conséquence
 - **Problème** : si avant de recevoir ce PDU, RT_2 génère un PDU RIP périodique → **convergence lente de l'algorithme !!**

▪ Règles supplémentaires du protocole RIP

– Règle de l'horizon coupé (« Split Horizon »)

- Lorsqu'un routeur génère un PDU RIP sur une interface i_j :
 - il ne mentionne pas dans le PDU les infos qu'il a obtenues d'un PDU reçu sur l'interface i_j



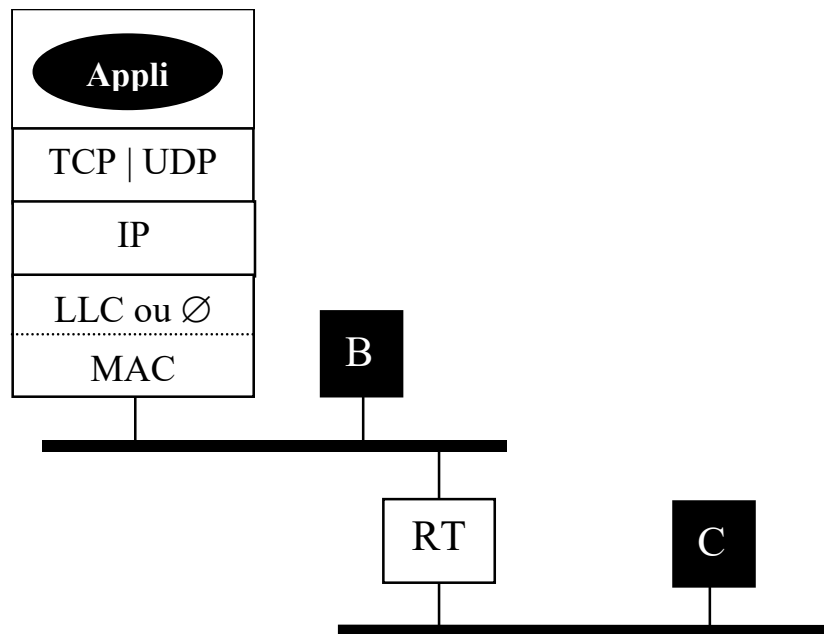
➤ Rmq

- $\exists$ d'autres circonstances qui ralentissent la convergence de l'algo
 - et dont qui impliquent l'émission de beaucoup de trafic RIP
 - $\Leftrightarrow$ limites de RIP
- Face (en particulier) à ces problèmes :
 - les protocoles de type « Link State », tel que OSPF
 - couramment utilisé par les opérateurs

4. ARP, RARP/DHCP, Proxy ARP et ICMP

4.1. Le protocole ARP

◆ Position du problème



- **Toute machine $\in$ LAN connecté à l'Internet possède :**
 - une @ physique (@ MAC) l'identifiant sur son réseau
 - une @ IP l'identifiant dans l'Internet
- **Toute donnée émise par un pgme appli de l'Internet engendre :**
 - la construction d'un segment TCP ou d'un datagramme UDP
 - puis la construction d'un paquet IP
 - puis la construction d'une trame MAC ou LLC/MAC
 - et enfin la génération de cette trame sur le support

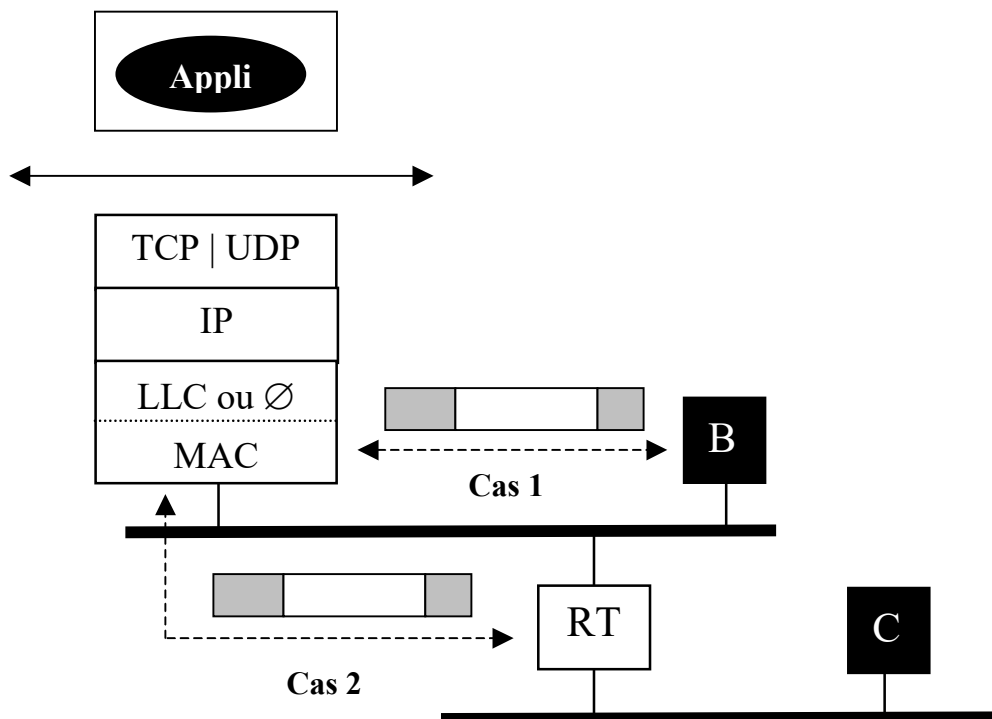
■ Problème

- L'identification du programme applicatif récepteur est effectuée par le programme émetteur qui fournit au système de com. sous-jacent :
 - l'@ IP de la machine sur laquelle s'exécute le pgme récepteur
 - un n° de port = n° port d'écoute utilisé par le pgme récepteur



– Question

- Comment le système de communication émetteur procède-t-il pour mettre à jour l'@ MAC_{dest} de la trame générée :
 - soit à destination de l'hôte destinataire (cas 1) ?
 - soit à destination d'un routeur intermédiaire du réseau (cas 2) ?



▪ Solution

– 2 possibilités :

- soit $\exists$ localement une table contenant pour chaque machine du (sous) réseau IP une correspondance : $@ IP \leftrightarrow @ MAC$
- soit il faut pouvoir récupérer cette correspondance

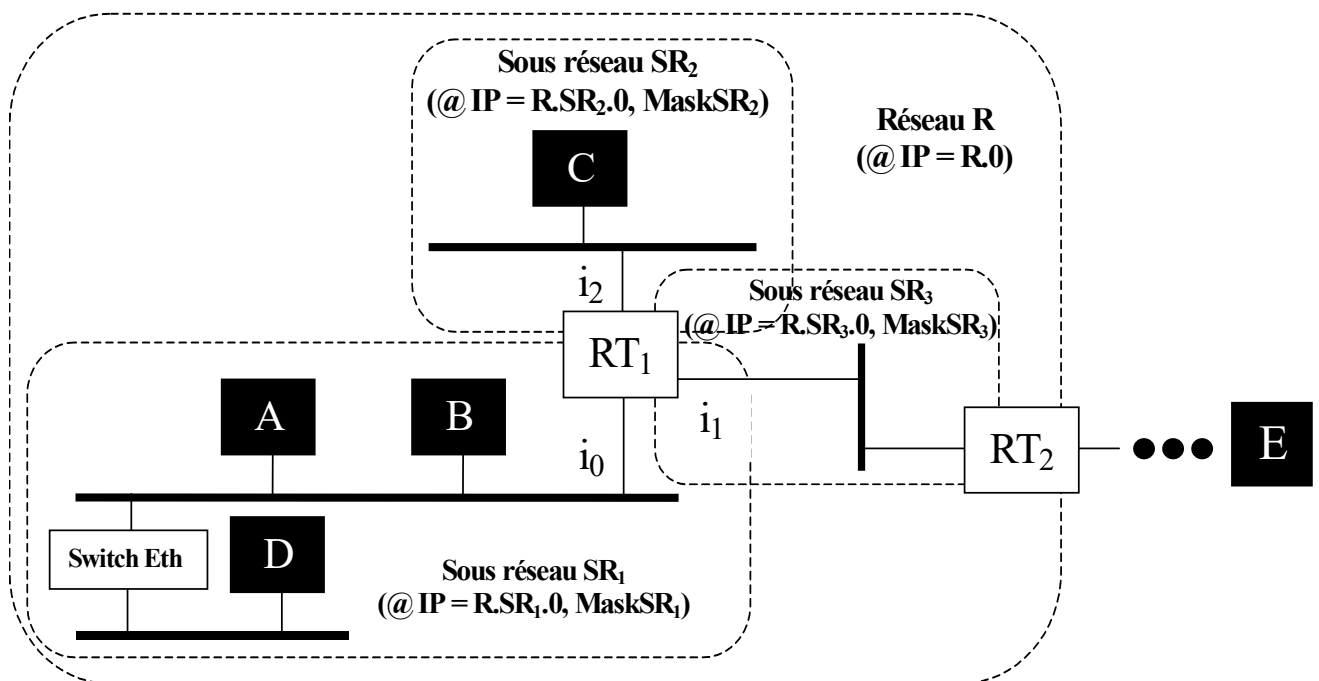
– En fait :

- $\exists$ un cache local contenant tout ou partie des correspondances
- et $\exists$ aussi un protocole qui permet de récupérer une correspondance absente du cache :

ARP : « Adresse Resolution Protocol »

– Question subsidiaire

- Où s'arrête la recherche de résolution d'adresse de A ?



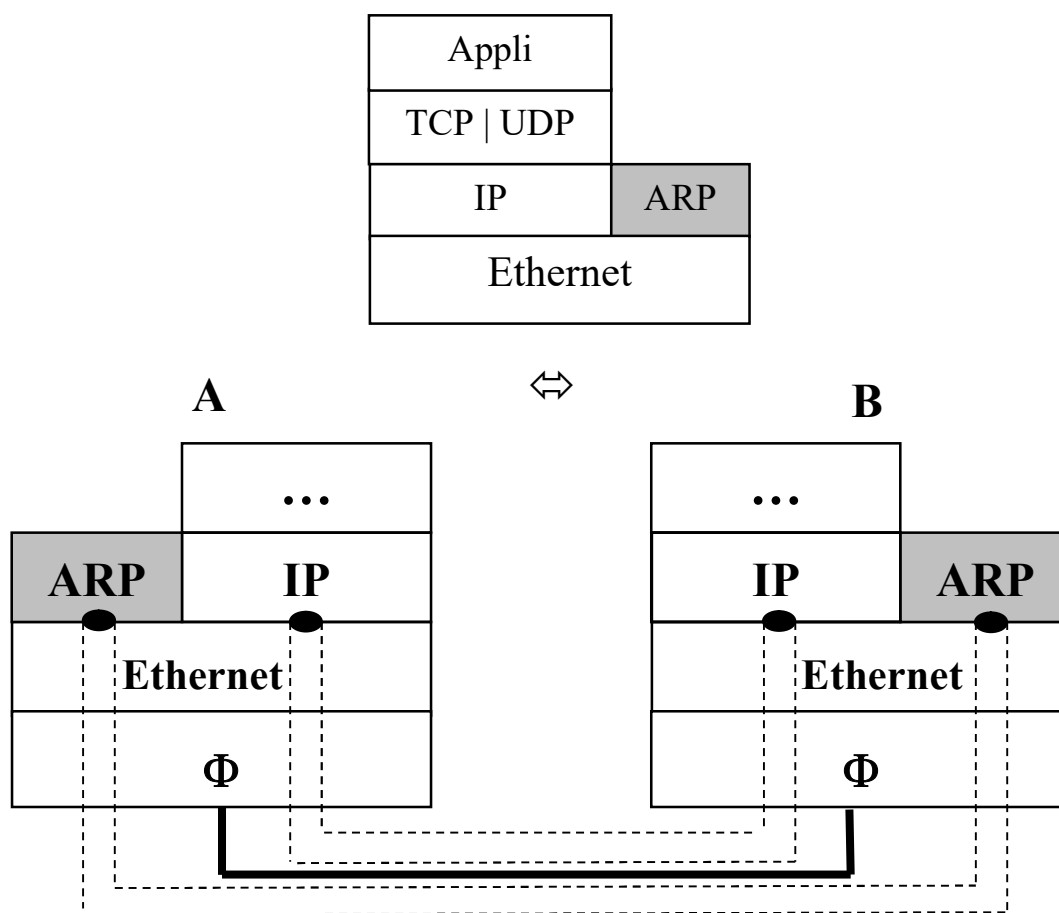
♦ Le protocole ARP (pour IP sur Ethernet)

1. Localisation dans l'architecture TCP/IP

- ARP est véhiculé par le protocole Ethernet

- mais ne transporte aucune donnée !

- Localisation sur un hôte



- Question : ARP est-il déployé sur les routeurs ?

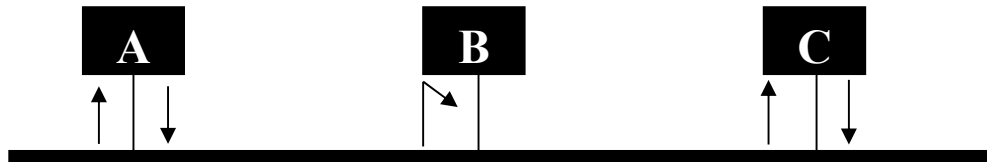
1. Format d'un paquet ARP

Hard Type	Proto Type	Hard Size	Proto Size	Op	@ Hard. Src	@ Proto Src	@ Hard. Cible	@ Proto Cible
-----------	------------	-----------	------------	----	-------------	-------------	---------------	---------------

- **Hard Type** : type d'@ physique
 - Hard Type = 1 $\Leftrightarrow$ Hard = Ethernet
- **Proto Type** : type d'@ du proto transporté dans les trames MAC
 - Proto Type = 08 00 en format hexacimal $\Leftrightarrow$ proto transporté = IP
- **Hard Size** : taille en octet des @ physique (ex : 6 pour Ethernet)
- **Proto Size** : taille en octet des @ du proto transporté (ex : 4 pour IP)
- **Op** : 1 pour une demande d'@ et 0 pour une réponse
- **@ Hard Src** : @ physique de l'émetteur d'un paquet ARP
 - $\Leftrightarrow$ @ Eth. de l'émetteur si Hard Type = 1
- **@ Proto Src** : @ proto de l'émetteur d'un paquet ARP
 - $\Leftrightarrow$ @ IP de l'émetteur si Proto Type = 0800
- **@ Hard cible** : @ physique recherchée
 - $\Leftrightarrow$ @ Eth. recherchée si Hard Type = 1
- **@ Proto cible** : @ proto correspondant à l'@ physique recherchée
 - $\Leftrightarrow$ @ IP ciblée si Proto Type = 0800

2. Le protocole au travers d'un exemple

- **Hyp** : recherche par A de l'@ Ethernet de C



- 1] Diffusion à tous d'un paquet ARP par A
→ via une trame Ethernet émise en mode broadcast
 - Op = 1 (requête)
 - @ Hard Src =
 - @ Proto Src =
 - @ Hard Cible =
 - @ Proto Cible =
- 2] Réception du paquet par B → action = rejet
- 3] Réception du paquet par C
 - **Action** = consommation et émission d'un paquet ARP de réponse à destination de A (via une trame Ethernet à destination de A)
 - Op = 0 (réponse)
 - @ Hard Src =
 - @ Proto Src =
 - @ Hard Cible =
 - @ Proto Cible =
- 4] Réception par A du paquet émis par C :
 - **Action** = enregistrement de l'@ Eth. de C dans le cache ARP de A

➤ Rmq

- **Tout paquet ARP est transporté dans une trame :**
 - Trame Ethernet si archi = /IP/Ethernet
 - Trame « SNAP » si archi = /IP/SNAP/LLC/MAC≠Ethernet
 - car LLC ne dispose pas de SAP prédéfini pour ARP
- **Pour un paquet ARP transporté dans une trame Ethernet**
 - champ « type » = 08 06 en format hexadécimal
- **Pour consulter le cache ARP**
 - commande : arp -a
- **Pour modifier le cache ARP**
 - nécessité d'être root
- **Dans le cas d'autres réseaux physiques qu'Ethernet :**
 - Même format de trames ARP
 - Mais autre spécification des règles du protocole ARP
 - cas des réseaux ATM par exemple

4.2. Les protocoles RARP et DHCP

◆ Introduction

■ Limite de l'adressage statique (à la main)

- Maintenance lourde pour l'administrateur de réseaux comportant un grand nombre de machines, en cas :
 - de changement de prochain routeur, de serveur de nom, ...
- Toute adresse affectée ne peut pas être utilisée, même si cette machine est hors service
 - Pb lorsqu'il y a plus de machines à gérer que d'@ IP disponibles !
 - Cas typique des fournisseurs d'accès à l'Internet, qui ont en général plus de clients que d'adresses IP à leur disposition
 - mais dont tous les clients ne sont pas connectés en même temps !

■ Solution = adressage dynamique (automatique)

- Permet à une machine de récupérer automatiquement une @ IP à partir de son adresse Ethernet
- Mise en œuvre :
 - Précurseur = RARP : Reverse ARP
 - Remplacé par : DHCP : Dynamic Host Configuration Protocol
 - seules les machines en service utilisent une @
 - toute modification de paramètres (@ routeur, serveurs de noms) :
 - est centralisée sur les serveurs DHCP
 - est transmise aux machines lors de leur (re)démarrage

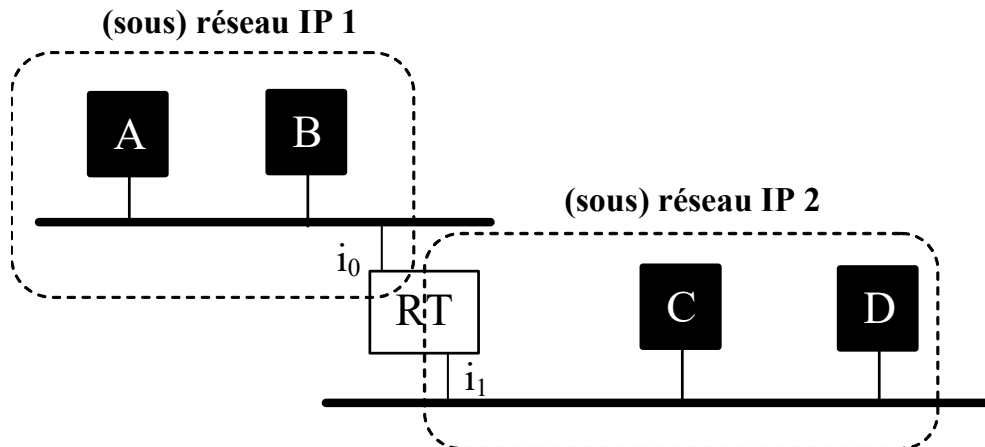
♦ DHCP (principes)

▪ Principes analogues à ceux de ARP (pour Ethernet)

- Format de trame analogue à celui de ARP
- A son démarrage, une machine A
 - génère d'une requête DHCP comportant l'adresse physique de A
 - via une trame Ethernet de broadcast
- Toute machine ne déployant pas le serveur DHCP rejette la trame
- La machine déployant le serveur DHCP
 - Affecte une adresse IP disponible à A
 - Enregistre localement la correspondance ($@eth\ A \leftrightarrow @IP\ A$)
 - Renvoie une réponse à A comportant
 - l'adresse IP affectée à A et sa durée de validité (notion de bail)
 - le masque du réseau
 - l'adresse du prochain routeur
 - pour mise à jour de la table de routage
 - l'adresse du serveur de nom
 - ...

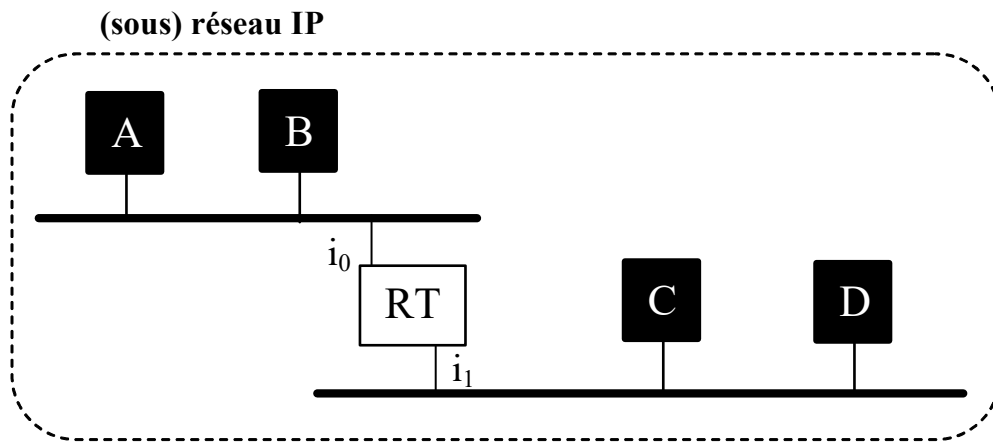
4.3. Proxy ARP (*hors du cours de cette année*)

- Algo de routage « classique » dans le cas d'une interconnexion par routeur IP de deux réseaux IP différents :



- Pour émettre un paquet IP « P » vers B
 - A consulte sa table de routage et génère une trame MAC encapsulant P d'@ $MAC_{dest} = @ MAC_B$
 $\Leftrightarrow$ @ MAC indiquée dans le cache ARP de A
- Pour émettre un paquet IP « P » vers C
 - A consulte sa table de routage
 - puis génère une trame encapsulant P d'@ $MAC_{dest} = @ MAC_{RT}$ (coté réseau IP 1)
 - Recevant cette trame et constatant que le paquet IP encapsulé est à destination d'un hôte du réseau IP 2 :
 - RT génère une trame encapsulant P d'@ $MAC_{dest} = @ MAC_C$

- $\exists$ une autre possibilité (moins utilisée) $\Leftrightarrow$ « Proxy ARP »



- Différences avec l’algo classique
 - Tous les hôtes se considèrent sur le même réseau (ou SR) IP
 - du point de vue d’un hôte : tous les autres hôtes du réseau sont donc directement accessibles !
- Pour émettre un paquet IP vers B
 - Rien ne change : A génère une trame d’@ MAC_{dest} = @ MAC_B
 - @ MAC indiquée dans le cache ARP
- Pour émettre un paquet IP vers C
 - Considérant que C est sur son réseau, A doit théoriquement générer une trame MAC à destination de C
 - à condition qu’elle puisse récupérer l’@ MAC de C !
 - qui n’est pas dans le cache ARP (ce qui est normal ...)
 - PROBLEME ?

▪ **Solution ⇔ proxy ARP**

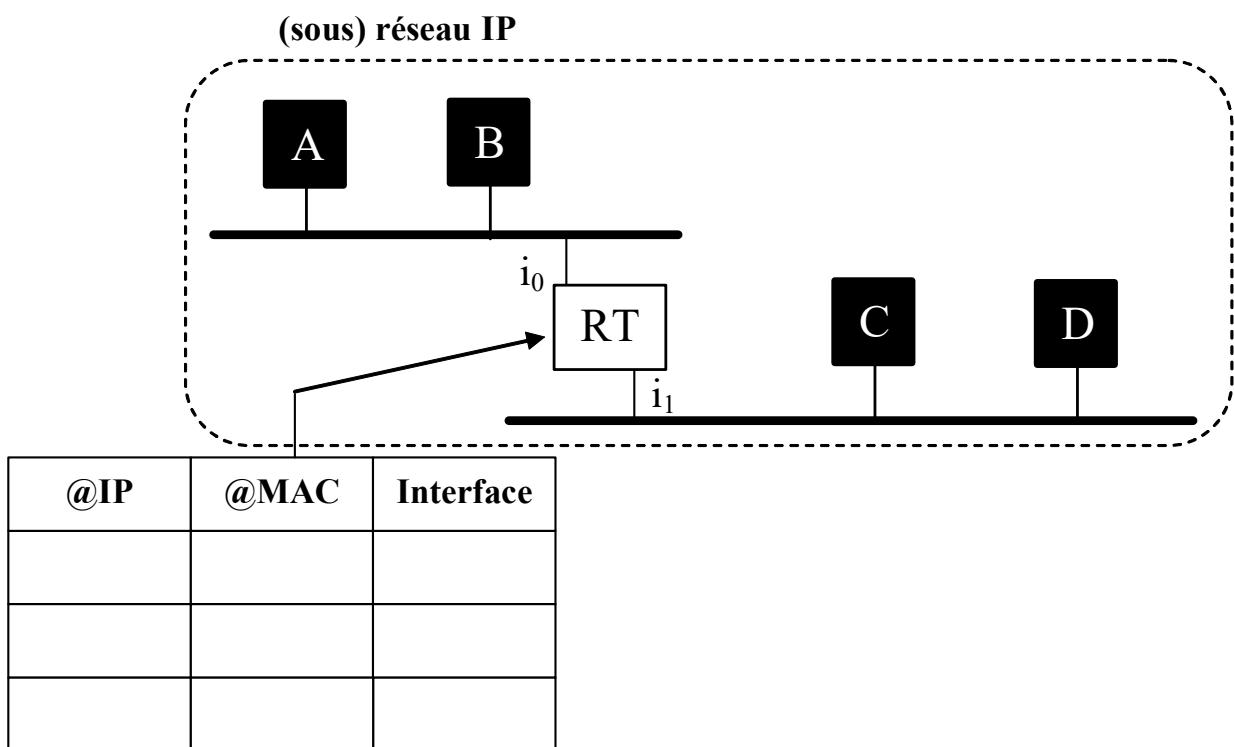
– Solution appliquée au niveau de RT

- nécessitant que RT connaisse la position des machines sur chacun de ses réseaux IP, i.e :

– RT tient à jour une table de correspondances

« @ IP, @ MAC, port d'accessibilité »

- et ce, pour toutes les machines de ses réseaux IP d'appartenance



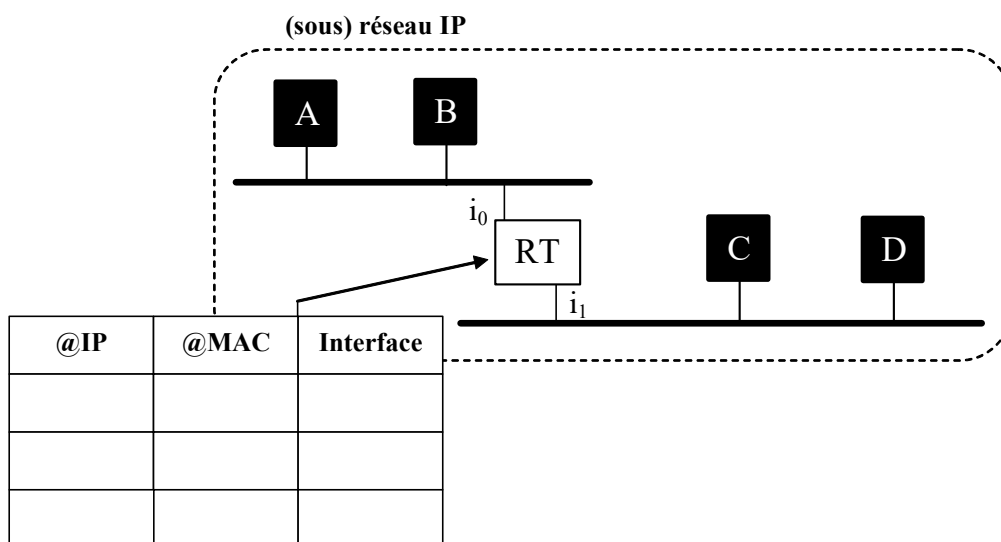
▪ Recherchant l'adresse MAC de C

- A génère en mode broadcast une trame MAC encapsulant un paquet ARP d'@IP_{cible} = @ IP_C
- Recevant cette trame sur l'interface i_0 et constatant que l'@IP_{cible} identifie un hôte accessible par une interface autre que i_0 :
 - RT génère une trame MAC à dest. de A, encapsulant un paquet ARP d'@MAC_{src} = son @ MAC (coté réseau IP 1)

- Recevant cette trame, A met à jour son cache ARP, puis génère une trame MAC encapsulant le paquet IP destiné à C :

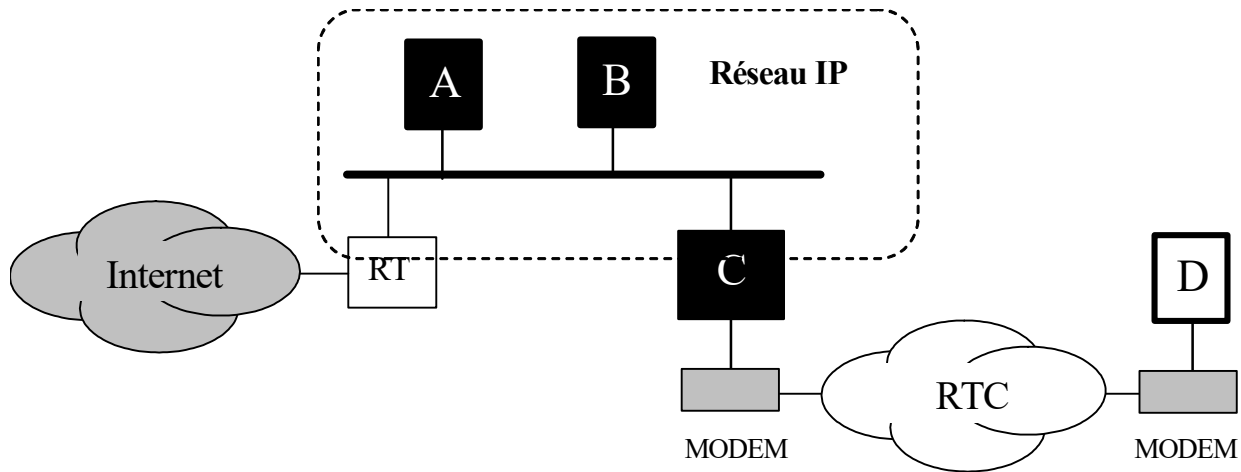
dont l'@ MAC_{dest} = @ MAC_{RT} coté réseau IP 1

- Recevant cette trame et constatant que le paquet IP n'est pas pour lui :
 - RT consulte sa table de correspondance ...
 - puis génère une trame MAC encapsulant le paquet IP d'@ MAC_{dest} = @ MAC_C



▪ **Application courante (il fut un temps ...) de « Proxy ARP »**

- lors d'une connexion d'une machine au réseau via modem et PPP



- Hyp

- D ne dispose pas d'une carte réseau → D n'a pas d'@ MAC
- en revanche : D se voit attribuer une @ IP lors de sa connexion au réseau via le fournisseur de service qu'elle utilise (ici C)
 - cf. PPP : Point to Point Protocol (cours Réseaux Grande Distance)

→ D appartient donc au même réseau IP que C

- Lorsque D génère un paquet IP à destination d'un hôte X :

- C génère sur le réseau IP 1 une trame MAC (encapsulant le paquet IP d'@ IP_{dest} = @ IP_X) d'@ MAC_{src} = @ MAC_C

- Lorsqu'une machine émet un paquet IP à destination de D :

- A (ou B ou RT) génère une trame MAC (encapsulant le paquet IP d'@ dest = @ IP_D) d'@ MAC_{dest} = @ MAC_C
- recevant cette trame, C route le paquet encapsulé vers le port auquel est connecté son modem

4.4. Le protocole ICMP

▪ Dans sa version actuelle

- IP est autorisé à perdre et/ou à déséquencer ses paquets
 - sans aucune forme de reprise ni de notification

▪ En fait

- La mise en œuvre du protocole IP est toujours accompagnée de celle d'un autre protocole, véhiculé par IP :

ICMP : Internet Control Message Protocol

- Rôles de ICMP
 - Donner des informations à l'hôte source d'un paquet IP lorsque ce dernier n'a pu être traité correctement :
 - soit par l'hôte destinataire
 - soit par un routeur intermédiaire
 - Répondre à des requêtes :
 - Existence d'une machine, masque du sous-réseau, ...

▪ Ex de « mauvais » traitement

- $\exists$ un message ICMP indiquant qu'un hôte dest. n'est pas atteignable
- $\exists$ un message ICMP indiquant que le processus de réassemblage du paquet a échoué
- ...

■ Format des messages ICMP

Type	Code	Somme de contrôle
Données supplémentaires		

- **Type** : 15 types de messages ICMP répartis en 2 classes :
 - les messages de requête ou de réponse
 - requête : une réponse est attendue de la part du destinataire
 - réponse : une requête a été reçue par le dest. qui y répond
 - type = 0, 8, 9, 10, 13, 14, 15, 16, 17, 18
 - les messages d'erreur ou d'indication
 - notifiant une erreur ou un renseignement au destinataire
 - type = 3, 4, 5, 11 et 15
 - **Code**
 - pour certain des messages d'erreur, $\exists$ des sous types identifiés par un code
 - **Données supplémentaires**
 - Fonction du type de message (le plus souvent = 32 bits à 0)
- ## ■ Tout message ICMP est véhiculé dans un paquet IP
- émis par et destiné à une machine identifiée par son @ IP

➤ Ex

- Messages de requête / réponse (**type = 0, 8, 9, 10, 13 à 18**)
 - requête d'écho $\Leftrightarrow$ type = 8, code = 0
 - permet de tester l'accessibilité à une machine du réseau
 - générée par appel à la commande :
 - ping « @IP ou nom d'une machine »
 - données supplémentaires :
 - ident. du paquet (16 bits) + n° de seq (16 bits)
 - réponse à une requête d'écho $\Leftrightarrow$ type = 0, code = 0
 - générée en réponse à une requête
 - données supplémentaires :
 - ident. du paquet + n° de seq
 - requête de netmask $\Leftrightarrow$ type = 17
 - généré par un hôte qui a besoin du masque de son réseau pour être configuré
 - réponse à une requête de netmask $\Leftrightarrow$ type = 18
 - généré par un hôte ou routeur configuré et qui retourne le masque du sous-réseau
 - données supplémentaires :
 - ident. du paquet + n° de seq + masque (32 bits)

▪ **Messages d'erreur (type = 3, 4, 5, 11 et 15)**

- Destinataire inaccessible $\Leftrightarrow$ type = 3
 - réseau inaccessible $\Leftrightarrow$ code = 0
 - machine inaccessible $\Leftrightarrow$ code = 1
 - ...
- Durée de vie expirée $\Leftrightarrow$ type = 11
 - généré par un routeur qui reçoit un paquet IP de TTL = 0
 - utilisé par la commande `traceroute`
 - pour connaître la route allant d'une machine à une autre
 - c'est à dire l'identité des routeurs traversés
- Indication de redirection $\Leftrightarrow$ type = 5
 - généré par un routeur qui connaît une route plus courte pour atteindre une machine
 - données supplémentaires :
 - @ du routeur qui offre une route plus courte

V. VPN, Proxy Applicatif et NAT, Multicast IP

1. Introduction

2. Le concept de réseau privé virtuel (VPN)

3. Proxy applicatif et translation d'adresse réseau (NAT)

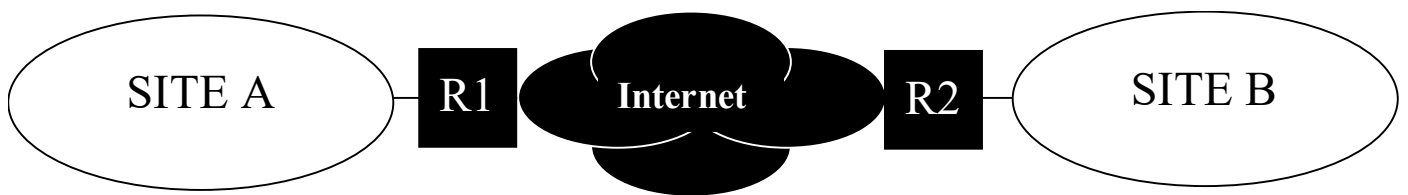
4. Couplage du VPN et du NAT

5. Gestion du multicast au niveau IP

1. Introduction

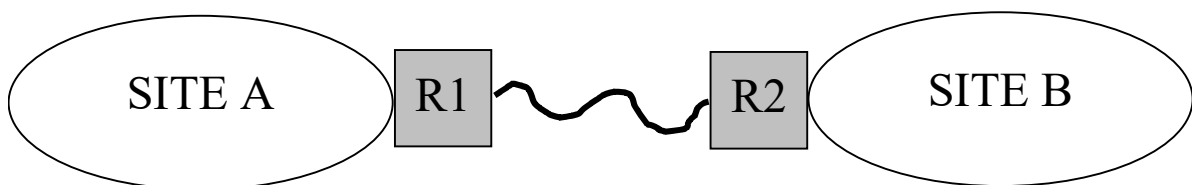
▪ Plusieurs problèmes dans l'actuel Internet parmi lesquels :

- manque de confidentialité dans les données échangées entre 2 sites distants (par ex d'une même entreprise)
 - les paquets IP échangés peuvent être facilement interceptés lorsqu'ils circulent dans l'Internet
- espace d'adressage limité
 - lié au codage des adresses IP sur 4 octets



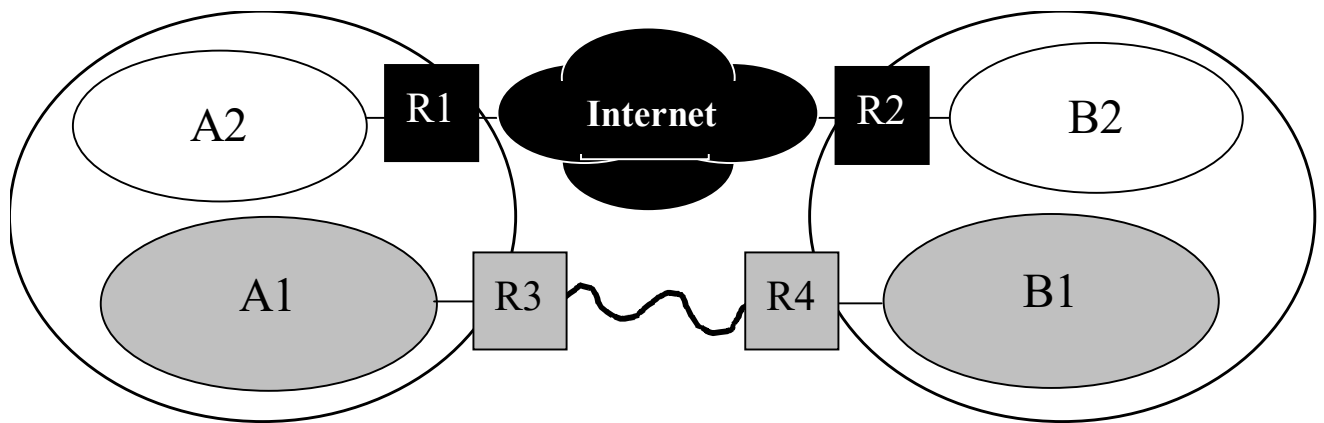
▪ Face au premier problème

- **Une solution** = constituer un **réseau privé** déployant au niveau de ses sites l'architecture TCP/IP mais interconnectant ses sites :
 - non pas par l'Internet ...
 - mais par un circuit dédié : ligne louée, VC/VP ATM, ...
 - aucune autre organisation n'aura accès aux données échangées



➤ Rmq

- Etre isolé de l'Internet n'est pas forcément une bonne solution : non accès aux transferts de fichiers ou aux mails avec l'extérieur, ...
→ choix effectué = celui d'un réseau hybride
- combinant réseau privé (entre A₁ et B₁)
- et réseau ouvert à l'Internet (A₂ sur le site A, B₂ sur le site B)



▪ Inconvénient de cette première solution : coût

- ligne louée, VP ATM, ... = **cher !**

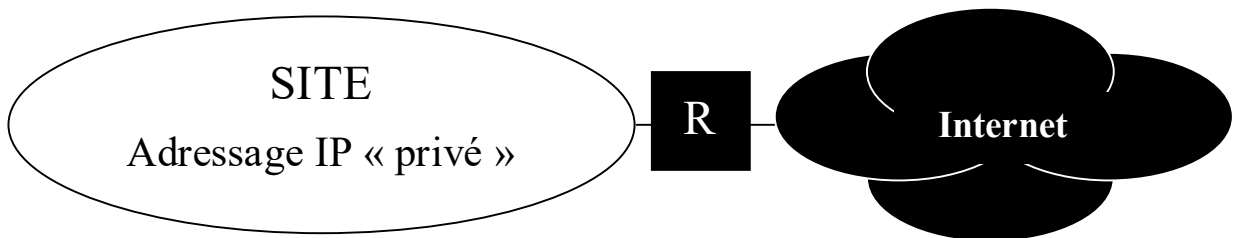
→ Problème ainsi posé :

- comment utiliser l'Internet (moins cher) tout en ayant une garantie de confidentialité sur les données échangées ??
 - une solution = « réseaux privés virtuels »

VPN : Virtual Private Network

▪ **Face au second problème** (limite de l'espace d'adressage)

- l'administrateur d'un site peut être amené à affecter :
 - des **adresses IP « privées »** à tout ou partie des hôtes du site



- **Def :** adresse IP privée $\Leftrightarrow$ @ non routable
 - i.e. non reconnue par les routeurs de l'Internet
 - issues d'une plage des @ IP (cf RFC n° ***)
 - ex : toutes les adresses de classes C débutant par 192.168
 - 192.168.1.0, 192.168.2.0, ...
- Problème = soit alors une entreprise
 - souhaitant que ses hôtes puissent accéder aux services de l'Internet
 - mais ne possédant pas (ou ne souhaitant pas posséder ...) un nombre suffisant d'@ IP valides (routables)
 - quelle(s) solution(s) technique(s) peut-elle alors adopter ?
- **3 solutions envisageables :**
 - la passerelle applicative → **proxy applicatif**
 - la translation d'@ réseau → NAT : *Network Address Translation*
 - la redirection de port → *port forwarding*

2. Le concept de réseau privé virtuel (VPN)

2.1. Concept initial

▪ Face au problème de confidentialité précédent

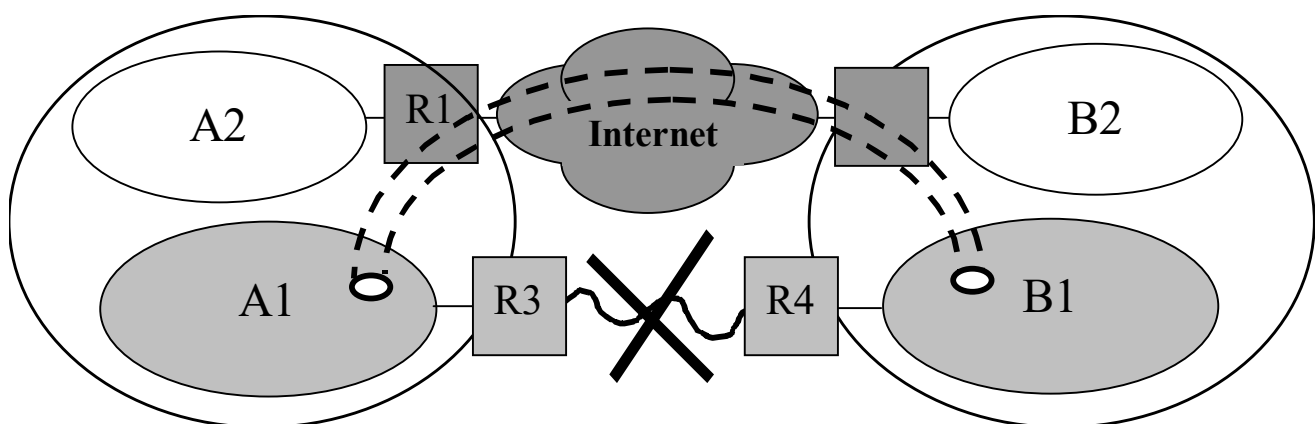
- une solution = concept de « **réseau privé virtuel** »

VPN : Virtual Private Network

- **privé** au sens défini en introduction
 - les paquets échangés ne sont pas interprétables par l'extérieur
→ besoin exprimé en terme de « **confidentialité** »
- **virtuel** car cette fois ci, on n'utilise pas de ligne dédiée
 - mais on « émule » une telle ligne au travers de l'Internet
→ concept de « **tunnel** »

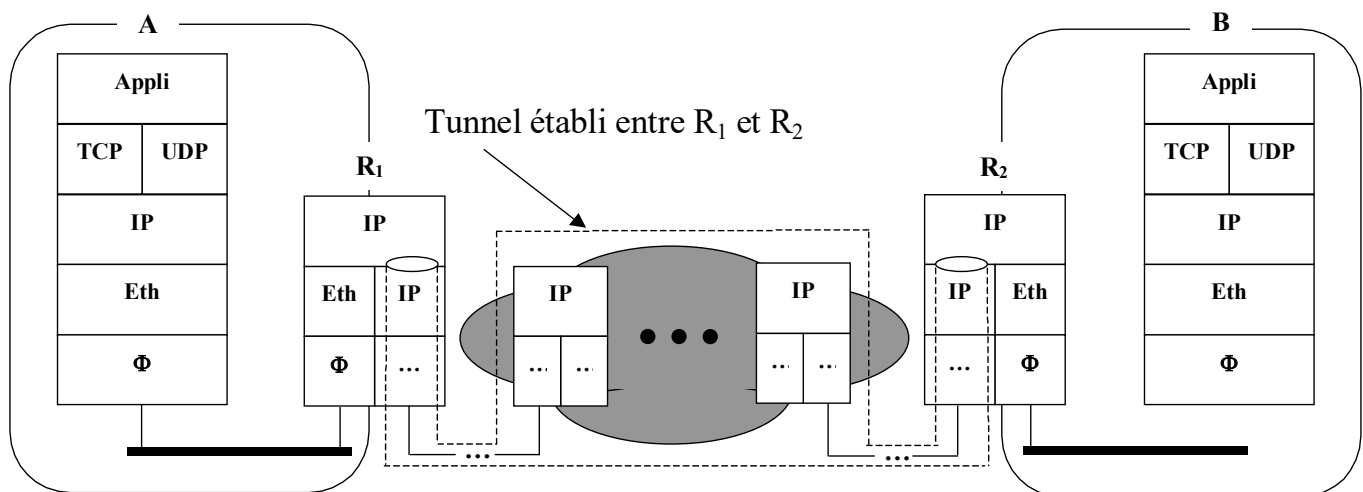
▪ Rmq

- Autre désignation d'un VPN : **tunnel sécurisé**



2.2. Principe de mise en œuvre

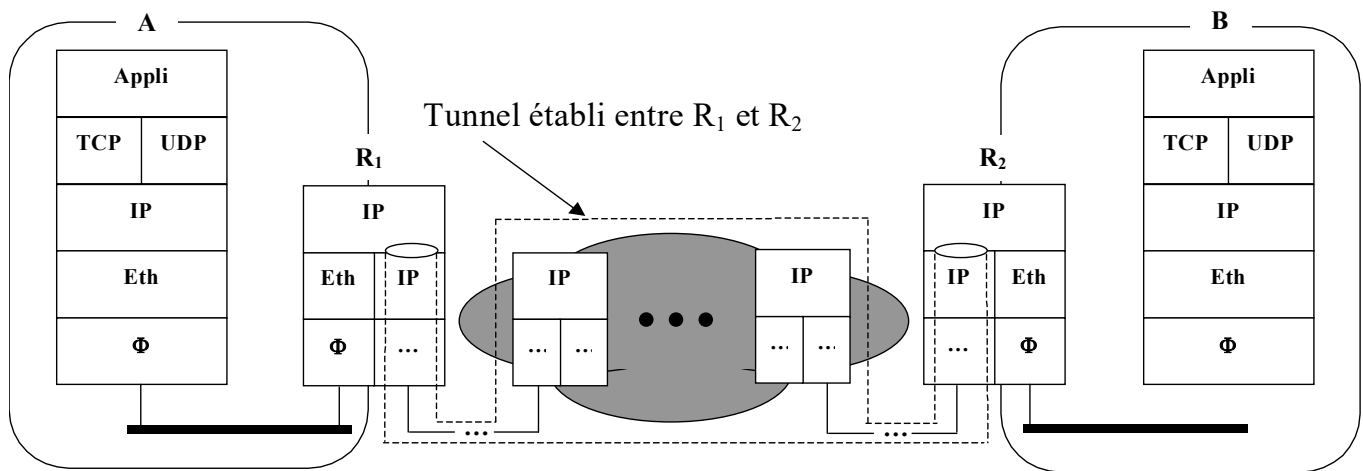
- 2 composantes dans la mise en œuvre d'un VPN
 - le **cryptage** (/ **chiffrement**) $\Leftrightarrow$ encodage des informations
 - le **tunneling** (*ici de niveau IP*) $\rightarrow$ mécanisme IMPORTANT !
 - consistant à créer un canal (ici IP) entre deux points
 - par exemple 2 routeurs (R_1 et R_2)
 - et à y échanger :
 - non pas des PDU TCP ou UDP (par ex.)
 - mais des paquets IP ! $\Leftrightarrow$ « **encapsuler de l'IP dans IP** »



➤ Rmq importante

- Tunneling = mécanisme également utilisé :
 - pour le déploiement du multicast au niveau IP dans l'Internet
 - pour le déploiement d'IPv6 dans l'Internet (IPv4)
 - ...

▪ **Techniquement, la solution consiste (VPN entre R₁ et R₂) :**



– Au niveau de R₁

- à crypter le paquet IP « P » à transférer (entête et charge utile)
- puis à le transmettre via un **tunnel** établi avec R₂, i.e :
 - à l’encapsuler dans un paquet IP « P’ » :
 - dont l’@ IP source est celle de R₁
 - dont l’@ IP destination est celle de R₂
 - à émettre le paquet IP P’

– Au niveau de R₂

- à recevoir le paquet IP P’
- à désencapsuler le paquet IP P
- à décrypter P puis à l’aiguiller vers la machine destinataire

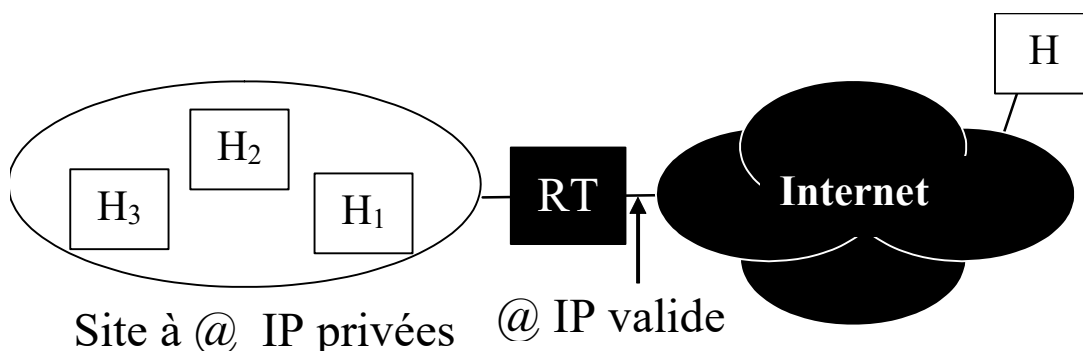
▪ **Intérêt**

- les routeurs de l’Internet pourront manipuler les entêtes du paquet P’
mais pas son contenu (i.e. le paquet P) !

3. Proxy applicatif et translation d'adresse réseau (NAT)

3.1. Position du problème

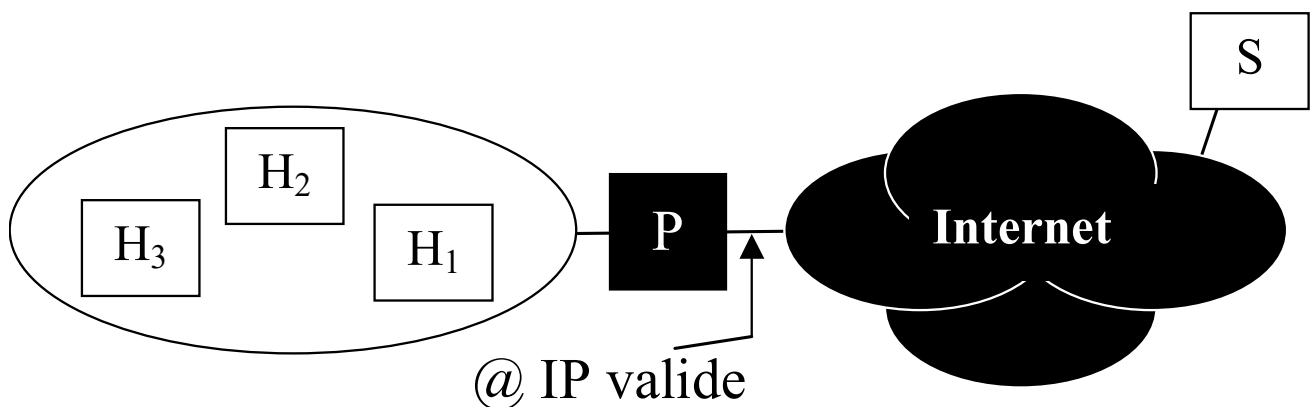
- **Face aux limites de l'espace d'adressage** (ou par volonté de l'administrateur du réseau à des fins de sécurité ou autres)
 - L'admin. d'un site peut être amené à utiliser des adresses IP privées
 - **Double problème du point de vue d'une machine du site :**
 - comment accéder aux services de l'Internet (ftp, ...) ? (Pb 1)
 - comment rendre ses services accessibles depuis l'extérieur ? (Pb 2)
- **Parmi les solutions envisageables :**
 - En réponse au premier problème (Pb 1) :
 - la passerelle applicative → **proxy applicatif**
 - la translation d'@ réseau → NAT (Network Address Translation)
 - En réponse au deuxième problème (Pb 2) :
 - la redirection de port → **port forwarding**



3.2. Proxy applicatif : principes de base

■ Principe

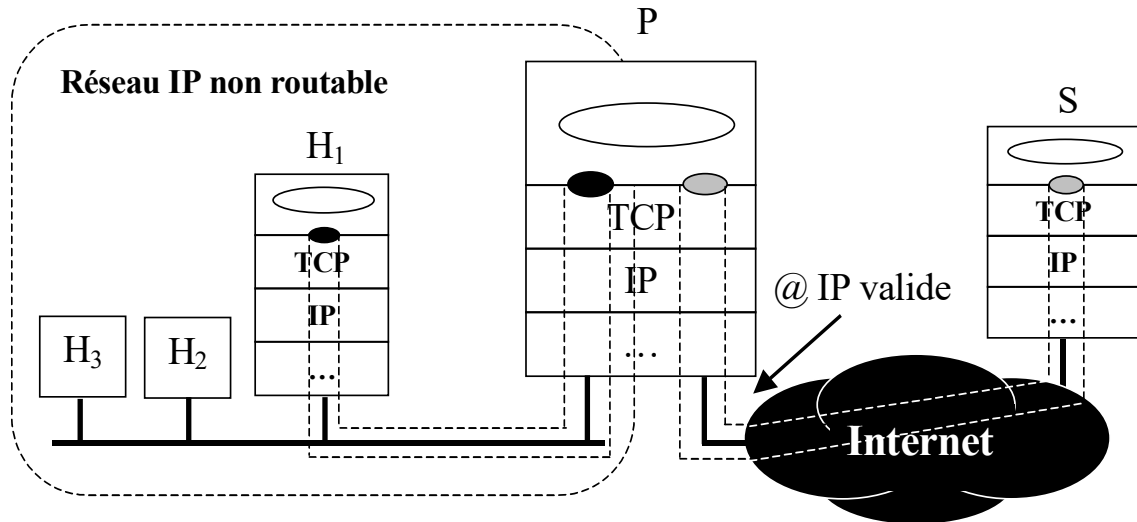
- Les hôtes du site ne disposent pas d'un accès IP extérieur
 - mais peuvent accéder à un certain nombre de services applicatifs installés sur une machine spécifique P
- qui elle, possède un accès IP extérieur, et donc une @ IP publique !



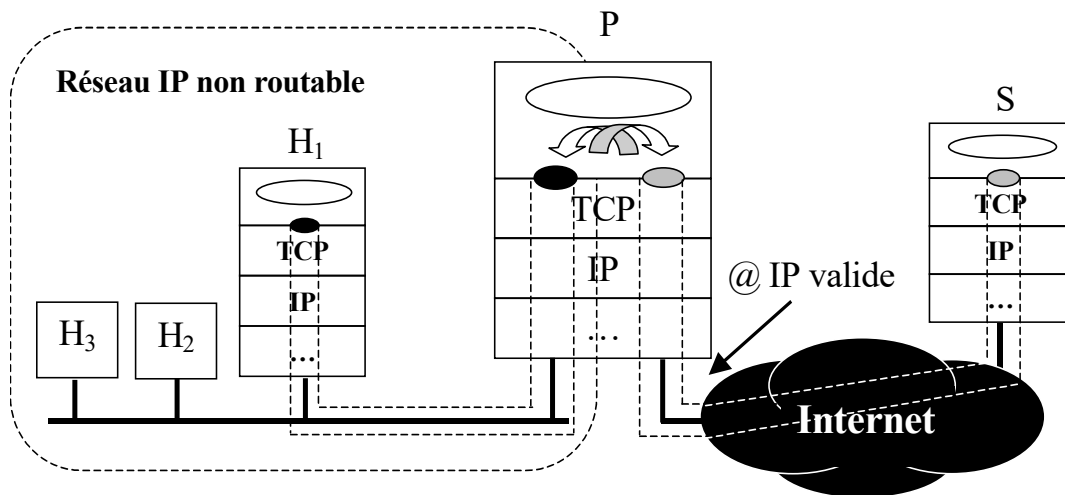
➤ Rmq :

- P n'est (généralement) pas un routeur IP, mais une machine accessible par les machines H_i et possédant une adresse publique

■ Principe (suite)



- Lorsqu'un hôte du site (H₁ par ex) invoque un service applicatif (ftp par ex) à destination d'un serveur distant (S) :
 - il établit une connexion TCP avec le proxy P (port 3128)
 - si invocation du service via un navigateur
 - l'adresse de P est mentionnée dans un cache
 - ex : Int Explorer : options Internet / connexions / paramètre LAN
 - sinon l'adresse de P doit être explicitement spécifiée
 - il transfère la commande à exécuter à P (via la connexion TCP)
 - P exécute la commande à la place de H₁
- ⇔ établissement d'une session applicative entre P et S



– Dans un second temps :

- tout paquet applicatif émis par H₁
 - sera transmis à P (via la connexion TCP) qui :
 - soit le relaiera tel quel à S sans l'interpréter
 - soit l'interprétera, et en fonction de sa sémantique :
 - exécutera la commande correspondante à destination de S ou ne l'exécutera pas
 - en fonction de la politique de sécurité appliquée
- ⇔ vrai sens du terme « proxy Applicatif »
- tout paquet applicatif émis par S
 - sera transmis à P (via la session applicative) qui :
 - soit le relaiera tel quel à H₁ sans l'interpréter
 - soit l'interprétera, et en fonction de sa sémantique :
 - exécutera la commande correspondante à destination de H₁ ou ne l'exécutera pas
 - en fonction de la politique de sécurité appliquée

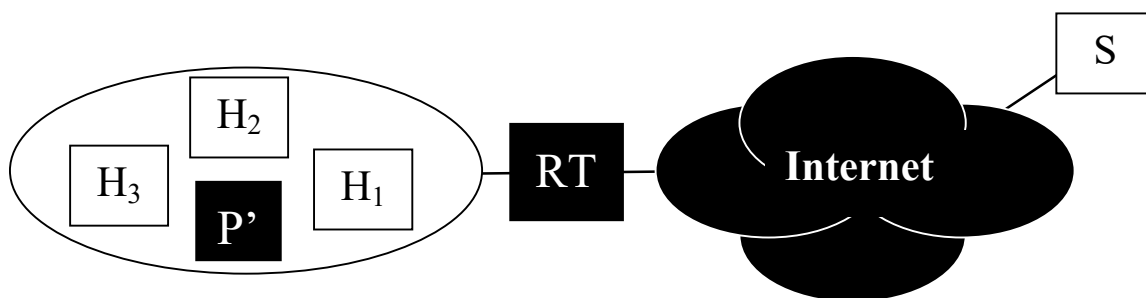
■ Intérêts et limites de la technique

- Les hôtes du site n'ont accès qu'aux seuls services déployés sur P
 - celui-ci pouvant refuser la requête selon la valeur du $\text{port}_{\text{dest}}$
 - intérêt potentiel en termes de **sécurité** du point de l'administrateur
 - limite du point de vue des usagers
- Les hôtes ne sont pas accessibles de l'extérieur
 - limite du point de vue des usagers
 - qui peut cependant être voulue par l'administrat(eur/ion) ...

■ Rmq : les proxy « cache »

- L'utilisation d'un proxy peut être envisagée (pour certains services tel que le web) avec ou sans problème d'adresse IP non routable

→ lorsque le proxy fait office de « cache »

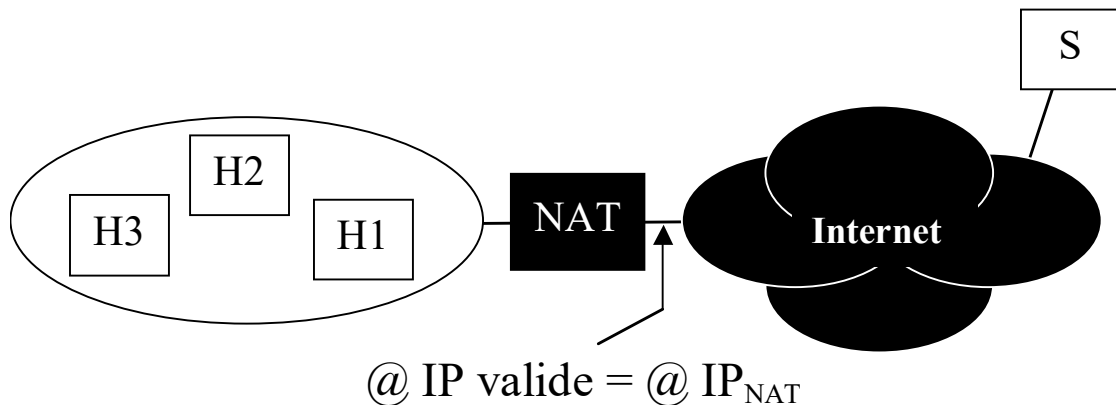


- Dans ce cas, l'adresse IP du proxy doit être explicitée par le hôte
- Motivations (pour le web) :
 - réduire la quantité de trafic échangé avec l'extérieur
 - minimiser le temps de rapatriement des données
 - ... au coût d'une « fraîcheur » moindre des données (existence de solution cependant => discutées durant cours)

3.3. NAT : principes de base

◆ Principe de fonctionnement

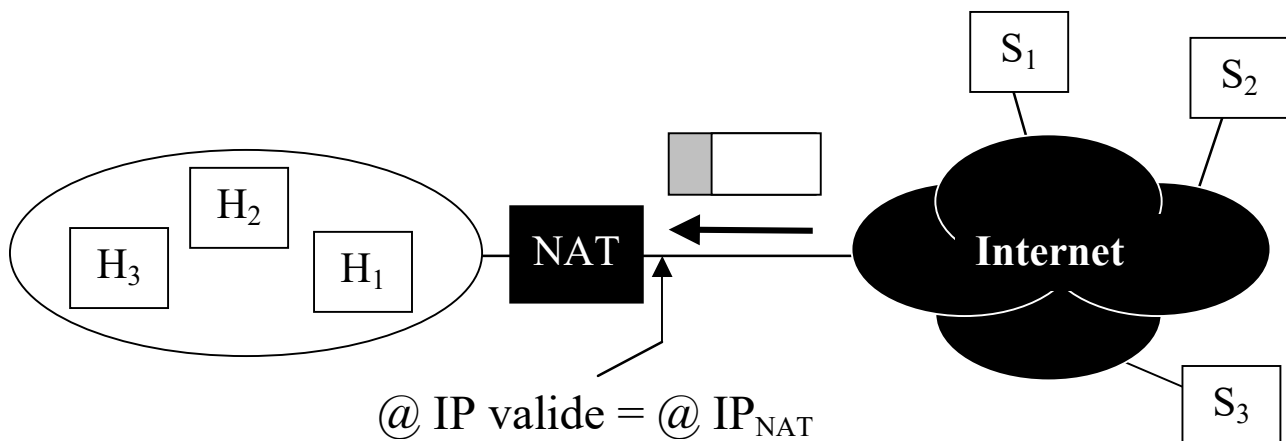
- Un routeur du site disposant d'un accès public à l'Internet (ie. @IP publique) déploie une fonction dite de NAT



- **A la différence d'un proxy applicatif** : un NAT ne déploie pas de service applicatif et ne fait que du routage au niveau IP
 - **Chaque hôte génère des paquets IP dont :**
 - l'adresse IP de destination est celle de la machine serveur ciblée
 - l'adresse IP source est une adresse privée (la sienne)
 - **Le NAT se charge alors de :**
 - remplacer l'@ IP source des paquets **sortants** par son @ IP valide
 - remplacer l'@ IP dest. des paquets **entrants** (qui correspond à son @ IP) par l'@ IP privée du « bon » hôte destinataire
- les hôtes ont donc accès à tous les services de l'Internet

▪ Question

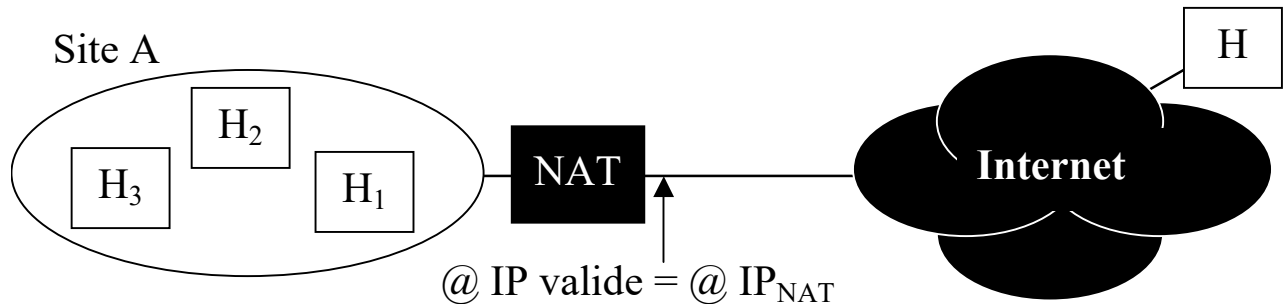
- En retour d'un paquet IP « sortant », i.e. quand un paquet IP arrive de l'extérieur (ayant donc comme @ IP_{dest} l'@ IP valide du NAT) :
- comment ce dernier identifie-t-il le « bon » hôte destinataire ??



- **Réponse** : par consultation d'une **table locale** contenant une liste d'association $\langle \text{@ IP distante}, \text{@ IP privée locale} \rangle$
 - lorsqu'un paquet entrant se présente, le NAT :
 - consulte l'@ IP source du paquet
 - cherche l'entrée correspondante dans sa table, et si elle existe :
 - remplace l'@ IP dest du paquet par l'@ IP privée indiquée dans la table (rejet du paquet si l'entrée n'existe pas)
 - puis aiguille le paquet vers le bon hôte
- **Question** : comment la table du NAT est elle mise à jour ?

◆ Mise à jour de la table du NAT

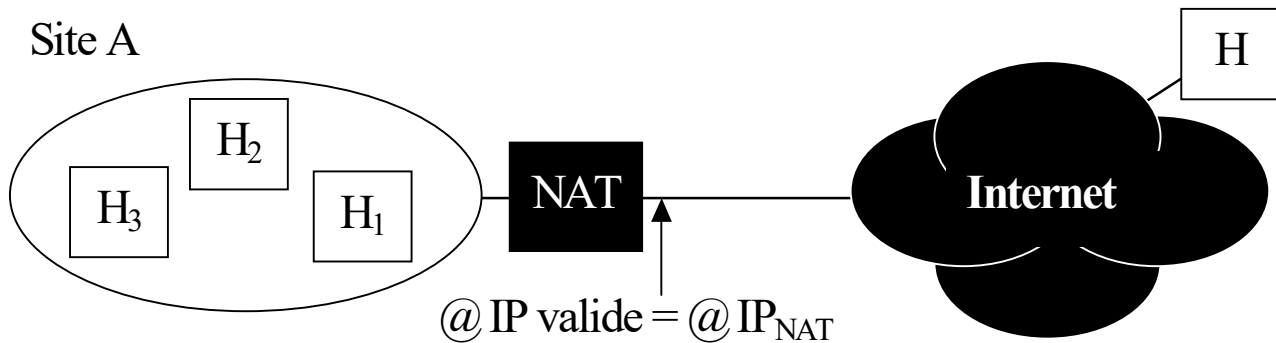
■ 3 techniques



- [1] Manuellement avant que le début des communications
 - Inconvénient : aspect statique de la technique
- [2] Dynamiquement sur occurrence d'un paquet IP sortant
 - création par le NAT d'une association entre l'@ IP source (privée) du paquet et l'@ IP destinataire (celle du hôte H par exemple)
 - Inconvénient : la com. ne peut pas être initiée depuis l'extérieur
- [3] Dynamiquement sur occurrence d'un paquet IP entrant
 - lorsqu'un hôte externe H invoque son serveur de nom de domaine (DNS) pour obtenir l'@ IP d'un hôte du site A, le DNS de A :
 - crée une association dans la table du NAT entre l'@ IP de H et l'@ IP privée du hôte ciblé
 - puis renvoie l'adresse IP du NAT (au lieu de celle de l'hôte)
 - Inconvénient qui en limite la mise en œuvre dans les faits :
 - modification nécessaire du DNS
 - solution qui ne fonctionne que si l'hôte externe invoque le DNS avant d'émettre ses paquets IP

▪ Rmq et exemple

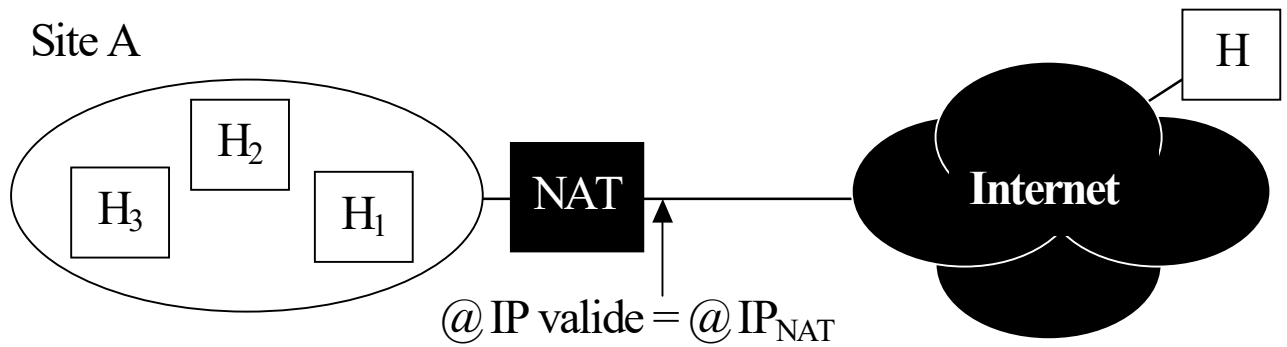
- La seconde technique est la plus utilisée
 - ⇔ dynamiquement sur occurrence d'un paquet IP **SORTANT**
- Exemple d'application de la 2^{ème} technique



- H₁ souhaite établir une session ftp (via TCP) avec la machine H
 - Hyp : le client ftp s'exécute sur H₁, le serveur sur H (la connexion TCP est donc initiée depuis H₁)
- Décrire :
 - le diagramme de séquences des PDU TCP échangés en précisant les numéros de port source et destination de chaque PDU
 - le diagramme de séquences des paquets IP échangés, en précisant les adresses IP source et destination de chaque paquet
 - les manipulations et mises à jour successives de la table du NAT (supposée initialement vide)

3.4. NAT multi adresses et NAT avec translation de port

▪ Inconvénient de la solution NAT de base



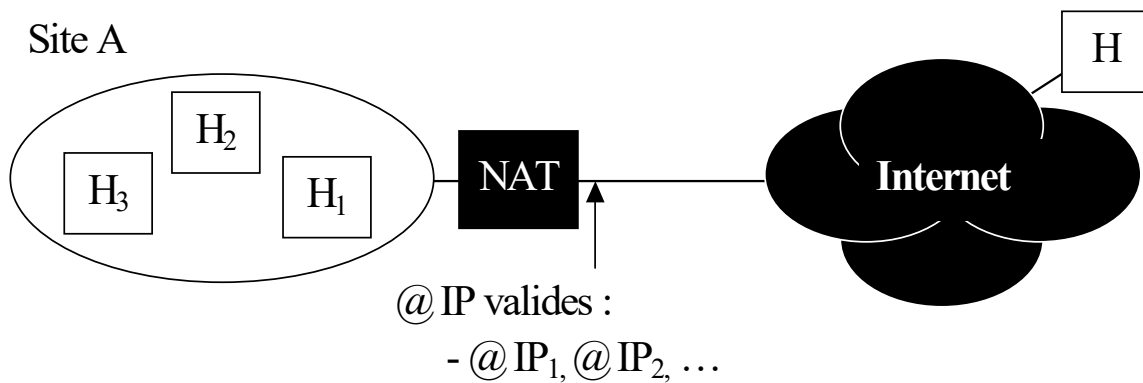
- Si 2 hôtes du site A (H_1 et H_2 par ex) veulent communiquer avec le même hôte externe H

→ PROBLEME !

- Pourquoi ?

■ 1^{ère} solution : NAT multi adresses

- Repose sur l'attribution de **plusieurs @ IP valides** (i.e. publiques) au NAT : @ IP₁, @ IP₂, ...



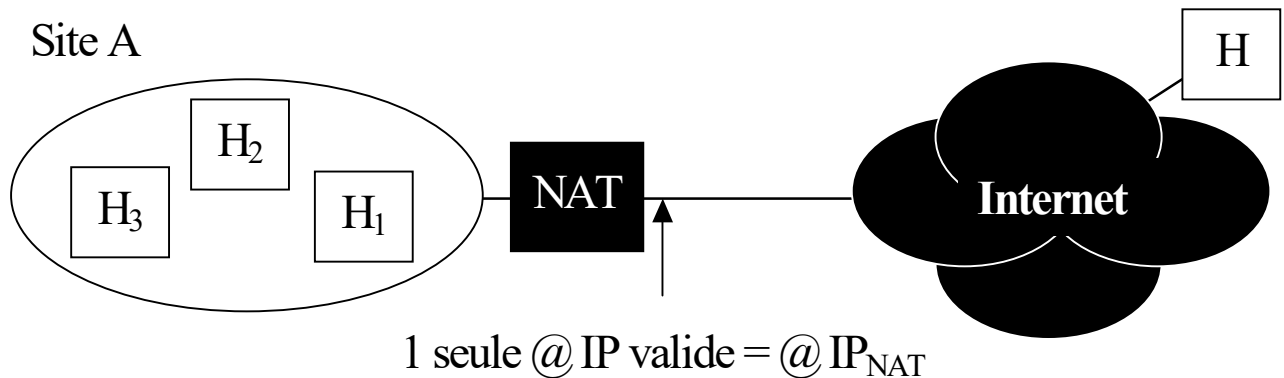
- Soient alors 2 hôtes du site A (H₁ et H₂ par ex) souhaitant communiquer avec le même hôte externe H
 - les paquets issus de H₁ ont comme @ IP source : @ IP_{H1} (privée)
 - les paquets issus de H₂ ont comme @ IP source : @ IP_{H2} (privée)
 - les paquets issus de H₁ ou H₂ ont comme @ IP dest : @ IP_H
 - MAIS une fois généré par le NAT :
 - les paquets issus de H₁ ont comme @ IP source : @ IP₁
 - les paquets issus de H₂ ont comme @ IP source : @ IP₂
 - les paquets issus de H₁ ou H₂ ont comme @ IP dest : @ IP de H
 - **ET** le NAT maintient dans sa table contenant une liste d'association
 - comportant non plus 2 mais 3 composantes :

((@ IP distante, @ IP valide, @ IP privée)

➔ **Problème cependant** : solution inefficace si le nombre de machines souhaitant accéder au serveur est grand

⇔ problème de passage à l'échelle (« scalability »)

■ 2^{ème} solution (« *scalable* » cette fois) : NAT avec *translation de port*

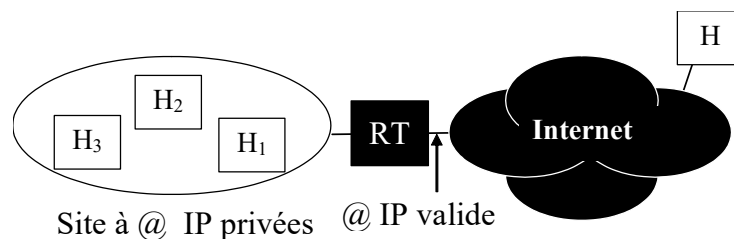


- Solution (à découvrir via l'exercice ci-dessous) basée sur la manipulation du n° de port source des PDU UDP ou TCP échangés
 - ne requérant qu'une seule adresse IP valide pour le NAT
- Exercice
 - En utilisant la base précédente, proposer une solution au problème
 - Raisonner en supposant que 2 requêtes d'établissement d'une session ftp sont établies via TCP depuis les machines H₁ et H₂ (applications clientes) vers le serveur ftp de la machine H

3.5. Port forwarding : principes de base

■ Position du problème

- Proxy applicatif et NAT sont inopérants lorsque la communication est initiée par une machine extérieure H (ayant une @IP publique)
 - Ex. : si H souhaite établir une session applicative (par ex. http)
 - entre un client s'exécutant sur H ...
 - et un serveur s'exécutant sur une machine du site privé, par ex. H₁
- ➔ l'@IP_{dest} du paquet émis par H serait alors privée et le paquet serait rejeté par le premier routeur rencontré



■ Principe de la solution de « port forwarding »

- Consiste à associer, dans une table locale à RT :
 - l'@IP de H₁ et le n° port du serveur de l'appli. ciblée (par ex. 80 si http) à un n° port autre que ceux « bien connus », par ex. : 8080
 - ... et à faire connaître (à l'extérieur) le serveur http s'exécutant sur H₁ par l'adresse : { @IP = @IP_{RT}, n° port = 8080 }
 - Recevant alors un paquet IP d'@IP_{dest} = @IP_{RT} et encapsulant un paquet TCP de n°port_{dest} = 8080
 - RT consulte sa table et redirige le paquet IP vers le serveur http de H₁ (en modifiant les @IP_{dest} et n°port_{dest} des paquets IP et TCP)
 - Tout paquet émis en suivant depuis H₁ dans le cadre de la session http aura comme @IP_{src} et n°port_{src} : @IP_{RT} et 8080
- **Rmq** : Proxy applicatif, NAT et *port forwarding* sont inopérants lorsque les 2 hôtes ont une @ IP privée ➔ solution = ?

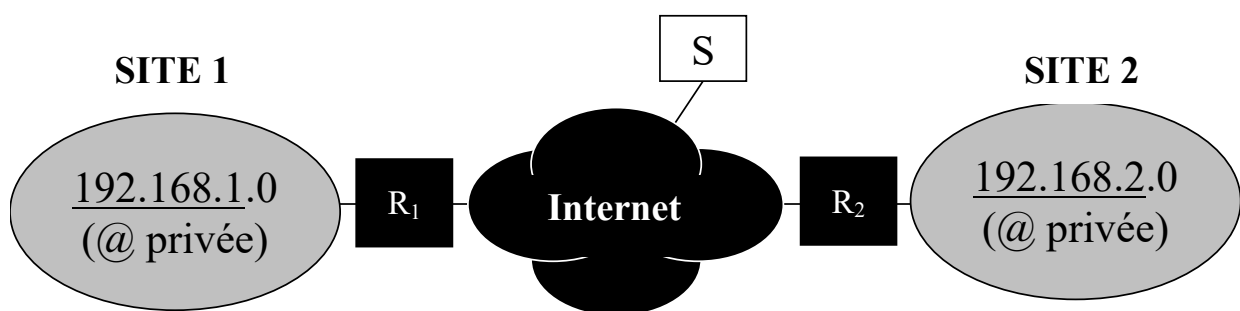
4. Couplage du VPN et du NAT

▪ Soit une entreprise dotée de plusieurs sites distants

- possédant chacun une adresse de réseau IP privée
- et souhaitant :

- 1) que ses sites (SITE 1 et SITE 2) puissent communiquer entre eux
 - avec (optionnellement) des garanties de confidentialité
- 2) souhaitant que ses sites aient également accès à l'Internet

→ solution = VPN avec adresses privées + NAT



▪ Rmq

- Pour les mêmes souhaits 1) et 2)
 - mais sans garantie de confidentialité entre les sites 1 et 2

→ solution = tunnel avec adresses privées + NAT

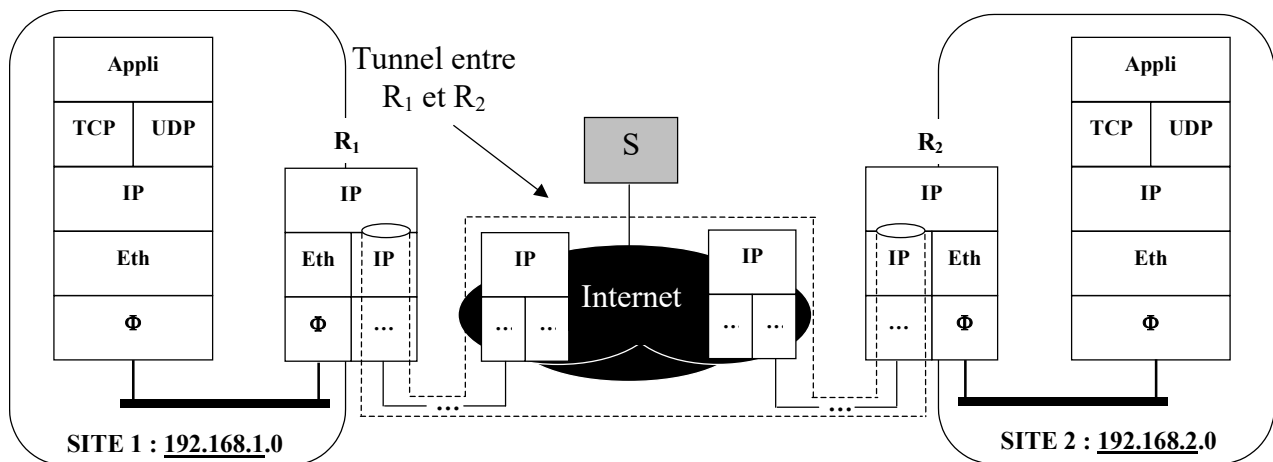
Autrement dit : pas de cryptage des paquets au travers du tunnel

- **Rappel** : VPN $\Leftrightarrow$ tunnel sécurisé

- i.e. tunnel avec cryptage des paquets circulant dans le tunnel

■ Techniquement

- Tout paquet IP émis depuis une machine du site 1 ou du site 2 à destination d'une machine de l'Internet (par ex. S)
 - sera NAT-isé,
- Tout paquet IP échangé entre machines des sites 1 et 2
 - sera VPN-isé



- c'est à dire : **crypté** par R₁ (si l'on souhaite des garanties de confidentialité)
- ... puis transmis via le **tunnel** IP établi entre R₁ et R₂, i.e :
 - encapsulé par R₁ dans un paquet IP
 - d'@ IP_{dest} = @ IP_{R2-valide} et d'@ IP_{srce} = @ IP_{R1-valide}
 - désencapsulé par R₂
- ... puis **décrypté** par R₂
- et finalement aiguillé vers la machine destinataire du site 2

■ Rmq

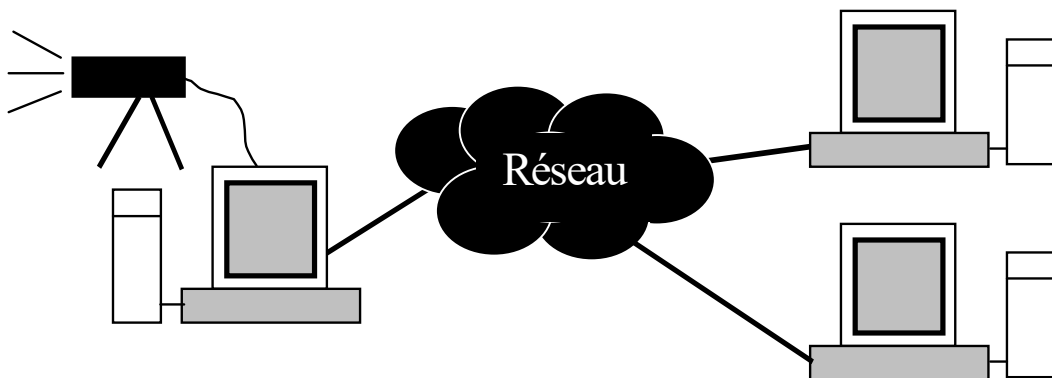
- Si pas de confidentialité requise $\Rightarrow$ suppression des phases de cryptage / décryptage sur la partie VPN

5. Gestion du multicast IP dans l'Internet (bases)

5.1. Introduction

- **Contexte = applications « multipoints »**

- Nécessitant des échanges de 1 vers N, et + généralement de N vers M
 - Visioconférences, jeux distribués, ...



- **Besoin : support de transmission « multicast »** permettant à des échanges multicast au niveau applicatif

- en minimisant le nb de paquets émis par les couches sous-jacentes

- **Solutions**

- A l'échelle d'un réseau Φ :

- LAN, MAN $\rightarrow$ résulte de la propriété de diffusion de ces réseaux
- certains WAN : ATM par exemple

- A l'échelle d'une interconnexion de réseaux Φ supportant chacun le multicast :

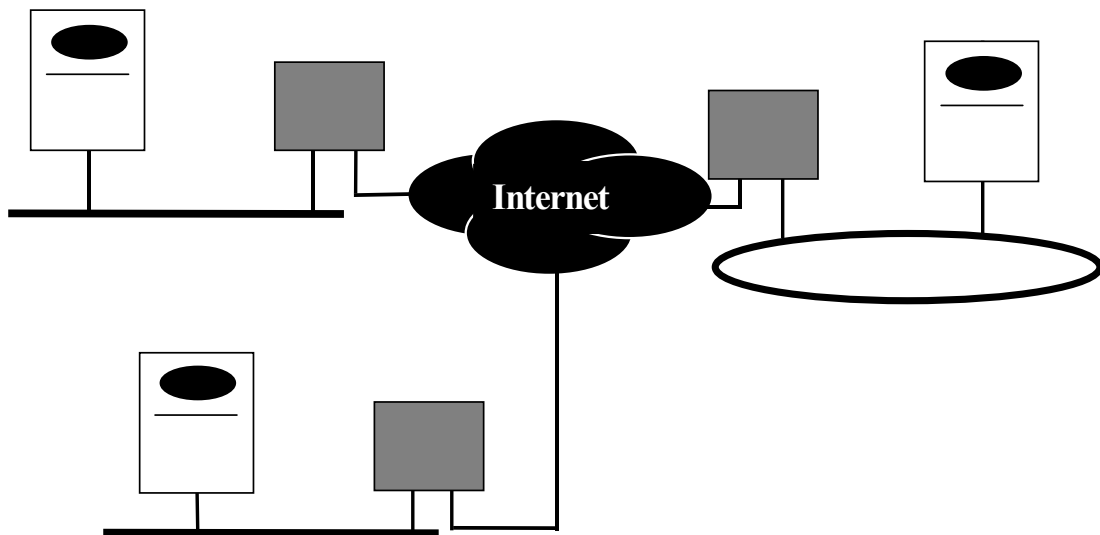
- Solution = IP multicast

5.2. IP Multicast

■ Histoire

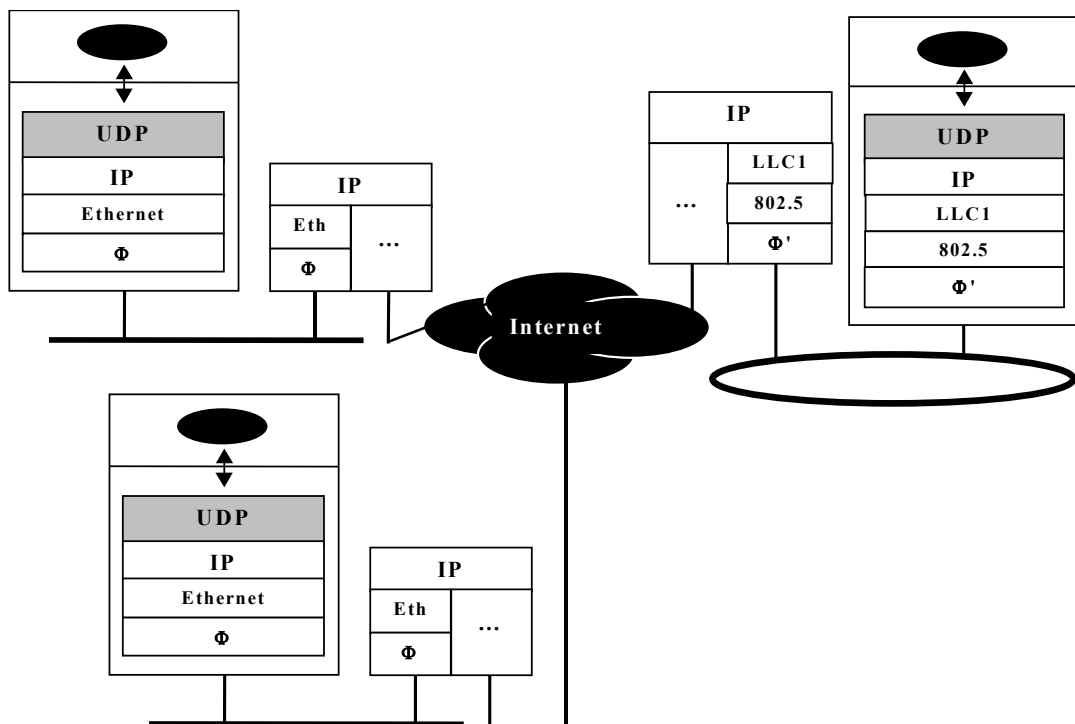
- Conçu par Steve Deering durant sa thèse entre 1988 et 91 [0]
- 1^{er} déploiement en 1992 → naissance du MBONE
- Solution étendue depuis (non traitée dans ce cours)

■ Modèle de communication



- Pour un processus applicatif :
 - adhésion ou retrait d'un groupe dynamiquement
 - réception de toutes les données émises à destination d'un groupe auquel on est abonné, ceci **sans garantie de QoS**
 - émission possible de données à destination d'un groupe auquel on n'est pas abonné

■ Concepts associés



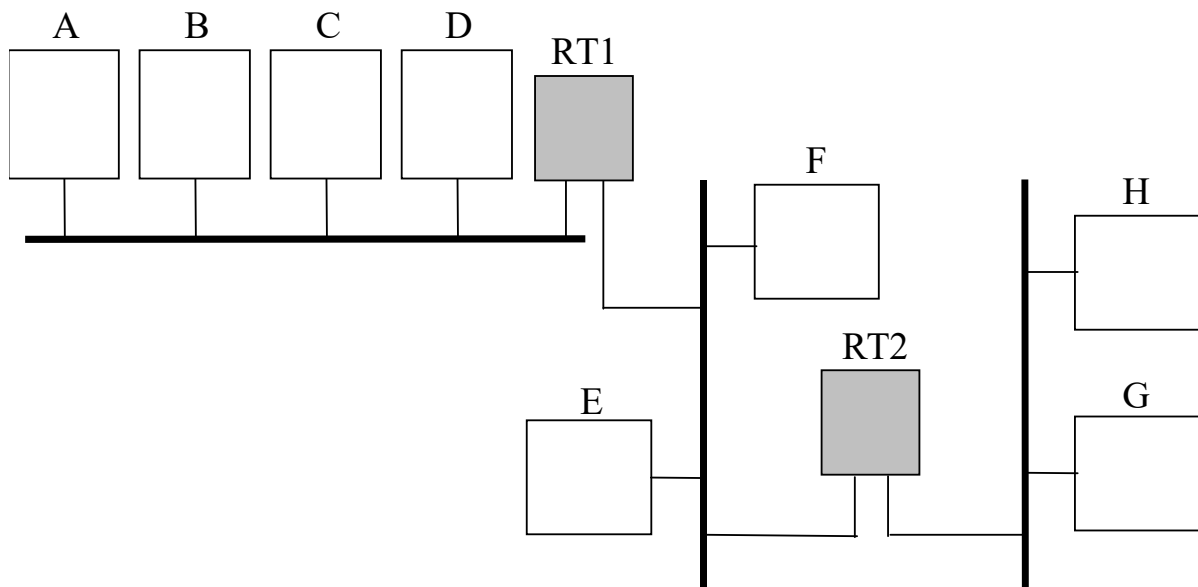
- Notion d'@ de groupe
 - @ IP de classe D (@ multicast)
- Processus d'adhésion et de retrait
 - via l'utilisation de primitives de service particulières
 - join(...), leave(...), ... →
- Emission/réception
 - similaire au modèle unicast → cf API socket : *sendto(...)*, *recvfrom(...)*
- QoS
 - de type *best effort* → UDP/IP Mcast
 - garantie ni d'ordre, ni de fiabilité

◆ Principes de mise en œuvre

■ Exemple introductif

– Soient 3 groupes de multicast auxquels se sont abonnés un ou plusieurs programmes applicatifs s'exécutant sur les machines B à G

- $G1 \rightarrow @IP_{G1} - \text{Hyp} : \text{Machines concernées} : B \text{ et } C$
- $G2 \rightarrow @IP_{G2} - \text{Hyp} : \text{Machines concernées} : C, D, E, F$
- $G3 \rightarrow @IP_{G3} - \text{Hyp} : \text{Machines concernées} : D, H, G$



– Scénario 1

- Emission par A d'un paquet IP « P » d'@ $IP_{dest} = @IP_{G1}$

– Scénario 2

- Emission par A d'un paquet IP « P » d'@ $IP_{dest} = @IP_{G2}$

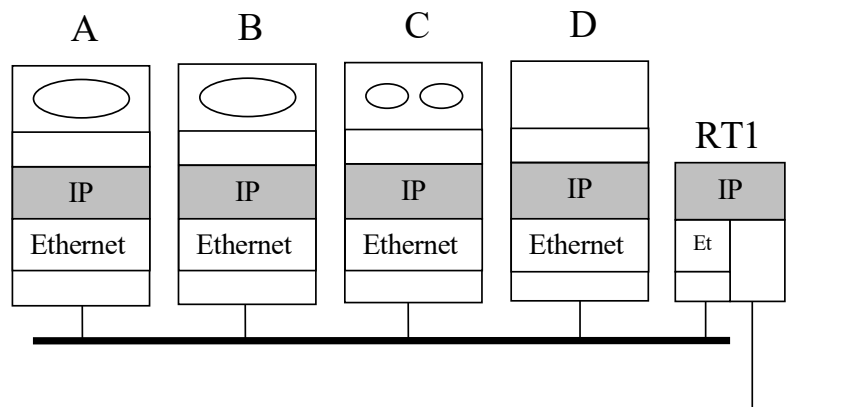
– Scénario 3

- Emission par A d'un paquet IP « P » d'@ $IP_{dest} = @IP_{G3}$

▪ Questions (besoins) :

- Comment se fait l'acceptation / rejet de la trame encapsulant P ? [1]
- Comment les routeurs savent-ils s'ils doivent router P et si oui directement ou bien vers quel prochain routeur ? [2]

▪ Réponse (solution) à [1]



- Pour toute interface Φ supportant le multicast
 - En émission
 - Tout paquet IP mcast P est encapsulé dans une trame MAC d'@
dest = @ MAC mcast « correspondant » à l'@IP dest de P
 - cf règle de correspondance dans le slide suivant
 - En réception
 - Lorsqu'un pgme applicatif adhère à une @ de groupe de niveau IP :
 - Activation sur la carte réseau d'un « groupe mcast de niveau MAC » construit suivant la même règle de mise en correspondance qu'en émission
 - Sur réception d'une trame MAC, si l'@MAC_{dest} de la trame identifie un groupe mcast de niveau MAC « actif » :
 - acceptation de la trame et délivrance des données user
 - Sinon : rejet de la trame

▪ Règle de mise en correspondance « @IP mcast ↔ @MAC mcast »

– Application au cas des réseaux Ethernet

- La RFC 1700 définit la « plage » des @ Ethernet mcast qui peuvent déduites d'une @IP mcast :

– cette plage va de $@_0 = 01:00:5E:00:00:00$ à $@_n = 01:00:5E:7F:FF:FF$

– Règle de construction d'une @ Eth. mcast à partir d'une @IP mcast

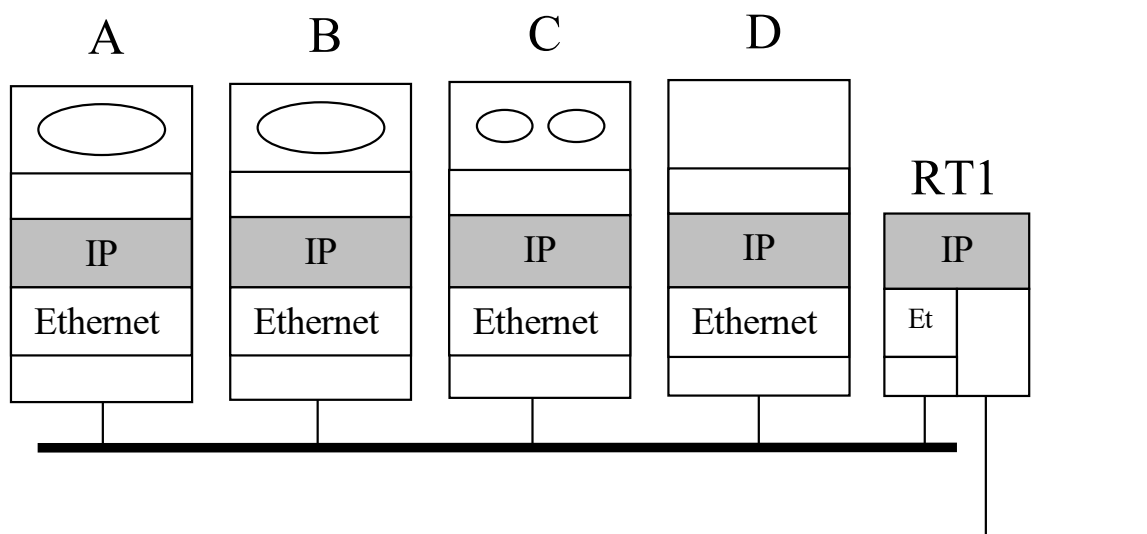
- Affectation des valeurs des 23 bits de poids faible de l'adresse $@_0$ à celles des 23 bits de poids faible de l'adresse IP mcast

- Ex :

– @ IP mcast = 224.32.16.1

– @ Ethernet mcast correspondante = 01:00:5E:XX:YY:ZZ

- Déterminer les valeur de XX, YY et ZZ



➤ **Rmq (il fut un temps ...)**

▪ **Pour toute interface Φ ne supportant pas le multicast**

- Pas de mise en correspondance @IP mcast $\leftrightarrow$ @MAC mcast
- En émission :

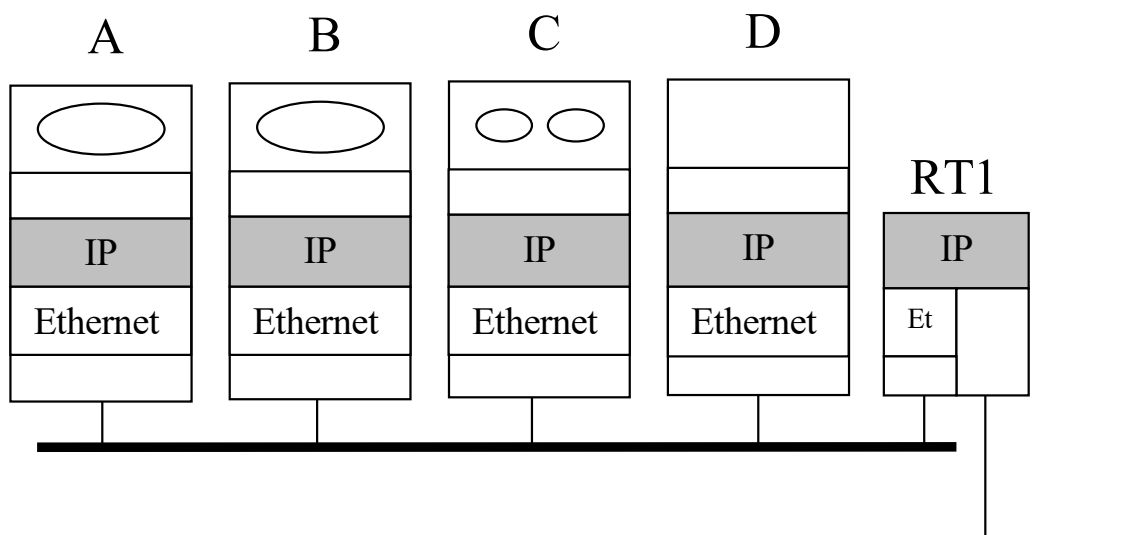
- tout paquet IP mcast P est encapsulé dans une trame MAC

d'@ dest = @ MAC broadcast

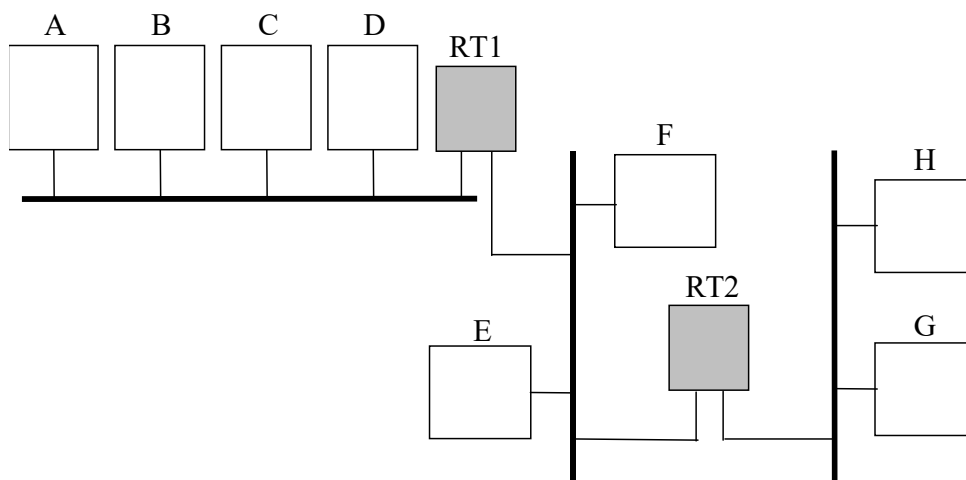
- En réception :

- acceptation systématique de la trame reçue
- et délivrance des données user à IP

➔ acceptation ou rejet des données au niveau IP !

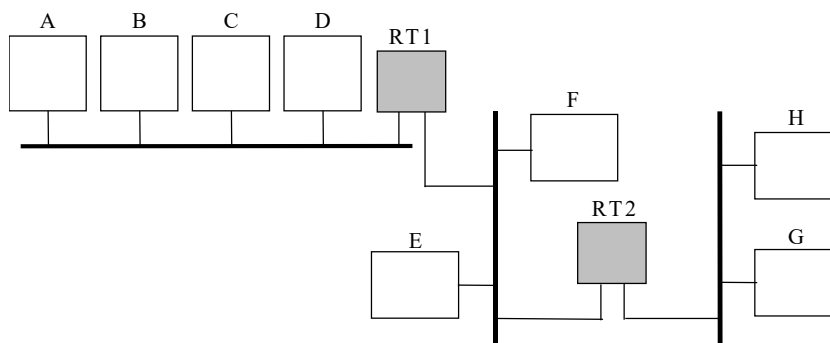


- **Réponse (solution) à [2] :** Comment les routeurs savent-ils s'ils doivent router P et si oui, directement ou bien via quels prochains routeurs ?
 - Mise en œuvre entre les routeurs d'un « **protocole de routage multicast** »
 - **Objectif :** permettre à chaque routeur
 - de savoir s'il doit aiguiller un paquet IP mcast entrant vers tout ou partie de ses interfaces Φ , via tel et tel prochains routeurs
 - **Rmq** : protocole le plus utilisé dans l'Internet = DVMRP
 - **DVMRP** $\leftrightarrow$ protocole de routage multicast intra domaine (NB : non étudié dans ce cours)
 - Parallèlement
 - Mise en œuvre d'un autre protocole : IGMP
 - IGMP : Internet Group Multicast Protocol
 - **Objectif :** permettre à tout routeur multicast
 - d'interroger les machines de ses réseaux IP de rattachement afin de savoir s'il y a des programmes en attente de données multicast



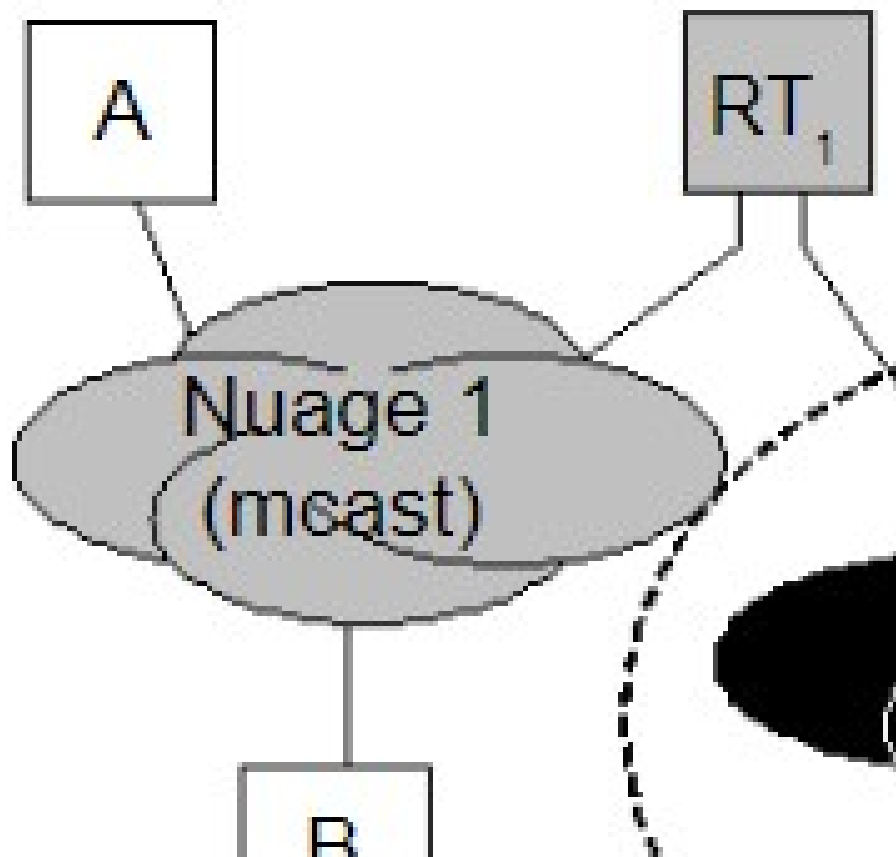
■ Le protocole IGMP

- Protocole véhiculé par IP (protocol = 2)
- 2 PDU IGMP principaux
 - « demande d'hôtes participants à un groupe » $\Leftrightarrow$ « à quel(s) groupe(s) mcast IP êtes-vous abonnés ? »
 - « compte rendu d'hôte participant à un groupe » $\Leftrightarrow$ réponse de type « moi, hôte X, je suis abonné à tel et tel groupe mcast IP »
- Hôte et routeurs émettent et reçoivent les PDU IGMP sur l'@ mcast 224.0.0.1 = adresse réservée à cette fin
- Chaque routeur émet périodiquement un PDU de demande à l'@ réservée : 224.0.0.1 avec un TTL = 1
 - TTL = 1 pour ne pas interroger les hôtes au-delà de ses réseaux IP
- Sur réception d'une demande, les hôtes « sont sensés » émettre un PDU de compte rendu par groupe auquel ils appartiennent
 - **Risque** : saturation du routeur source face au nombre de réponses
 - Ex : 10000 réponses en même temps $\Rightarrow$ saturation du réseau et des ressources du routeur $\Leftrightarrow$ problème de mise à l'échelle
 - Solution
 - Tirage au sort d'un temps d'attente avant émission des CR
 - Non réponse si réception d'un CR identifiant un groupe local



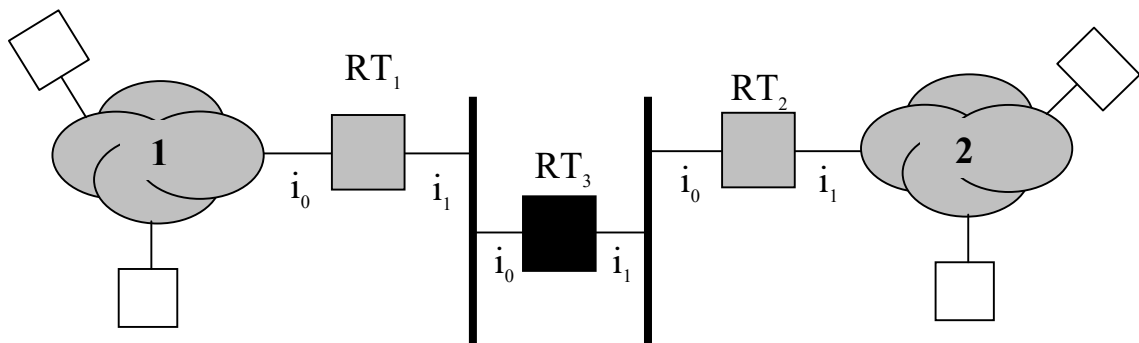
♦ Déploiement d'IP multicast dans l'Internet actuel

- 1991 : aucune machine ne déploie IP multicast
- 1992 : naissance du MBONE (Mcast BONE)
 - Interconnexion de machines (hôte/routeur) de l'Internet sur lesquelles sont déployées IP multicast
- Problème
 - Interconnexion de ces machines (en blanc pour les hôtes / en gris pour les routeurs) par le biais de routeurs NON MULTICAST (en noir)
 - Sur réception d'un paquet IP d' $@_{\text{dest}} = @ \text{ mcast}$
 - Action d'un routeur non mcast = rejet du paquet !



▪ **Solution = « Tunneling » IP ($\Leftrightarrow$ « *IP_{mcast} dans IP* »)**

- **Idée** : établir un tunnel entre R_1 et R_2 qui masque les paquets IP mcast au routeur RT_3
- Concrètement
 - Hyp
 - RT_1 souhaite router un paquet IP mcast P vers le nuage 2 via RT_2
 - Au niveau de RT_1
 - Encapsulation de P dans un paquet IP P' **non multicast** émis de RT_1 à destination de RT_2
 - Emission de P' sur l'interface i_1
 - Au niveau de RT_3
 - Traitement du paquet P' reçu et routage vers RT_2
 - Au niveau de RT_2
 - Sur réception de P' , désencapsulation de P et routage de P vers le nuage 2



VI. Les protocoles de Transport UDP et TCP

1. Introduction

2. UDP : User Datagram Protocol

3. TCP : Transmission Control Protocol

1. Introduction

1.1. Liaison vs Transport

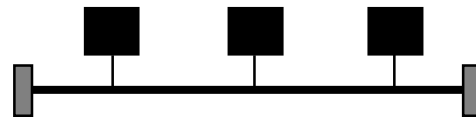
▪ Deux façons d'interconnecter deux hôtes distants

– soit directement

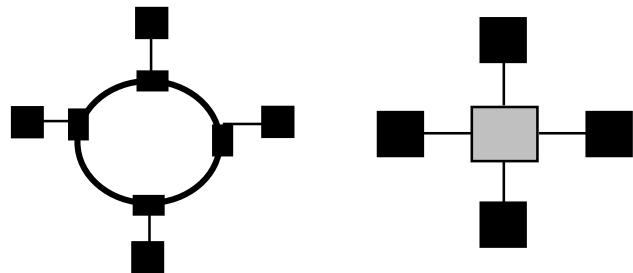
- lien Φ de point à point



- lien Φ multipoint

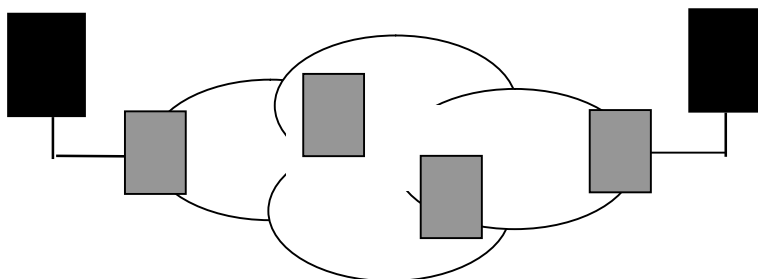


- N liens Φ de point à point



– soit indirectement

- par le biais d'un réseau commuté



▪ Dans les deux cas

– Principal problème commun

- Détection et reprise des pertes survenues entre les 2 machines et résultant :

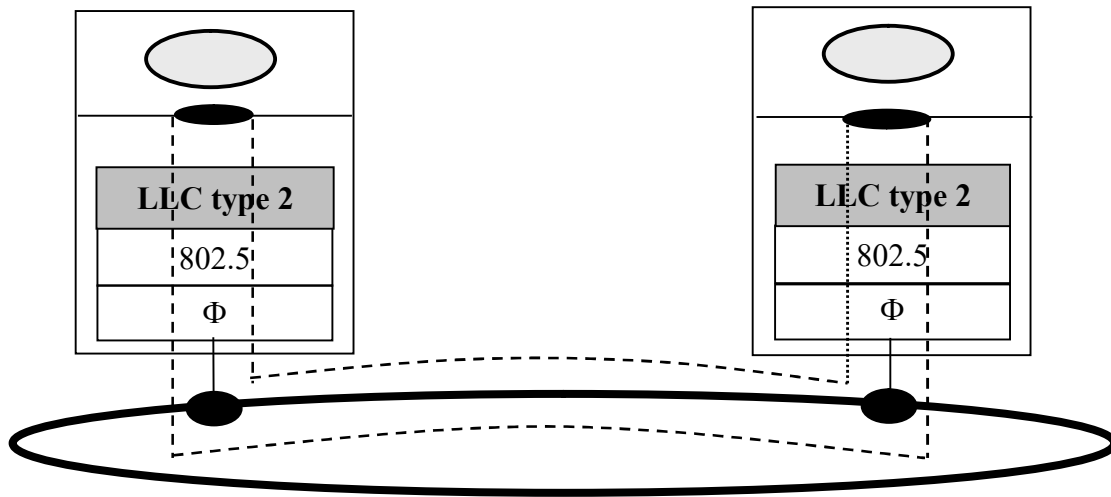
Interconnexion directe	Interconnexion indirecte
▪ erreur bit non corrigibles – rejet au niveau MAC ▪ congestion de la machine réceptrice (hôte ou routeur)	▪ erreur bit non corrigibles ▪ congestion d'un switch/routeur ▪ congestion de l'hôte récepteur ▪ en prime : déséquencelement

– Résolution du problème

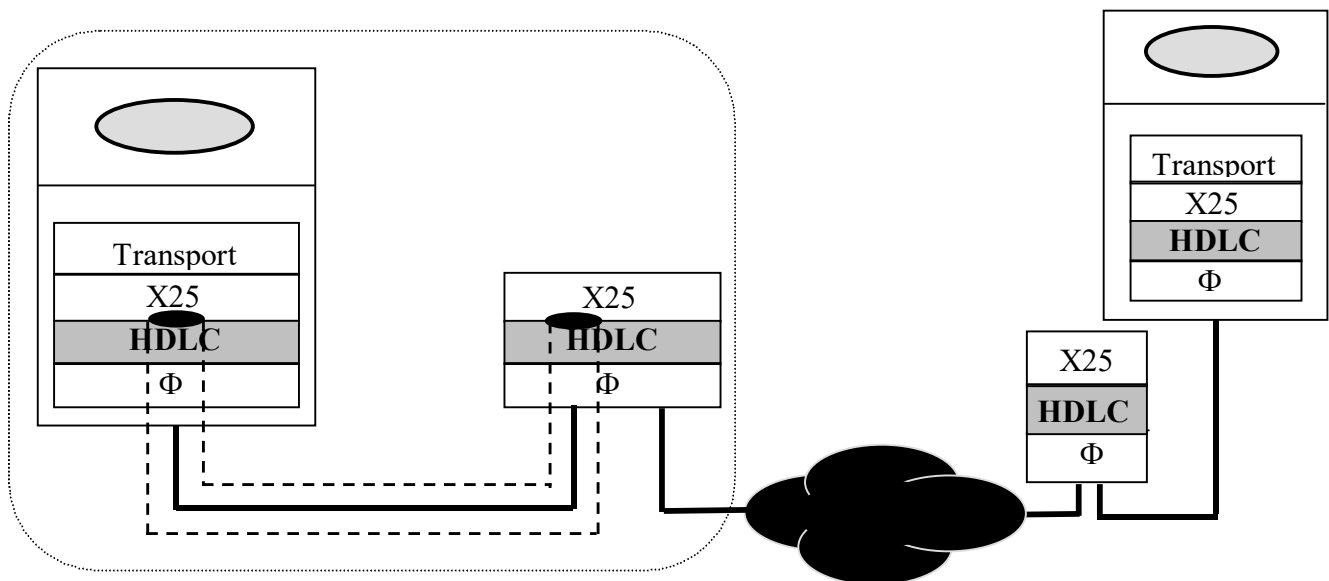
- Repose sur la mise en œuvre d'un protocole entre machines émettrice et réceptrice dont l'objectif est d'émuler :
 - pour le(s) entités de niveau supérieur ...
 - ... la disponibilité d'un canal de communication exempt de pertes et d'erreurs bit
- ⇔ canal de communication (ou service) fiable :
- de point en point
 - si les 2 machines sont **directement connectées**
 - de bout en bout
 - si les 2 machines (hôtes) sont **indirectement connectées**

▪ **Dans le 1^{er} cas : interconnexion directe**

- Emulation du canal de point à point par mise en œuvre :
 - d'un protocole « **de niveau Liaison** »
- **Ex 1** : cas d'un réseau purement local

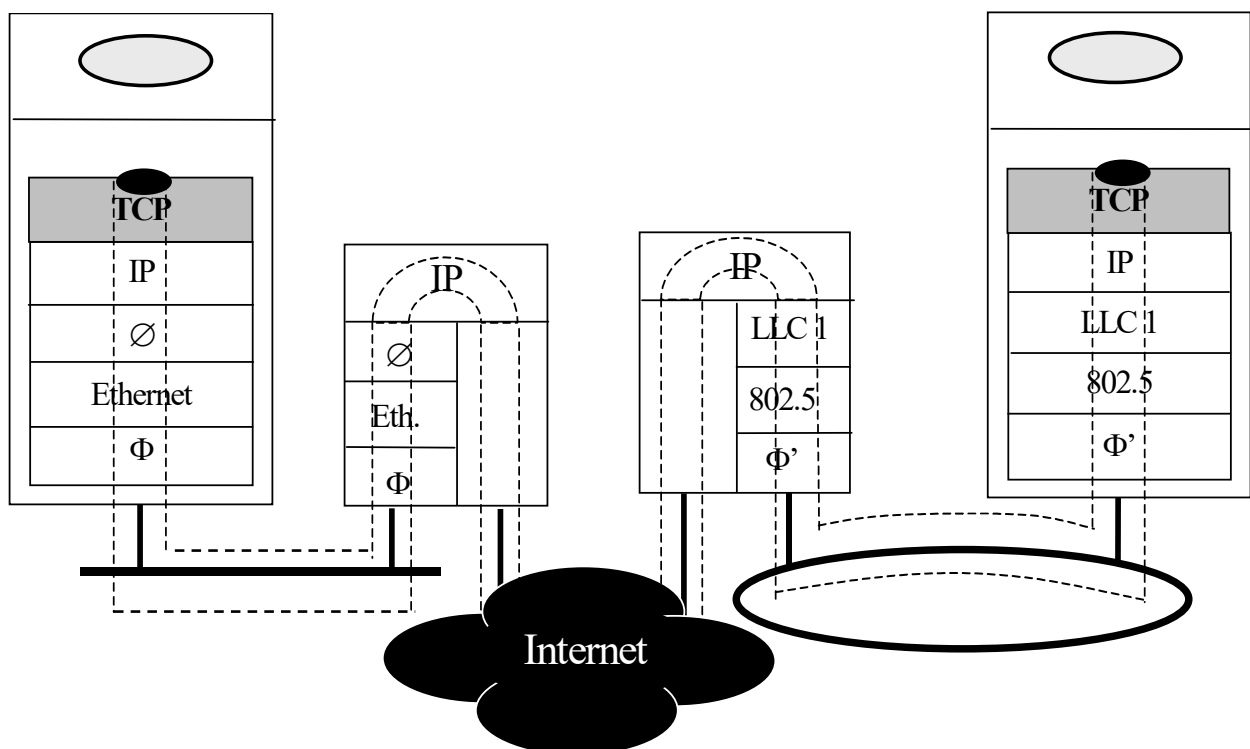


- **Ex 2** : cas d'un réseau commuté (type X25)



▪ **Dans le 2^{ème} cas : interconnexion indirecte**

- Emulation du canal de bout en bout par mise en œuvre :
 - d'un protocole « **de niveau Transport** »
- **Ex** : cas de l'Internet (IP)
 - Protocole = TCP



- Rmq
 - Un canal de niveau Transport relie toujours des processus applicatifs (via leur socket)
 - Si UDP au lieu de TCP
 - Alors : canal sans garantie de fiabilité ni d'ordre

▪ Rmq importante

- Plusieurs ≠ entre protocoles de niveau Liaison et Transport

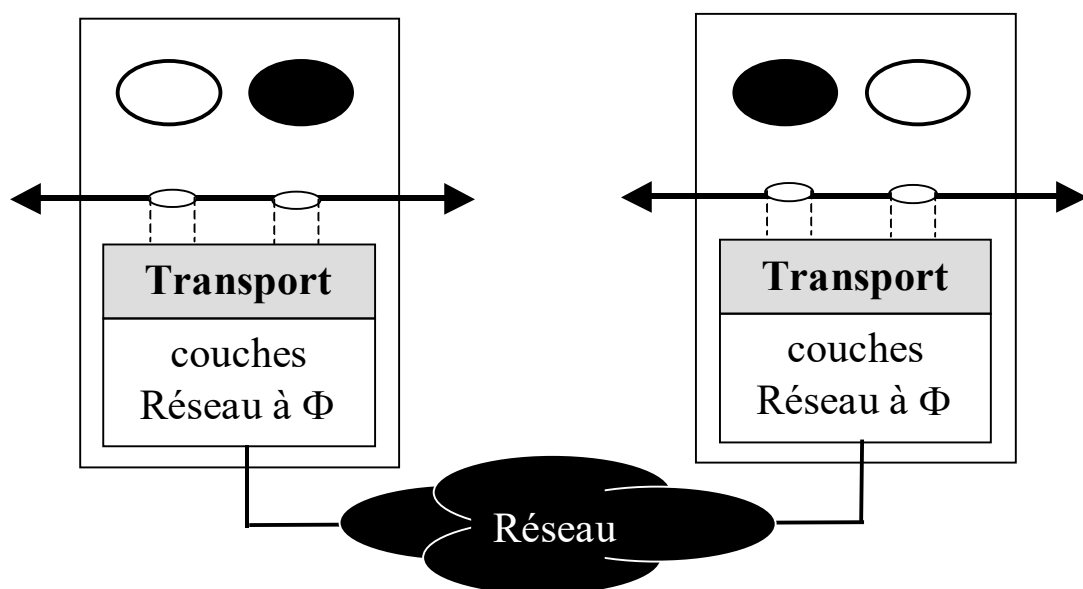
→ liées au contexte de mise en œuvre

- Pour un canal de point à point → NIVEAU LIAISON
 - pas de déséquencelement des trames échangées
 - pas (peu) de variabilité dans le délai de transit des trames
 - sources de perte
 - rejet d'une trame erronée (niveau 2)
 - rejet d'une trame reçue par congestion de la machine dest. (niveau 2)
- Pour un canal de bout en bout → NIVEAU TRANSPORT
 - déséquencelement possible des PDU Transport selon le type de commutation appliqué au niveau Réseau
 - variabilité dans le délai de transit des PDU Transport
 - sources de perte
 - rejet d'une trame erronée (niveau 2)
 - rejet d'un paquet par congestion d'un nœud intermédiaire (niveau 3)
 - rejet d'un PDU Transport reçu par congestion du hôte dest (niveau 4)

1.2. Services d'une couche Transport

▪ 2 types de service

- assurant tous les deux un transfert de données
 - de processus applicatif à processus applicatif
 - **bidirectionnel**, i.e. : communication possible dans les deux sens
- mais garantissant une QoS différente selon qu'ils sont :
 - **avec connexion** (TCP dans l'Internet)
 - établissement d'une connexion entre processus applicatifs distants avant échange des données
 - QoS = ordre total + fiabilité totale
 - **sans connexion** (UDP dans l'Internet)
 - échange hors connexion des données entre processus applicatifs
 - QoS = rien : ni ordre, ni fiabilité



▪ 2 modes de transfert d'une donnée applicative D

– le mode message (ou datagramme)

- du point de vue du service de Transport :
 - D est considérée comme un message indépendant des messages déjà soumis ou à venir



la limite entre 2 données soumises consécutivement doit être préservée lors de leur délivrance à l'utilisateur récepteur

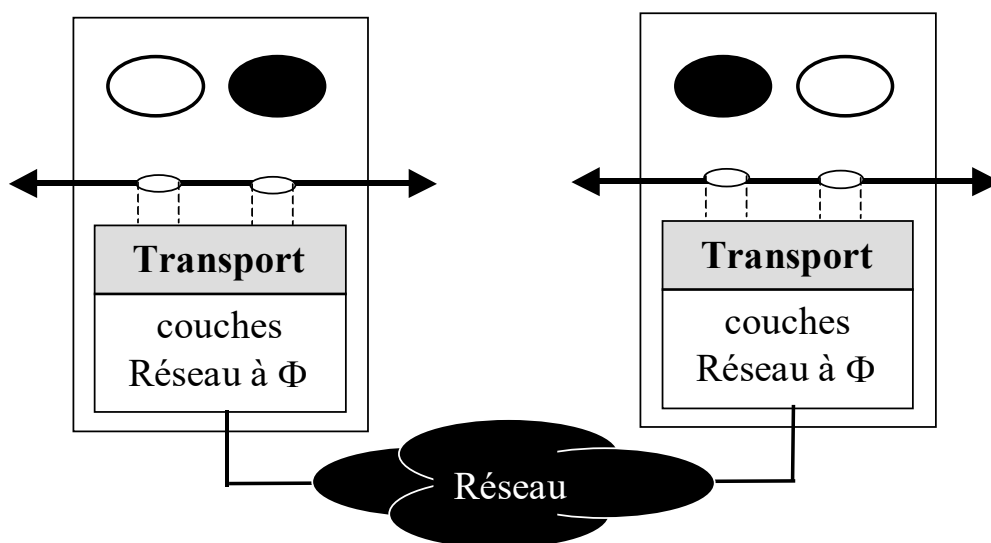
– le mode flux d'octets

- du point de vue du service de Transport :
 - D est considérée comme faisant partie d'un unique flux d'octets



la limite entre 2 données soumises consécutivement n'a pas à être préservée lors de leur délivrance à l'utilisateur récepteur

(la notion de message n'existe pas)



▪ Application à l'Internet

- 2 protocoles de Transport majeurs dans l'Internet : **TCP et UDP**
 - TCP (Transmission Control Protocol)
 - protocole avec connexion
 - fournit un service de transfert de données de bout en bout :
 - avec connexion
 - en mode flux d'octets
 - offrant comme QoS : ordre et fiabilité
 - conçu pour être déployé dans un environnement réseau sans garantie d'ordre ni de fiabilité, plus précisément :

dans une interconnexion IP de réseaux Φ à QoS $\neq$

- UDP (User Datagram Protocol)
 - protocole sans connexion
 - fournit un service de transfert de données de bout en bout :
 - sans connexion
 - en mode message
 - sans aucune garantie d'ordre ni de fiabilité

1.3. Fonctions d'une couche Transport

■ Identification de la machine destinataire

→ Nécessaire pour invoquer le service sous-jacent

– Dans l'Internet

- Une @ de niveau Transport (utilisée par un pgme applicatif pour désigner son interlocuteur au service sous-jacent) est composée :

- de l'@ IP de la machine sur laquelle s'exécute l'interlocuteur
- du n° de port utilisé par ce dernier

→ l'identification de la machine destinataire est donc contenue dans l'@ fournie par l'application émettrice !

■ Mise à disposition d'une interface de programmation (API)

- permettant l'accès à 2 types de canaux (ou services)
 - l'un à QoS = Ordre + Fiabilité, l'autre à QoS = rien

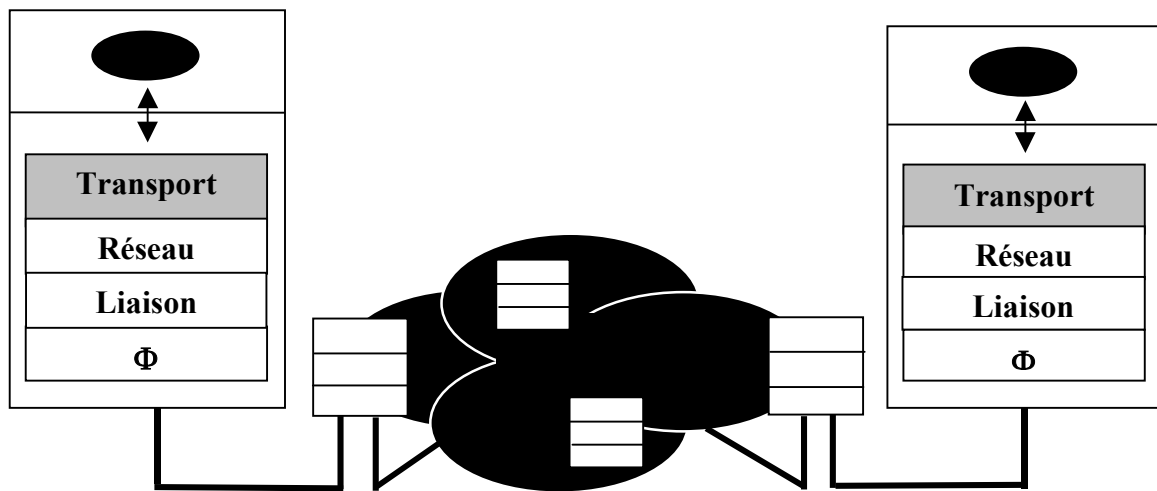


l'API socket dans l'Internet

■ Résolution des problèmes liés à un transfert de données

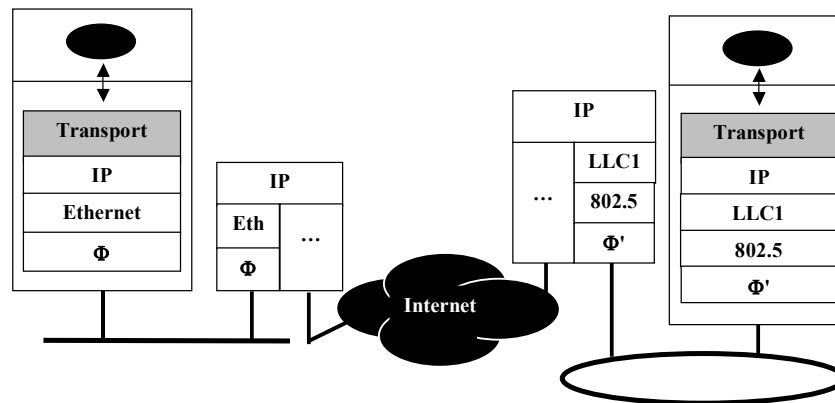
- au travers d'un réseau commuté (pertes, déséquilibrages, ...)
- et ce :
 - en fonction de la QoS à garantir !

■ Imperfections potentielle du réseau sous-jacent



- **En terme d'ordre** : « déséquencelement des paquets »
 - conséquence (possible) d'une commutation des paquets au travers du réseau sans connexion au préalable
- **En terme de fiabilité** : « perte de paquet(s) »
 - conséquence de pertes non récupérées aux niveaux sous jacents :
 - rejet d'une trame sur détection d'erreur(s) bit
 - rejet d'un paquet par congestion d'un switch / routeur
 - **en outre** : indépendamment du réseau sous jacent :
 - pertes possibles par congestion des buffers de rec du socket destinataire
- En termes de limite sur la taille des données utilisateurs
- En termes de limite sur le nb de points d'accès au service Réseau
 - ⇔ nombre limité de « canaux » offerts par le niveau Réseau

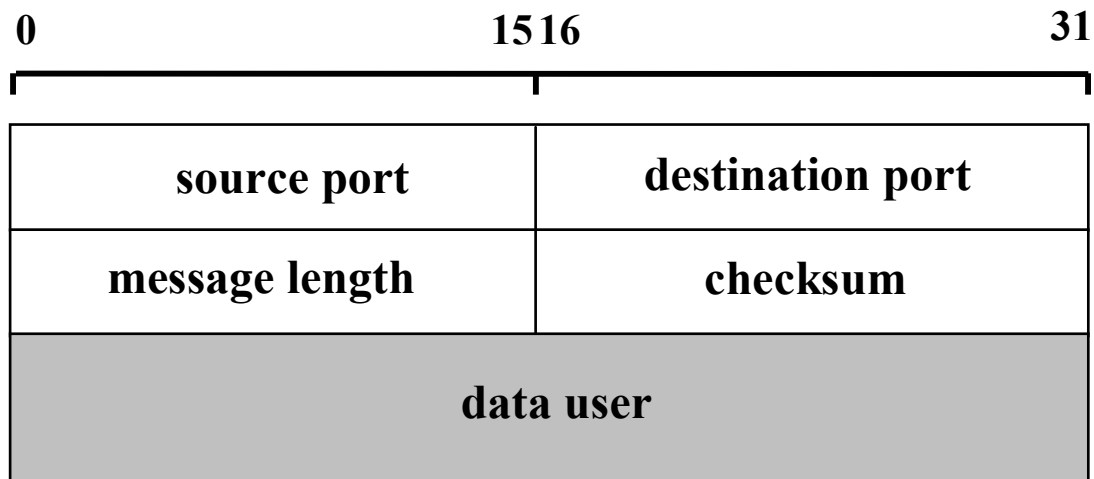
■ Application à l'Internet



- Caractéristiques du réseau sous-jacent (IP)
 - commutation en mode datagramme
 - perte et/ou déséquenceement possibles des paquets IP
 - un seul point d'accès par protocole de Transport
 - un canal offert à TCP, un canal offert à UDP
 - pas de limite sur la taille des données user
 - pas d'hypothèse sur les réseaux Φ traversés
- Fonctions en conséquence attendues d'un Transport :
 - garantissant ordre et fiabilité → **TCP**
 - MUX/DMUX des canaux TCP sur canal IP
 - contrôle des pertes
 - contrôle des déséquenceements : intégré dans la reprise des pertes
 - + mise en relation des appli avant le début des échanges
 - ne garantissant rien (ni ordre ni fiabilité) → **UDP**
 - MUX/DMUX des canaux UDP sur canal IP

2. UDP : User Datagram Protocol [RFC 768]

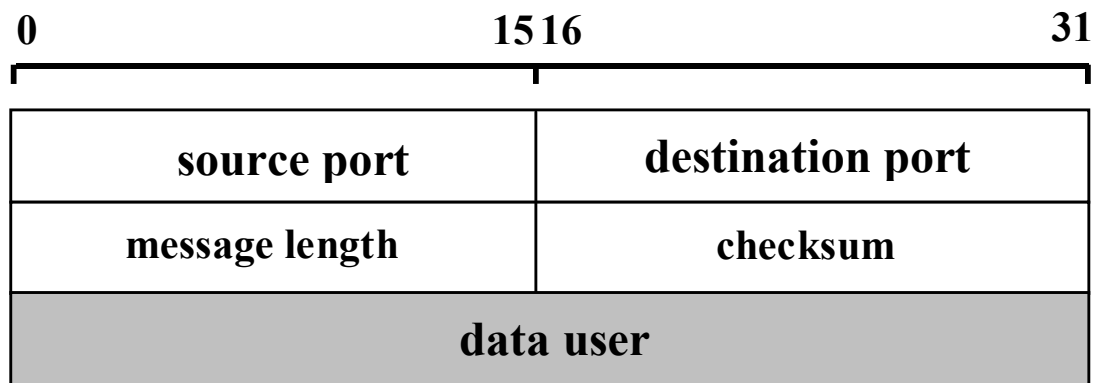
2.1. Format des PDU



- **source port**
 - n° de port source du paquet
- **destination port**
 - n° de port destination du paquet
- **message length**
 - longueur totale du paquet en octets (entête incluse)
- **checksum (contrôle d'erreur) :**
 - si = 0 ⇔ non appliqué
 - sinon ⇔ calculé sur l'entête + le pseudo header
 - pseudo header = adresses IP source et destination

2.2. Mécanismes protocolaires

■ Une seule phase : le transfert des données



– Mécanismes mis en œuvre

- Génération d'un PDU UDP à chaque invocation de la primitive d'émission par le processus applicatif émetteur
- MUX/DMUX des canaux UDP sur l'unique canal IP dédié à UDP
 - Objectif : un canal UDP par couple d'instances d'application
- Contrôle d'erreur portant sur l'entête des paquets ainsi que les adresses IP source et destination

➤ Rmq importantes

- Préservation de la frontière des messages soumis en émission lors de leur délivrance au processus applicatif récepteur
- Pas de segmentation/réassemblage des messages trop longs au regard de la MTU des réseaux traversés
 - car mécanisme mis en œuvre au niveau IP

3. TCP : Transmission Control Protocol [RFC 793, 1323]

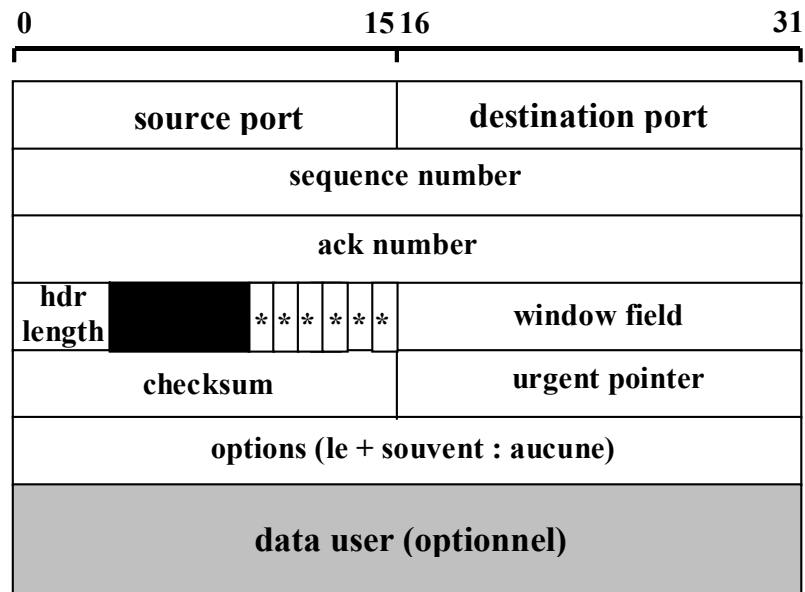
3.1. Format des PDU

- Entités TCP émettrice et réceptrice échangent des « **segments** »
 - ⇔ 20 octets d'entête (min) + données user optionnelles
- **2 règles**
 - Taille d'un segment ≤ 65535 octets
 - Taille d'un segment $\leq$ MTU du réseau auquel appartient la machine moins la longueur de l'entête IP (- lg entête LLC s'il y a)
 - un paquet IP généré par segment TCP

➤ Rmq (rappel)

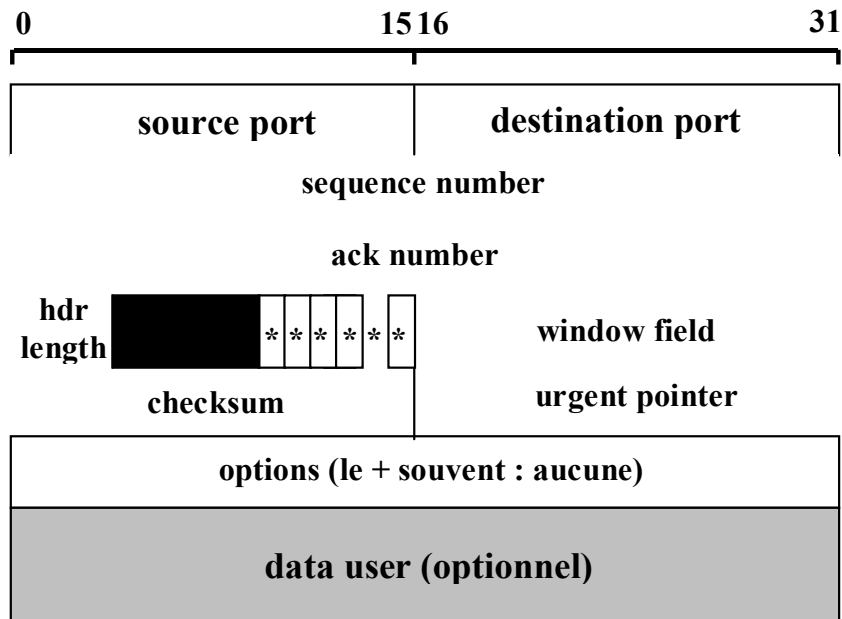
- **Si le paquet IP passe par un réseau intermédiaire de MTU inférieur à la taille du paquet**
 - fragmentation du paquet en plusieurs paquets par l'E(IP) du routeur d'entrée du réseau
 - réassemblage du paquet initial par l'E(IP) du hôte dest. et :
 - mise à disposition des données user à TCP si paquet IP entièrement réassemblé
 - rejet des fragments déjà reçus sinon

■ Format d'un segment TCP



avec : * * * * * = **URG | ACK | PSH | RST | SYN | FIN**

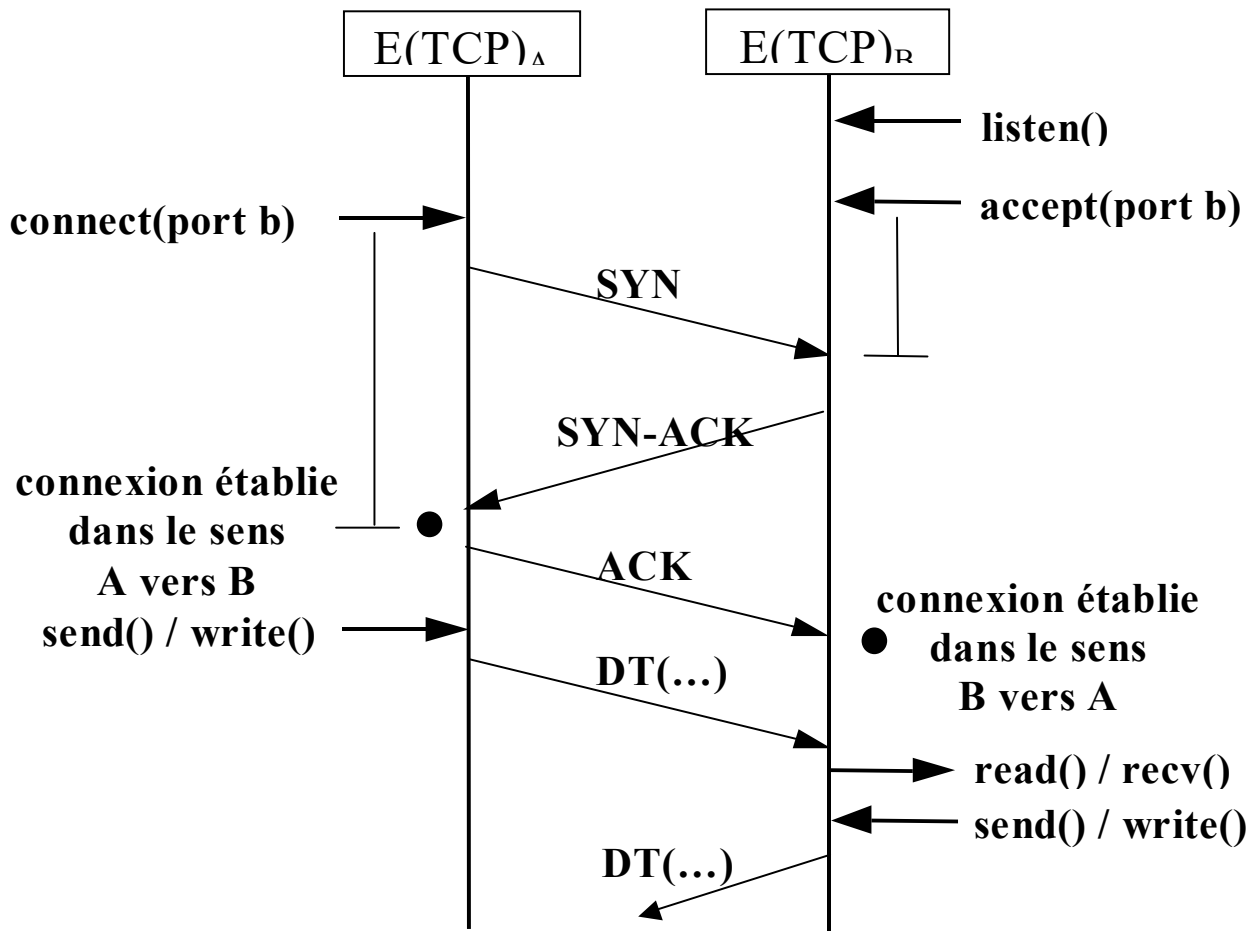
- **source et destination port** : 1^{ère} partie des @ TSAP src et dest
- **sequence number** : n° du 1^{er} octet de données du segment
- **ack number** : n° du prochain octet attendu
- **hdr length** : taille de l'entête TCP
 - nécessaire du fait de la taille variable de l'entête TCP
 - indique la position du 1^{er} octet de données dans le segment
- **checksum** : contrôle d'altération sur l'ensemble du segment
 - + pseudo entête
- **urgent pointer** : n° du dernier octet de données urgente
- **window field**
 - indique le nombre **d'octets de données user** que peut recevoir l'entité TCP émettrice du paquet



- URG | ACK | PSH | RST | SYN | FIN = 6 bits (valeur = 0 ou 1)
 - URG = 1
 - indique que le segment transporte des données urgentes
 - ACK = 1
 - indique que le champ « ack number » est valide
 - PSH = 1
 - indique que le segment contient des données à émettre puis à délivrer immédiatement au proc. applicatif récepteur
 - RST = 1
 - indique que la connexion doit être ré-établie (reset) ou pour rejeter une demande d'établissement de connexion
 - SYN = 1
 - indique que le segment est utilisé pour établir une connexion
 - FIN = 1
 - indique que le segment est utilisé pour libérer une connexion

3.2. Phase d'établissement de connexion

▪ Etablissement à 3 messages : SYN, SYN-ACK, ACK



▪ Durant cette phase

- négociation entre $E(TCP)$ communicantes :
 - du premier n° de séquence utilisé **pour chaque sens**
 - Rmq : n° du 1^{er} octet de données user et NON du 1^{er} segment !
 - PE = Prochain n° à Emettre et PA = Prochain n° Attendu
 - de la taille initiale du crédit d'émission **pour chaque sens**
 - optionnellement : de la taille max des segments

▪ Coté E(TCP)_A appelante

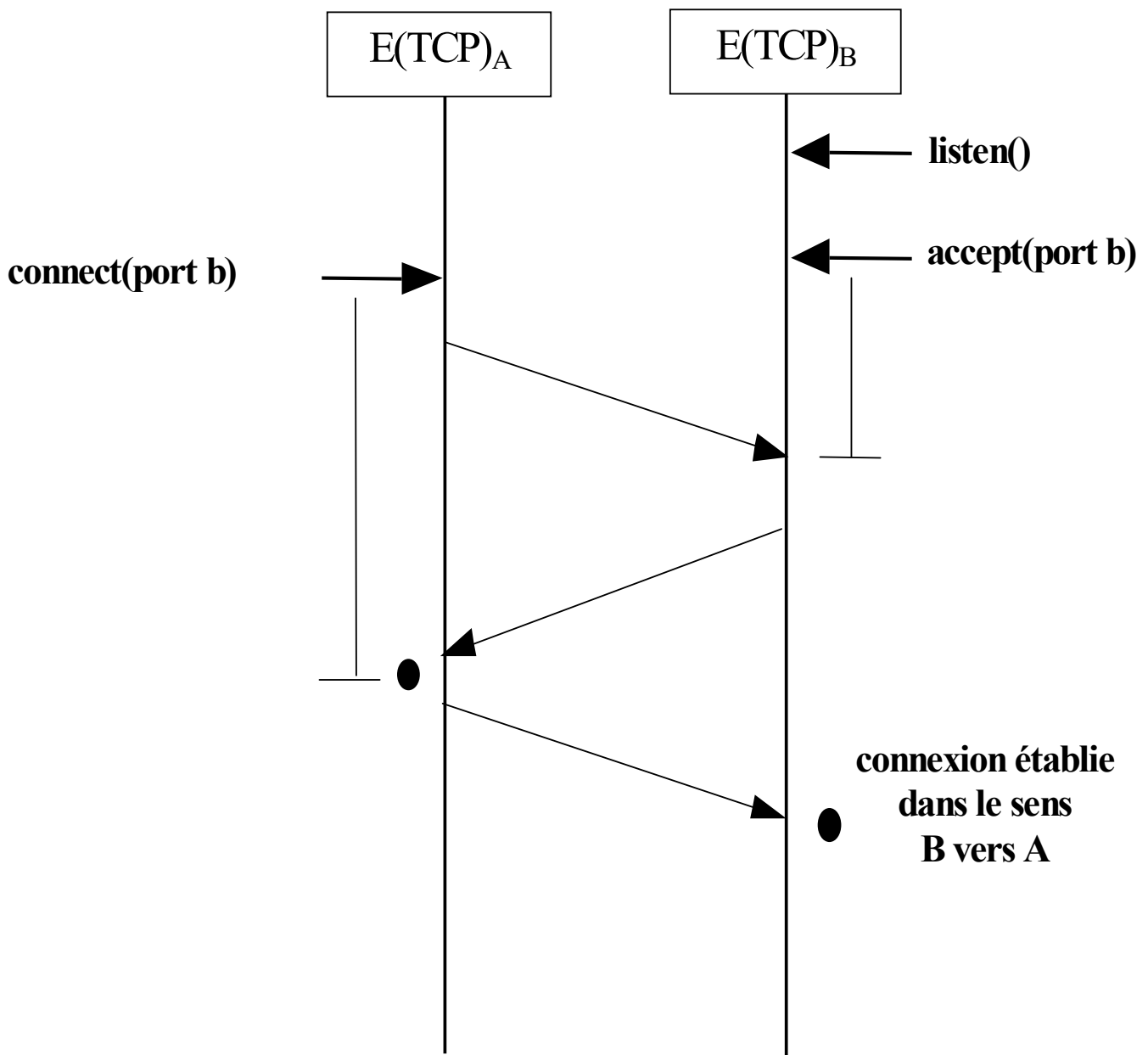
- Sur occurrence d'une requête de connexion depuis le socket d'@ { $@IP_A$, port = a} vers le socket d'@ { $@IP_B$, port = b}
 - $PE \leftarrow 32$ bits de poids faible de l'horloge = x
 - où : PE = prochain n° de séquence à émettre
 - émission d'un segment SYN = (dest_port=b, scr_port=a, SYN=1, seqnum=x, win_field=wf1)
 - $PE \leftarrow PE + 1$ car SYN $\Leftrightarrow$ émission d'un DT de 1 octet
 - mise en attente d'un segment SYN acquittant la requête ou d'un segment RST en cas d'échec
- Sur réception du ACK-SYN (dest_port=a, scr_port=b, SYN=1, ACK=1, seqnum=y, acknum=x+1, win_field=wf2) acquittant la requête
 - $PA \leftarrow y + 1$ et crédit $\leftarrow wf2$
 - où : PA = prochain n° de séquence attendu et crédit = nombre d'octets de données applicatives que peut envoyer l'E(TCP)_A
 - connexion établie dans le sens A vers B
 - émission d'un segment ACK acquittant le ACK-SYN reçu (dest_port=b, scr_port=a, ACK=1, acknum=y+1)
 - début du transfert des données de A vers B
 - $PE = x+1$, $PA = y+1$ et crédit = wf2
- Sur réception d'un RST (dest_port=a, scr_port=b, RST=1, ACK=1, acknum=NS+1)
 - échec de la tentative de connexion de A vers B
 - notification de l'échec au proc applicatif appelant

▪ Côté E(TCP)_B appelée

- Sur occurrence d'une demande d'acceptation de connexion entrante sur le socket de n° de port = b
 - $PE \leftarrow 32$ bits de poids faible de l'horloge = y
 - Mise en attente d'un segment SYN à destination du socket
- Sur réception d'un SYN = (dest_port=b, scr_port=a, SYN=1, seqnum=x, win_field=wfl)
 - **Si** $\exists$ processus en attente sur un socket de n° port = b **Alors** :
 - $PA \leftarrow x + 1$ (Rmq : + 1 car SYN coûte 1 octet)
 - crédit $\leftarrow wfl$
 - émission d'un ACK-SYN (dest_port=a, scr_port=b, SYN=1, ACK=1, seqnum=y, acknum=x+1, win_field=wf2)
 - mise en attente d'un ACK acquittant la requête de connexion
 - Sinon
 - émission d'un RST = (dest_port=a, scr_port=b, RST=1, ACK=1, acknum=x+1)
- Sur réception d'un ACK = (dest_port=b, scr_port=a, ACK=1, acknum=y+1, window_field=wfl)
 - connexion établie dans le sens B vers A
 - début du transfert des données de B vers A avec :
 - $PE = y+1$, $PA = x+1$ et crédit = wfl

➤ **Ex**

- Décomposer la phase d'établissement d'une connexion (réussie) ayant conduit à :
 - Coté A : PE = 32803, PA = 15402, crédit = 8000 octets
 - Coté B : PE = 15402, PA = 32803, crédit = 24000 octets



3.3. Phase de transfert des données

■ Principaux mécanismes mis en œuvre

1] Emission et remise des données user de façon $\neq$ à UDP !

- en émission
 - génération d'un segment TCP contenant la donnée à émettre
 - ou collection de plusieurs données avant émission du segment
- en réception
 - pas de gestion de la frontière des données soumises en émission
- **Rmq** : possibilité pour le processus applicatif émetteur
 - de forcer l'émission immédiate d'une donnée
 - d'envoyer des données urgentes, i.e. non soumises au contrôle de flux de TCP

2] **MUX/DMUX** des canaux TCP sur l'unique canal IP dédié à TCP

- un canal TCP par couple d'instances d'application

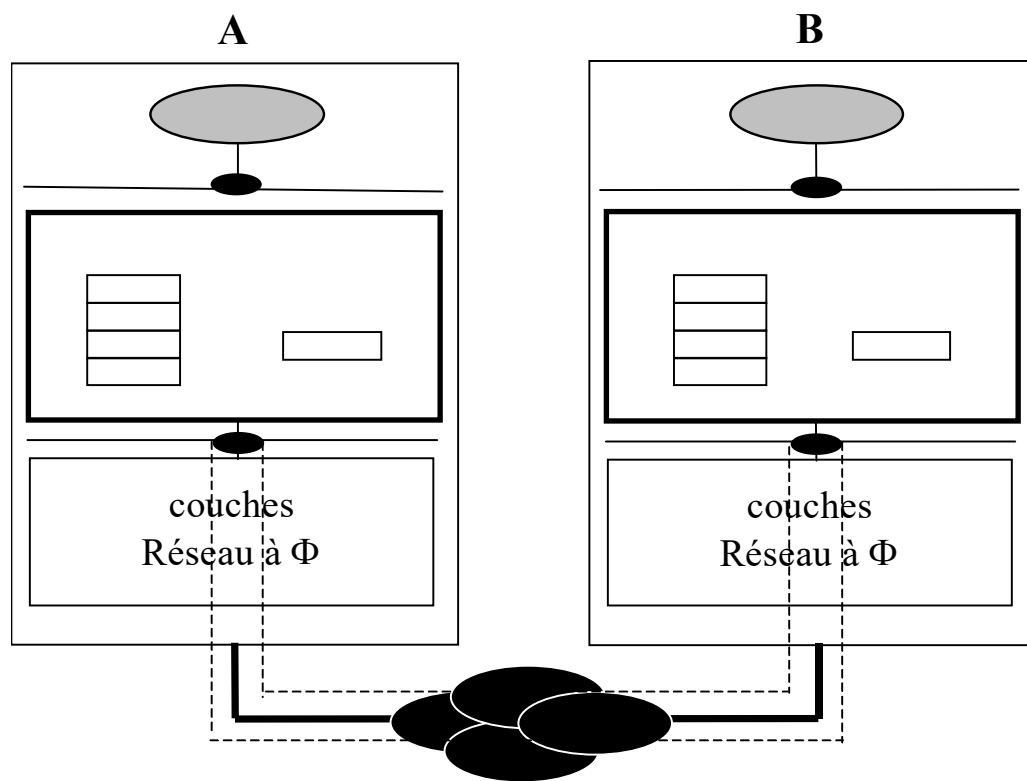
3] **Contrôle des pertes** incluant :

- reprise des pertes avec politique de rejet sélectif en réception
- contrôle de flux par fenêtre de taille variable
- contrôle des congestions du réseau de bout en bout

♦ Contrôle de flux par fenêtre de taille variable

1. Position du problème (rappel)

- Chaque hôte dispose de tampon(s) mémoire (« **buffers** ») dans lesquels sont stockées
 - les données applicatives à émettre
 - les données applicatives reçues en provenance du réseau



- **INDEPENDAMMENT** du réseau sous-jacent :
 - si le processus applicatif émetteur émet ses données à un rythme $>$ temps de consommation de ces données par le processus récepteur
 - remplissage des **buffers** de réception ...
 - et rejet des données qui arrivent en suivant
- ⇒ pertes par congestion du hôte destinataire

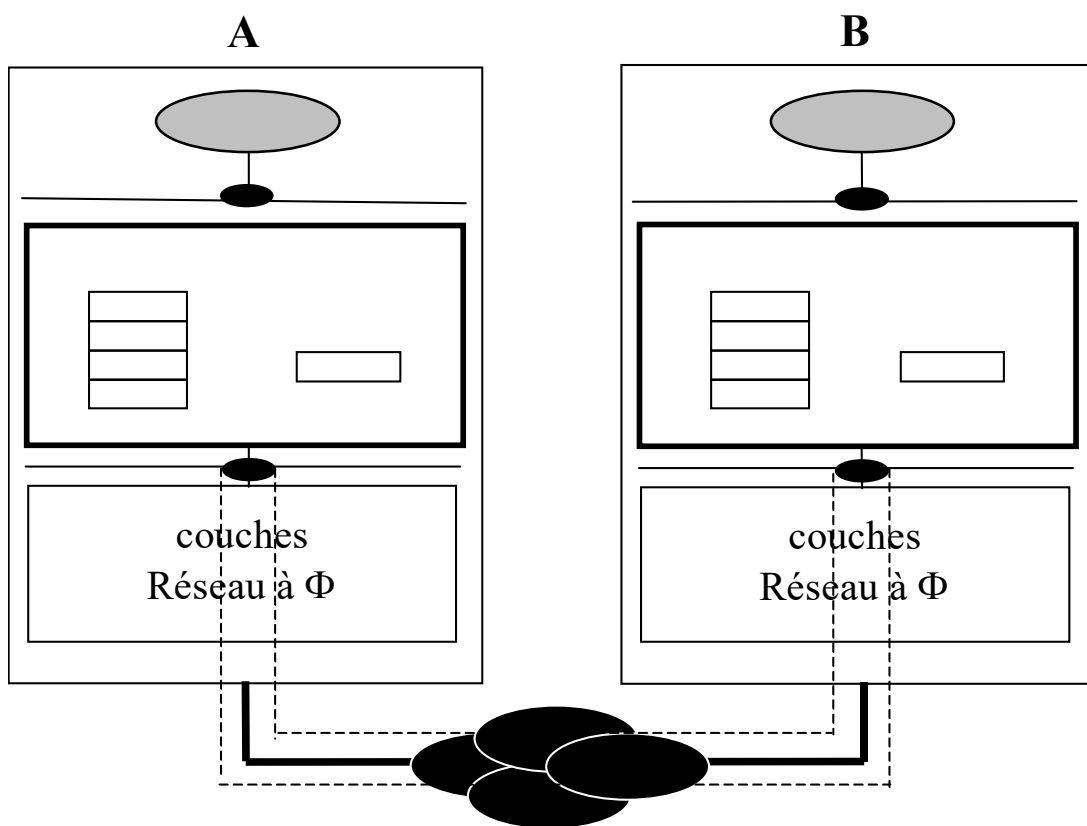
▪ Objectif d'un contrôle de flux

- Permettre au système de communication de l'hôte récepteur de réguler le flux d'émission de l'hôte émetteur
 - en fonction de l'état de remplissage de ses buffers de réception
- Rmq
 - Niveau de stockage des données reçues ou à émettre
 - Couche directement accessible par les pgmes applicatifs
 - Liaison → réseau purement local
 - Transport → réseau commuté
 - exception : X25
 - contrôle de flux de bout en bout au niveau 3 (X25)
 - contrôle de flux de point à point au niveau 2 (HDLC)
 - Au niveau Liaison (rappel)
 - ⇔ contrôle de type tout ou rien
- Inadéquat dans un contexte de bout en bout car :
 - délai de transit **variable** des PDU
 - contrôle simultané d'un grand nombre de connexions

2. Le contrôle de flux au niveau Transport

▪ Principe

- repose sur la notion de « **crédit d'émission** »
- variable maintenue par chaque E(T) indiquant le nb d'octets de donnée user qu'elle a le droit d'envoyer à cet instant

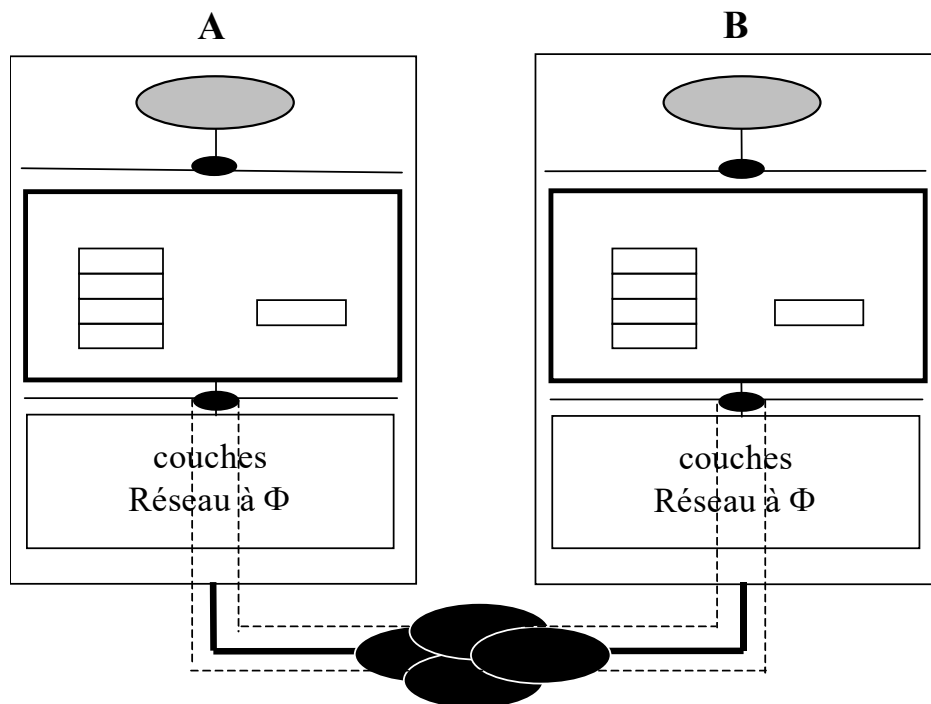


– Règle

- **Si** crédit > 0 **alors émission possible** d'un nouveau segment TCP
 - dont la taille des données user n'excède pas le crédit disponible
- **Sinon** arrêt des émissions ...
 - et blocage du processus applicatif émetteur lorsque les buffers d'émission sont pleins

- **Algo de mise à jour du crédit d'émission (version simplifiée)**
 - Hyp simplificatrices
 - On suppose que le crédit est exprimé en nb de buffers disponibles
 - Les PDU contenant des données user (PDU DT) sont supposés de taille fixe = taille d'un buffer
 - A chaque émission d'un PDU DT, l'E(T) émettrice :
 - décrémente son crédit : $\text{crédit} \leftarrow \text{crédit} - 1$
 - A chaque émission d'un PDU DT ou CTLE, l'E(T) émettrice :
 - transmet à l'E(T) réceptrice via les champs *ack number* et *window field* de l'entête du PDU :
 - le n° du prochain PDU DT attendu
 - $\text{ack_num} \leftarrow \text{PA} = \text{nseq du prochain PDU attendu}$
 - le nombre de buffers libres dont elle dispose en réception
 - $\text{win_field} \leftarrow \text{nb buffers libres}$
 - Sur réception d'un PDU DT ou CTLE
 - l'E(T) réceptrice met à jour son crédit d'émission :
 - $\text{crédit} \leftarrow \text{win_field} + \text{ack_num} - \text{PE}$
- où : PE désigne le n° du prochain PDU à émettre

➤ Ex



▪ Hypothèses

- Génération d'un ACK
 - suite à la réception d'un DT
 - à chaque lecture par l'utilisateur récepteur
- Après établissement de la connexion
 - côté $E(T)_A$:
 - nb de buffers libres en réception (NBL) = 5 ; crédit = 3
 - PE = 0 ; PA = 10
 - côté $E(T)_B$:
 - NBL = 3 ; crédit = 5
 - PE = 10 ; PA = 0

▪ **Algo mis en œuvre dans TCP (sans hyp simplificatrices)**

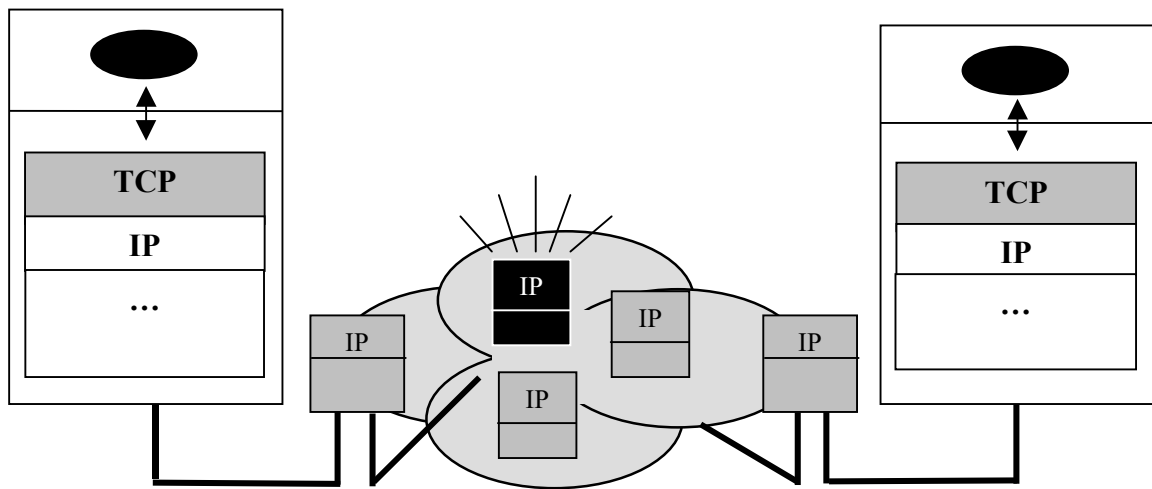
- A chaque émission d'un segment TCP (contenant N octets de données user), l'E(TCP) émettrice :
 - décrémente son crédit N octets : $\text{crédit} \leftarrow \text{crédit} - N$
 - A chaque émission d'un segment TCP, l'E(TCP) émettrice :
 - transmet à l'E(T) réceptrice via les champs *ack number* et *window field* de l'entête du PDU :
 - le n° du prochain octet de données user attendu = PA
 - $\text{ack_num} \leftarrow \text{PA}$
 - le nombre d'octets de buffers libres en réception
 - $\text{win_field} \leftarrow \text{nb d'octets de buffers libres}$
 - Sur réception d'un segment TCP :
 - l'E(TCP) réceptrice met à jour son crédit d'émission :
 - $\text{crédit} \leftarrow \text{win_field} + \text{ack_num} - \text{PE}$
- où : PE désigne le n° du prochain octet de données user à émettre

♦ Le contrôle de congestion

1. Préambule : l'algo du démarrage lent (slow start)

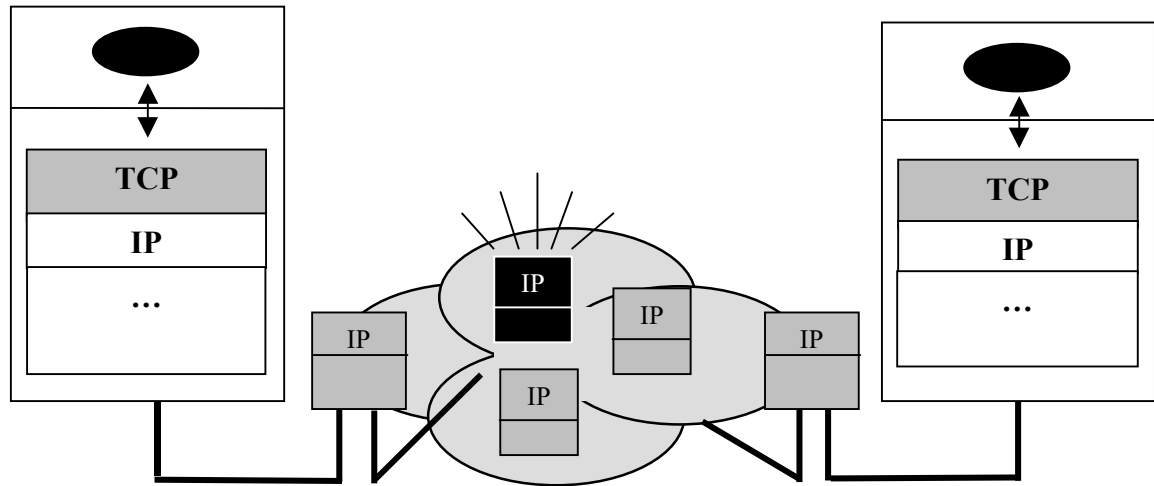
■ Position du problème

- Lorsqu'elle émet des segments TCP, l'E(TCP) émettrice ne doit pas envoyer plus d'octets que ne peut en stocker l'E(TCP) réceptrice
 - « Crédit » d'émission géré dans le cadre du contrôle de flux
- Problème
 - Même en n'envoyant pas plus de « crédit » octets, des pertes peuvent survenir par congestion des **routeurs intermédiaires** !



- Question qui se pose alors
 - Combien d'octets l'entité TCP émettrice peut elle envoyer consécutivement sans saturer les routeurs intermédiaires ??
 - Réponse
 - Pas évident mais voici comment faire au mieux ...

▪ Solution appliquée par TCP



– Repose sur la gestion par l'E(TCP) émettrice :

d'une « fenêtre de congestion » (FC)

- indiquant le nb max d'octets que l'E(TCP) émettrice peut émettre consécutivement sans recevoir d'acquittement
- avec à tout moment :

Obligation pour l'entité TCP émettrice de ne pas
envoyer plus de $\min \{FC, \text{Crédit}\}$ octets

– L'idée étant de faire augmenter progressivement la taille de FC

- tant qu'il n'y pas de congestion ...

⇔ mécanisme du SLOW START

(démarrage lent)

▪ Description de l'algorithme du slow start

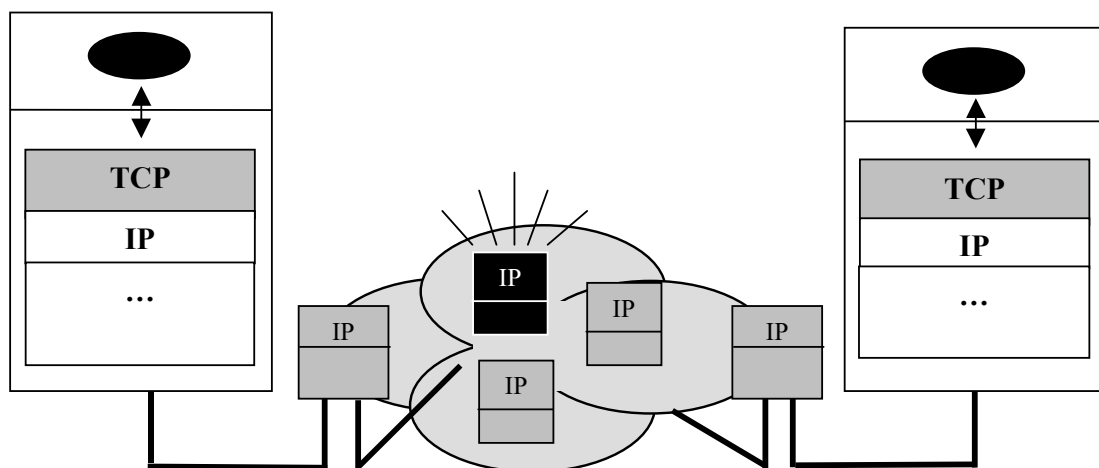
- **Hyp** : dans la suite, on considère que $\text{Crédit} > \text{FC}$ et que l'E(TCP) émettrice a toujours des données user à émettre
 - Initialement
 - $\text{FC} \leftarrow \text{Taille maximale d'un segment TCP } (T_{\max}S)$
 - Coté E(TCP) émetteur
 - Début des émissions en n'envoyant que le $\min \{\text{FC}, \text{Crédit}\}$
- ⇔ émission de 1 segment de taille $T_{\max}S$
- Sur réception de l'acquittement (ACK) de ce segment :
 - $\text{FC} \leftarrow \text{FC} * 2 (= 2 * T_{\max}S)$
- => Envoi de 2 segments TCP de taille $T_{\max}S$
- Sur réception des ACK de ces 2 segments :
 - $\text{FC} \leftarrow \text{FC} * 2 (= 4 * T_{\max}S)$
- => Envoi de 4 segments TCP de taille $T_{\max}S$
- ... et ainsi de suite
 - tant que FC est inférieur à une valeur seuil S de taille max $S_{\max}$
 - A partir du moment où $\text{FC} = S$ et tant que $\text{FC} < S_{\max}$
 - Sur réception des ACK des n segments de la fenêtre FC :
 - $\text{FC} \leftarrow \text{FC} + T_{\max}S$
- => Envoi de $n+1$ segments TCP de taille $T_{\max}S$
- etc. jusqu'à ce qu'une congestion intervienne ...

2. Le contrôle de congestion

■ Position du problème

- Lorsqu'un routeur reçoit des paquets IP à un débit supérieur à ses capacités de sortie :
 - remplissage de ses buffers et rejet des paquets entrants si les buffers sont pleins $\Leftrightarrow$ **congestion**
- Or, sur expiration du timer associé à un segment TCP perdu :
 - l'E(TCP) émettrice retransmet le segment

→ **contribue à aggraver la congestion !**



■ Solution appliquée par TCP

$\Leftrightarrow$ contrôler les congestions du réseau de BOUT EN BOUT

- en diminuant **FORTEMENT** la quantité d'octets émis par chaque source TCP en cas de congestion
 - afin que les routeurs congestionnés vident leurs buffers
- puis en ré augmentant progressivement cette quantité

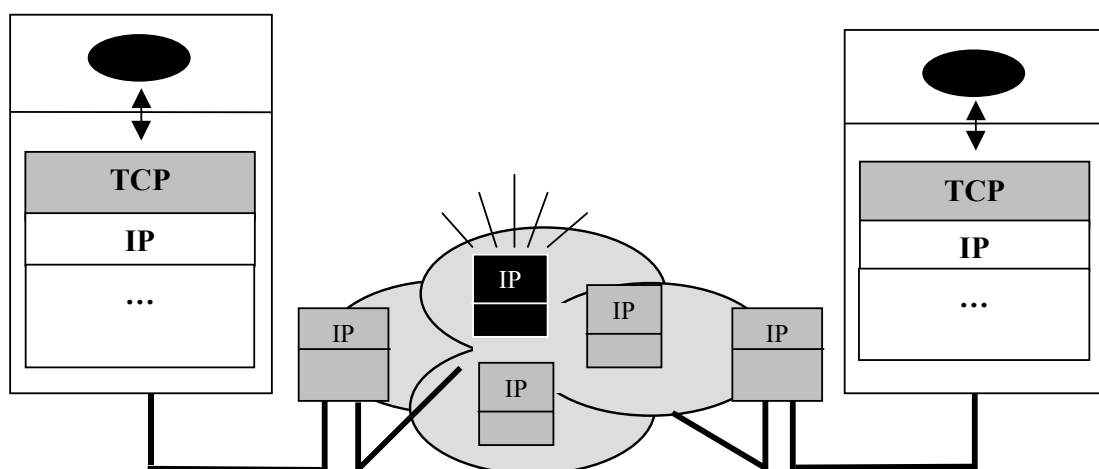
▪ Contrôle de congestion en 2 étapes

- 1] repérage d'une congestion
- 2] application de deux algorithmes :
 - l'évitement de (nouvelles) congestions : « **congestion avoidance** »
 - le (re)démarrage lent : « **slow start** »

1] Repérage d'une congestion

▪ Sur expiration d'un timer

- l'E(TCP) émettrice fait l'hypothèse que le segment a été perdu suite au rejet par un routeur congestionné du paquet IP l'encapsulant
 - Rmq : hyp cohérente car la perte d'un paquet IP est :
 - soit la conséquence d'erreurs(s) bit non corrigibles
 - soit le résultat d'une congestion → **cas le + probable**



2] Algorithme d'évitement de congestion

■ Principe de l'algorithme

- Sur détection d'une congestion
 - Evitement de nouvelles congestions (*congestion avoidance*)
 - 1] par **Réduction du seuil S** (le même que celui du slow start) :
 - $S \leftarrow FC / 2$
 - Puis 2] par Redémarrage lent des émissions (*slow start*)
 - $FC \leftarrow T_{\max} S$ = taille maximale d'un segment TCP
 - Avec chaque fois que toute la fenêtre FC a été acquittée :
 - si $FC < S$ alors : Accroissement exponentiel de FC
 - $FC \leftarrow FC * 2^{(1)}$
 - si $FC \geq S$ et $FC \leq S_{\max}$: Accroissement linéaire de FC
 - $FC \leftarrow FC + T_{\max} S$
 - **Rmq** : si $FC \geq S_{\max}$ alors FC n'évolue plus
 - jusqu'à ce qu'une congestion intervienne ...
 - dans ce cas : retour à l'étape 1] du contrôle de congestion

¹ En fait, pas tout à fait exact -> cf . algo page suivante

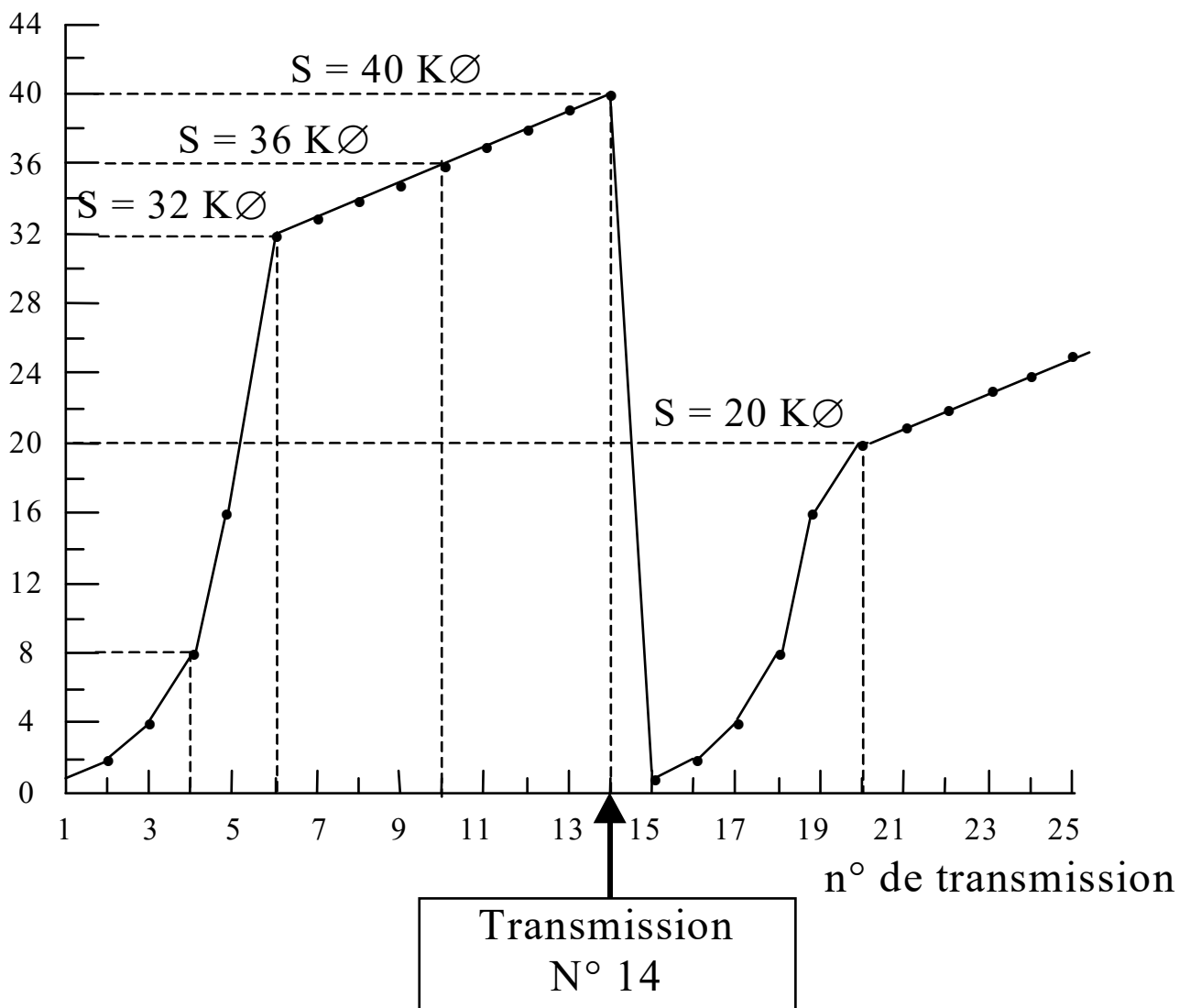
▪ Algo

- A l'initialisation de la connexion
 - $FC \leftarrow T_{\max}S$ = taille max d'un segment TCP
 - $S \leftarrow S_{\max}$ = taille max des données user d'un paquet IP (64 KØ)
- Tant qu'aucune congestion n'est détectée
 - Tant que $FC < S$:
 - à chaque acquittement **d'un segment TCP** (de taille $T_{\max}S$)
 - $FC \leftarrow FC + T_{\max}S$
 - Rmq : légèrement différent de :
 - $\left\{ \begin{array}{l} \text{– à chaque acquittement de toute la fenêtre FC :} \\ \quad \text{▪ } FC \leftarrow FC * 2 \end{array} \right.$
 - Lorsque $FC \geq S$:
 - à chaque acquittement **de toute la fenêtre FC** :
 - Si $FC < S_{\max}$ Alors :
 - $FC \leftarrow FC + T_{\max}S$
- Sur détection d'une congestion
 - $S \leftarrow FC / 2$
 - $FC \leftarrow T_{\max}S$

➤ Ex

- On se place après détection de la toute 1^{ère} congestion
 $\Rightarrow S = 32 \text{ K}\emptyset$ et $FC = 1$ segment de taille = $T_{\max}S$ (supposé = $1\text{K}\emptyset$)
- Une transmission correspond à l'émission de toute la fenêtre FC (par le biais de segments TCP de taille $T_{\max}S$)
- L'un des segments de la transmission n° 14 est supposé perdu
- Hyp : crédit d'émission (contrôle de flux) tjs supérieur à FC

FC (en multiple de $T_{\max}S$ et à l'issue de la transmission considérée)



■ Problème des flux TCP « unfriendly »

- Depuis quelques années : déploiement de nouvelles applications
 - audioconférence
 - vidéoconférence
 - plus généralement : applications « multimédias »



Débits importants

+

Contraintes temporelles incompatibles avec
les mécanismes de TCP

- Solution adoptée par les développeurs de ces applications :
 - utilisation d'UDP
 - **Rmq** : pas de garantie temporelle pour autant !
- **Problème** : pas de contrôle de congestion dans UDP !!

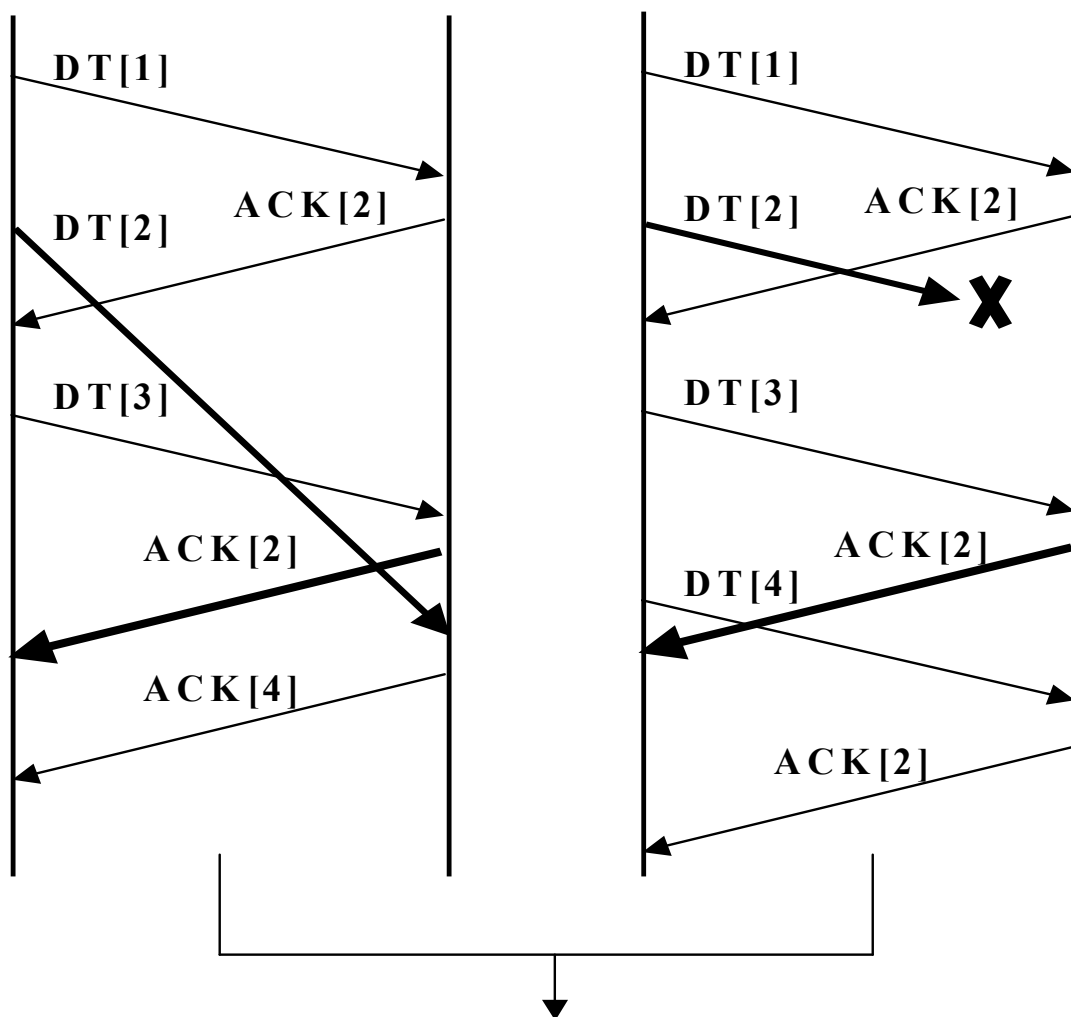


Si congestion du réseau : pénalisation des flux TCP
même si ces congestions sont dues
à des flux UDP !

♦ Autres mécanismes

1. Algorithmes de retransmission et de recouvrement rapides

■ Position du problème



2^{ème} ACK [2] lié à une perte ou à un déséquencelement ??

→ impossible de savoir pour TCP_A

– MAIS :

- Si réception d'un 3^{ème} ACK [2] (cf cas de droite) alors :

Forte chance qu'il s'agisse d'une perte !

■ Description des mécanismes

- Si réception par l'E(TCP) émettrice de 3 ACK dupliqués alors :
 - Hypothèse est faite côté TCP émetteur
 - qu'il y a eu **perte** de segment TCP (et non déséquencelement) !
 - ET :
 - « retransmission rapide » du segment perdu
 - i.e. avant l'expiration du timer de retransmission associé !
 - PUIS :
 - mise en œuvre du mécanisme de « **recouvrement rapide** »
 - composé :
 - de l'algorithme d'évitement de congestion :
 - $S \leftarrow FC / 2$
 - mais pas de celui du (re) démarrage lent car :
 - $FC \leftarrow S + 3 * T_{\max}S$
 - puis :
 - si un autre ACK dupliqué arrive : $FC \leftarrow FC + T_{\max}S$
 - si un nouvel ACK arrive : $FC \leftarrow S$

➤ Rmq

▪ Les différentes versions de TCP

- Tahoe [RFC 2581]
 - inclus les algorithmes de :
 - slow start, congestion avoidance, fast retransmit
- Reno [RFC 2581] et New Reno
 - inclus les algorithmes de :
 - slow start, congestion avoidance, fast retransmit et fast recovery
 - New Reno : extension de la version Reno qui considère la perte de plusieurs paquets consécutifs
 - version utilisée dans l'Internet actuellement
- SACK [RFC 2518]
 - introduit un mécanisme d'acquittement sélectif
 - version préconisée pour les années à venir
- Vegas
- ...

♦ La gestion du timer de retransmission

▪ Position du problème

- L'Internet génère des délais de transit variables
- **Problème :**

Evaluation du timer de retransmission
d'un segment ?!

▪ 2 méthodes [Van Jacobson 88]

- basées sur une évaluation dynamique du RTT
 - ⇔ estimation du temps aller-retour d'un segment
- 1ère méthode
 - A l'émission d'un segment : armement d'un timer T (t=0)
 - Sur réception de son acquittement :
 - arrêt de T (t = M) et **mise à jour du RTT :**
 - $RTT = a.RTT + (1-a).M$
 - où a = poids accordé à l'ancien RTT par rapport au nouveau
(généralement $a = 7/8$)
 - puis :
 - timer de retransmission $\leftarrow b.RTT$ où $b > 1$ (généralement 2)

- 2^{ème} méthode
 - A l'émission d'un segment : armement d'un timer T (t=0)
 - Sur réception de son acquittement :
 - arrêt de T (t = M) et **mise à jour du RTT**
 - $RTT = a.RTT + (1-a).M$
 - et :
 - évaluation de la déviation moyenne D du RTT nouveau par rapport au RTT ancien
 - $D \leftarrow a.D + (1-a).abs(RTT - M)$
 - puis :
 - timer de retransmission $\leftarrow RTT + 4.D$
 - Rmq : amélioration de l'algorithme [Karn]
 - pas de mise à jour du RTT à partir d'un segment retransmis
 - à la place : **$RTT \leftarrow 2.RTT$**
 - beaucoup d'implémentations à l'heure actuelle

♦ Algorithme de Nagle

■ Position du problème

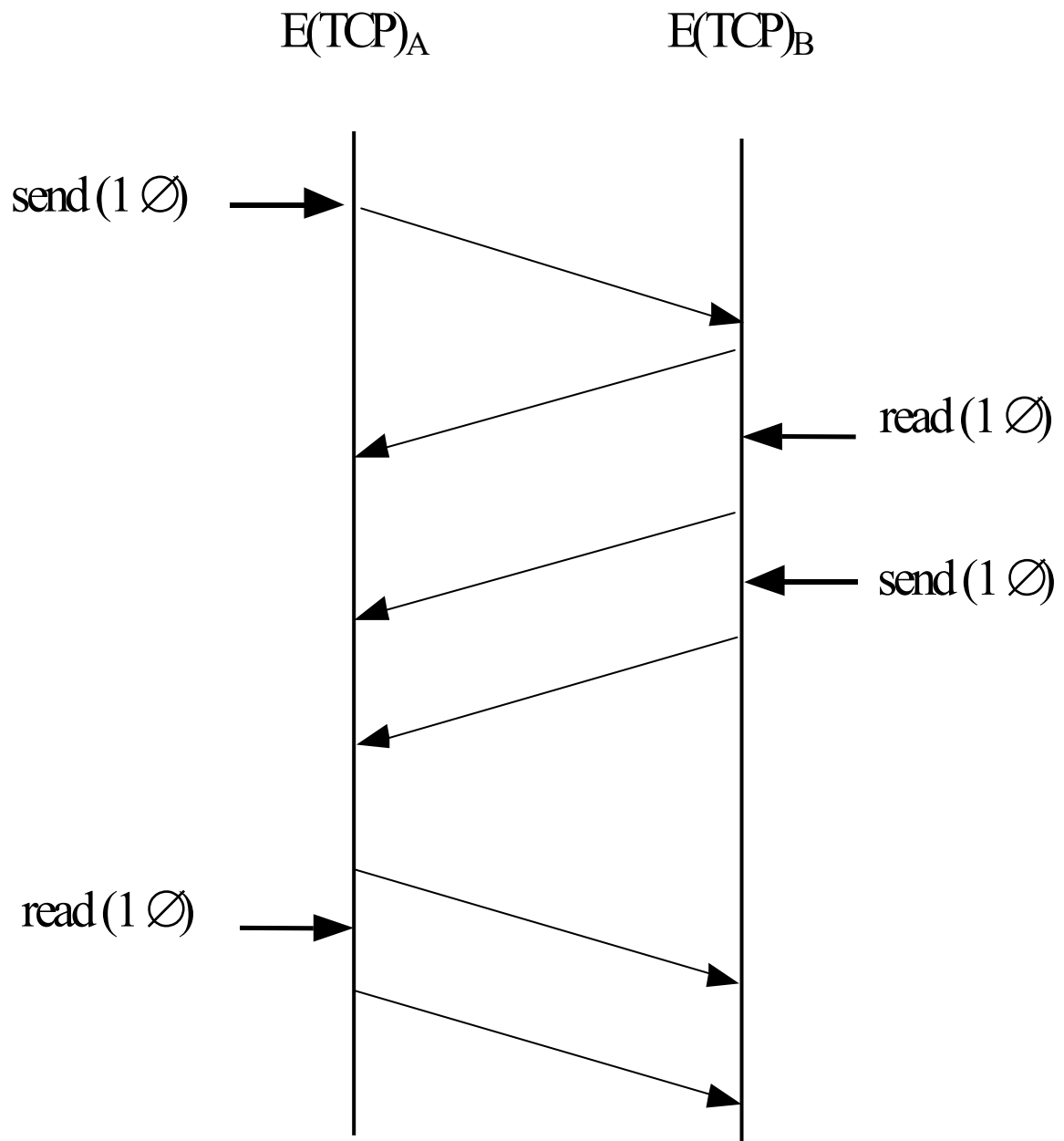
– Sur exécution d'un *send*(n octets) avec $n < \text{taille de la fenêtre d'émission FE}$ et $< \text{crédit d'émission}$, l'entité TCP locale peut :

- soit envoyer un segment de charge utile = n octets
 - Avantage
 - Emission immédiate de la donnée
 - Inconvénient
 - peu efficace en terme d'utilisation de BP si n est petit !
- soit attendre un ou plusieurs autres *send*()

puis envoyer un segment de charge utile + importante (toujours $\leq \text{taille FE}$ et $\leq \text{crédit}$)

- Avantage
 - moins d'overhead $\rightarrow$ meilleure utilisation de la BP
 - et surtout : moins de paquets IP dans le réseau !
- Inconvénient
 - mais temps d'attente avant émission des premiers n octets

- **Cas pire pour le réseau : application de terminal virtuel**



▪ **Solution = Algo de Nagle [Nagle 1984]**

- Règle initiale
 - pas plus de un segment TCP de 1 octet de charge utile
 - non encore acquitté et en transit dans le réseau
- En cas de *send* octet par octet
 - 1er octet envoyé
 - stockage des octets suivants et émission dans un seul segment :
 - si taille = x % de la taille max des segments ($x \leq 100$)
 - ou sur réception de l'ack du 1^{er} octet envoyé

➤ **Rmq**

- **∃ différentes implantations de l'algorithme de Nagle**
- **Pour bp d'implémentation de TCP**
 - sur réception d'un segment :
 - retard volontaire de l'émission de l'ACK si pas de données users à transmettre

♦ Algo de Clark (problème du « silly window control »)

■ Position du problème

– Hyp

- Le processus applicatif émetteur envoie de gros volumes de données (plusieurs Koctets)
- Le processus applicatif récepteur lit octet par octet (ou qq octets par qq octets)

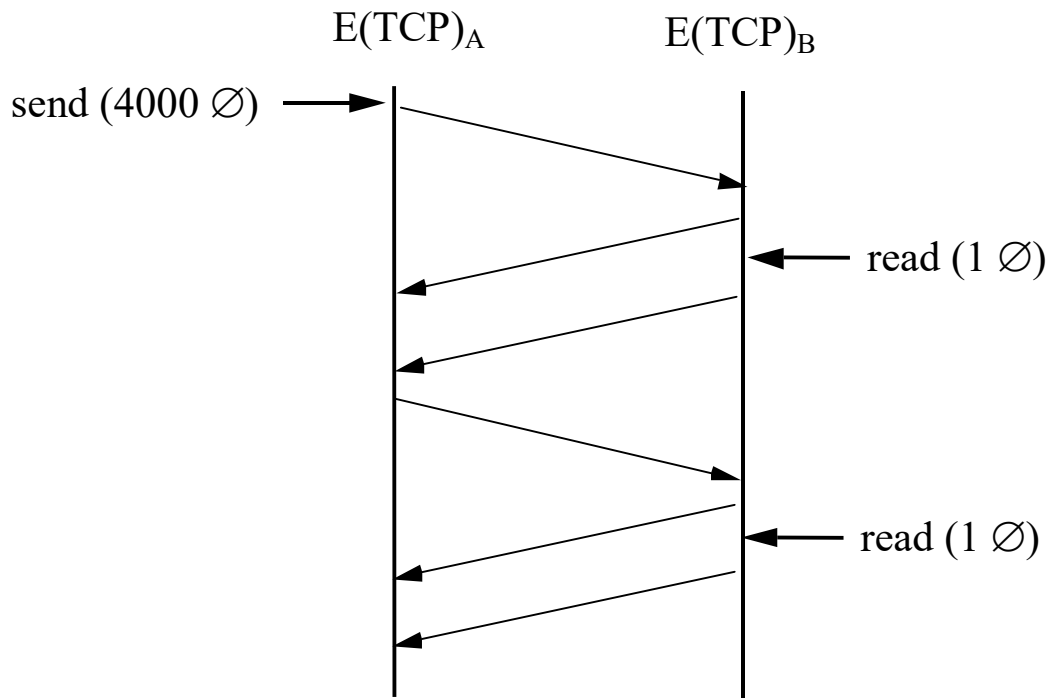
– Conséquences

- Fermeture « rapide » de la fenêtre d'émission FE de l'E(TCP) émettrice
- Génération d'un ACK par l'E(TCP) réceptrice à chaque lecture
- Emission d'un segment de 1 octet et fermeture de FE
- Génération d'un ACK par l'E(TCP) réceptrice à chaque lecture
- Emission d'un segment de 1 octet
- ...



Intuitivement peu efficace en terme
d'utilisation de la bande passante !

➤ **Ex**



▪ **Solution [Clark 82]**

– consiste à interdire à l' $E(TCP)$ réceptrice d'envoyer un ACK

si la mise à jour de sa fenêtre de réception

est « insignifiante »

- mais à générer cet ACK que lorsque la fenêtre de réception s'est (par ex) réouverte de moitié

– Rmq

- Algo de Nagle et de Clark sont complémentaires

3.4. Phase de libération de connexion

▪ Fermeture symétrique en deux temps

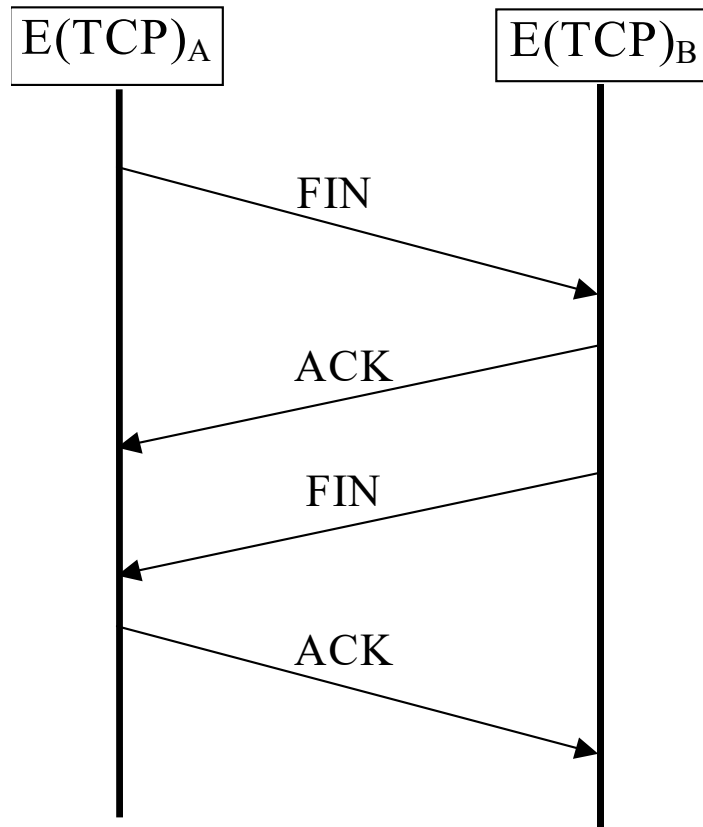
- fermeture du sens A vers B
- fermeture du sens B vers A

▪ Fermeture de la connexion dans le sens A vers B

- Sur exécution d'un *shutdown()* par le proc. s'exécutant sur A
 - ⇔ « je souhaite fermer la connexion dans le sens A vers B »
 - émission par $E(TCP)_A$ d'un segment $FIN = (b, a, FIN = 1, seqnum = x)$ contenant ou non des données user
 - Sur réception par $E(TCP)_B$ du segment FIN
 - fermeture du flux de données de A vers B
 - et émission d'un segment $ACK = (a, b, ACK = 1, seqnum = y, acknum = z)$ avec $z < x+1$
- Rmq
 - Sur exécution d'un *shutdown()* par le proc. s'exécutant sur B
 - émission par $E(TCP)_B$ d'un segment $FIN = (a, b, FIN = 1, seqnum = *)$ contenant ou non des données user
 - ⇔ « je souhaite fermer la connexion dans le sens B vers A »

➤ **Ex**

- Fermeture symétrique en 4 segments



➤ **Rmq**

- **Gestion d'un timer d'inactivité par TCP**
 - nécessaire en cas de pertes « infinies » des segments FIN
- **En cas de fermeture brutale (close() ou ^C) de la part de l'un ou l'autre des processus applicatif (par ex A)**
 - fermeture de la connexion de A vers B
 - puis fermeture immédiat de la connexion de B vers A